FH University of Applied Sciences
TECHNIKUM WIEN

Structured Programming Lab

Bachelor Program Business Informatics

19 Aug, 2024
Version 2.0.239

www.technikum-wien.at

Contents

HelloWorld **4**
The first program . . . 4
HelloWorld - Walkthrough . . . 6

Formatted Output **8**
Printing Functions . . . 8
Special Characters . . . 8
Formatting Integer Values . . . 9
Formatting Strings . . . 10
Formatting Floating Point Values . . . 11
Formatted Output - Summary . . . 12

Calculations **14**
Integer Arithmetic . . . 14
Double Arithmetic . . . 15
Mixed Arithmetic Expressions . . . 16
Types . . . 16
Variables . . . 19
Identifiers . . . 20
Java Naming convention . . . 20
Expressions . . . 21
Literals . . . 21
Literals - Overview . . . 21
Operators and Expressions . . . 23

Formatted Input **27**
Reading Integer Values . . . 27
Reading Double Values . . . 28
Reading Strings . . . 28
Reading Strings - MiniExercise . . . 29

Selection Statements **31**
Conditional Statement . . . 31
Single Alternative . . . 35
Consecutive ifs . . . 38
Nested Ifs . . . 41
Multiple Alternative . . . 43
Switch/Case . . . 44
Relational and Conditional Operators . . . 45

Iteration Statements **47**
Iteration Introduction . . . 47
`while` statement . . . 48
`for` statement . . . 52
`do` statement . . . 55
Special Cases . . . 58
Jump commands . . . 59
Iteration summary . . . 59

Strings **61**
Introduction . . . 61
String Operations . . . 61
Characters . . . 62

SimpleFunctions **64**
Introduction . . . 64
Examples . . . 65
Function parameters and variables . . . 66
Function overloading . . . 66
Summary . . . 67

SimpleRecursion **68**
Introduction . . . 68
Recursion in programming . . . 68
Recursive Fibonacci . . . 70
Towers of Hanoi . . . 71

Arrays **76**
Introduction . . . 76
Theory . . . 78
Summary . . . 79

ArraysandFunctions **80**
Arrays as function parameters . . . 80
Arrays as return type . . . 81
For-each . . . 83

MultidimensionalArrays **84**
2D Arrays . . . 84
Multidimensional Arrays . . . 86

Sort **87**
Introduction . . . 87
BubbleSort . . . 87
InsertionSort . . . 90
SelectionSort . . . 92
MergeSort . . . 94
QuickSort . . . 96

Search **100**
Linear/Binary search . . . 100
Linear search . . . 101
Binary search . . . 101
Big-O Notation . . . 103

CompoundDatatypes **104**
Motivation . . . 104
Simple Classes . . . 104
Classes and references . . . 106
Classes and equality . . . 107
Classes and functions . . . 108
Classes as attributes . . . 109
Classes and initialization . . . 111
Classes and Arrays . . . 111

Stack and Queue **113**
Stack . . . 113
Queue . . . 114
Implementation . . . 115
ArrayStack . . . 116
ArrayQueue . . . 119

Graphs **120**
Motivation . . . 120
Graph Terms . . . 122
Data structures for graphs . . . 123
Dijkstra's Algorithm . . . 124
Implementation . . . 127
More graph problems . . . 129
Even more graph problems . . . 130
Rendering graphs . . . 130

HelloWorld

The first program

Introduction

- the first program we implement is the HelloWord program.
- this very simple program prints “Hello World!”
- in the code example below we see the basic structure of every Java Programm:
  - a class `HelloWorld` that contains the `main()` function
  - when starting a programm the statements in the `main()` function are executed.

HelloWorld - Code

```
1  package HelloWorld;
2
3  /*
4      my first program
5  */
6
7  public class HelloWorld {
8
9      public static void main(String[] args) {
10         System.out.println("Hello World!"); //a print statement
11     }
12
13 }
```

HelloWorld - Output

```
Hello World!
```

HelloWorld - Flowchart

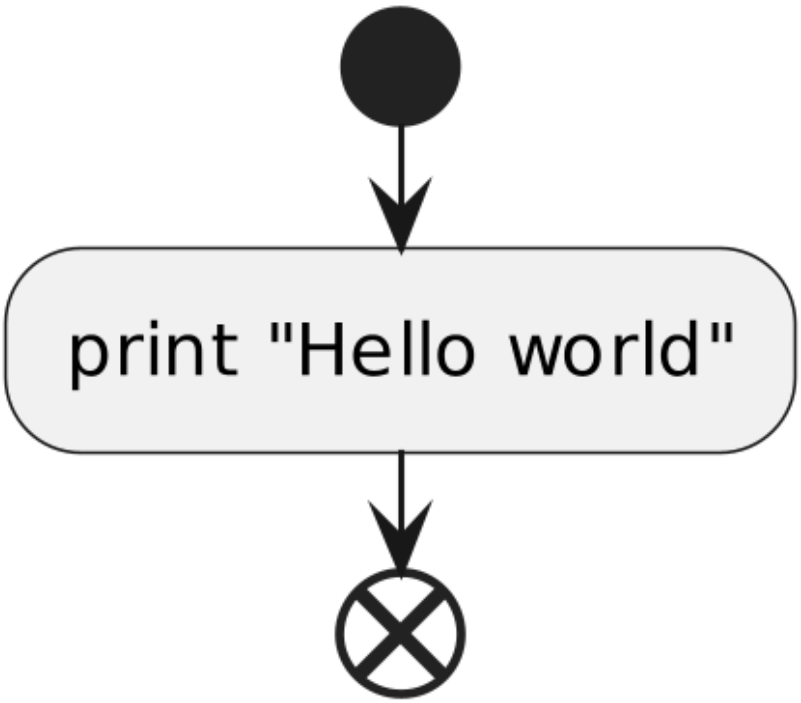


Figure 1: HelloWorld Flow Chart. Note the shapes of the start, end, and output nodes.

HelloWorld - Annotated Flowchart

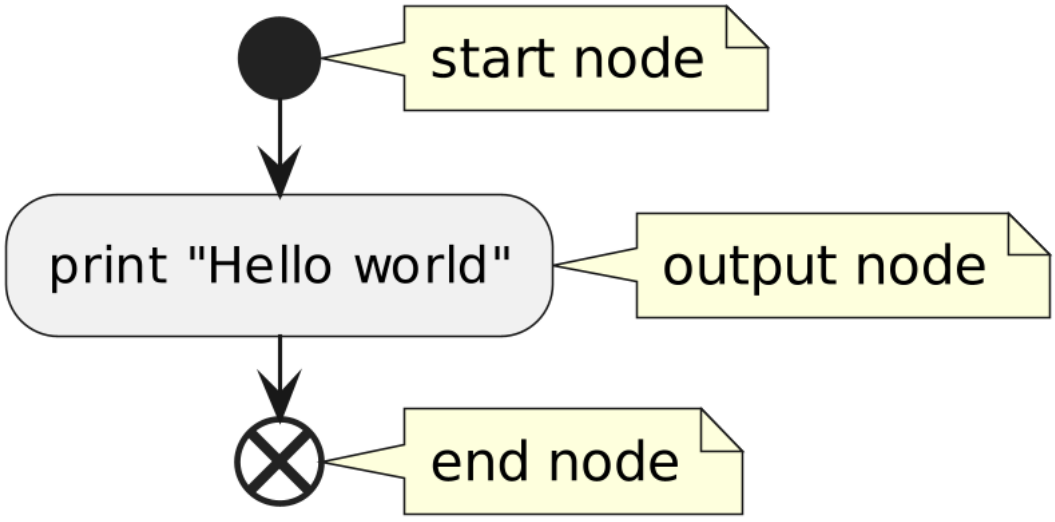


Figure 2: HelloWorld Annotated Flow Chart.

HelloWorld - Walkthrough

Package Declaration

```
1  package HelloWorld;
```

- Java programs are organized in **packages**
- a package is similar to a directory
- in our example, the **class** `HelloWorld` is in the **package** `HelloWorld`
- generally, package and class names are not the same

Comment

```
3  /*
4      my first program
5  */
```

- a **comment** used for documentation for the human reader
- comments start with `/*` and end with `*/`
- everything in between is the comment
- comments are not **compiled** or **executed**, they are meant for documentation

Class Declaration

```
7  public class HelloWorld
```

- name of the class/program: “HelloWorld”

Content of Class

```
7                          {
8      public static void main(String[] args) {
9          System.out.println("Hello World!"); //a print statement
10     }
11
12 }
```

- this **class** contains one single **function**, the `main` function

Main Function

```
9      public static void main(String[] args) {
10         System.out.println("Hello World!"); //a print statement
11     }
```

- **execution** of the program starts in `main` (in line 10)

Content of the `main` Function

```
9                                           {
10          System.out.println("Hello World!"); //a print statement
11      }
```

- the `main` function has one **statement**
- in `main` we **call** the `println` **function**
- we **pass** the text to the `println` function as a **string** in parenthesis
- statement ends with a semi-colon `;`

Line end comment

```
9                                           {
10                                          //a print statement
11      }
```

- starts with two forward slashes `//`
- the rest of the line is ignored
- documentation for the human reader, is not compiled nor executed

String

```
9                                           {
10                          "Hello World!"
11      }
```

- the **string** (of characters) we want to print
- strings start and end with **double quotes** `"`
- the string is **delimited** by double quotes

Formatted Output

Printing Functions

PrintDemo

```java
public class PrintDemo {
    public static void main(String[] args) {
        System.out.println("Hello");
        System.out.print("World!");
        System.out.println();
        System.out.printf("Goodbye World!");
    }
}
```

OutputFunctionsDemo - Output

```
Hello
World!
Goodbye World!
```

OutputFunctionsDemo - Summary

Table 1: Output functions

output function	description
`print`	prints the argument (of any type)
`println`	prints the argument and starts a new line
`printf`	formatted output - use format strings

Special Characters

SpecialCharacterDemo

```java
package FormattedOutput;

public class SpecialCharacterDemo {

    public static void main(String[] args) {
        System.out.printf("Eins");
        System.out.printf("\nZwei");
        System.out.printf("\nDrei\nVier\nFünf");
        System.out.printf("\n\\ \"");
        System.out.printf("\n\"\\\\ \\\" \"");
        System.out.printf("\n%%");
    }
}
```

SpecialCharacterDemo - Ouput

```
Eins
Zwei
Drei
Vier
Fünf
\ "
"\\ \" "
%
```

Special Characters - Summary

Most special characters are **escaped** with a backslash (`\`).

Table 2: Escape sequences for special characters

escape sequence	character description	character appearance (if printable)
`\\`	backslash	`\`
`\"`	double quote	`"`
`\n`	new line	
`\t`	tabulator	
`%%`	percent sign[1]	`%`

Formatting Integer Values

IntegerFormattingDemo

```
1  public class IntegerFormattingDemo {
2
3      public static void main(String[] args) {
4          System.out.printf("Freitag, 15.09.2017\n");
5          System.out.printf("Freitag, %d.09.2017\n", 15);
6          System.out.printf("Freitag, %d.%d.%d\n", 15, 9, 2017);
7          System.out.printf("Freitag, %d.%2d.%d\n", 15, 9, 2017);
8          System.out.printf("Freitag, %d.%02d.%d\n", 15, 9, 2017);
9          System.out.printf("Freitag, %d.%-2d.%d\n", 15, 9, 2017);
10         System.out.printf("Freitag, %d.%+2d.%d\n", 15, 9, 2017);
11         System.out.printf("Freitag, %d.%+-8d.%d\n", 15, -9, 2017);
12
13         System.out.printf("Freitag, %02d.%02d.%4d\n\n", 15, 9, 2017);
14     }
15 }
```

IntegerFormattingDemo - Output

[1] in `printf`

```
Freitag, 15.09.2017
Freitag, 15.09.2017
Freitag, 15.9.2017
Freitag, 15. 9.2017
Freitag, 15.09.2017
Freitag, 15.9 .2017
Freitag, 15.+9.2017
Freitag, 15.-9      .2017
Freitag, 15.09.2017
```

Formatting Integer - Summary

- with the `printf()` function we can define formats with **format strings**
- declare placeholders for integer values with `%d`
- define a (minimum) **width** for a value by adding a number, e.g. `%6d` (print an integer number in decimal format with width 6
- if the value needs less characters we have several options
  - `%6d` leading spaces
  - `%-6d` left aligned
  - `%06d` leading zeros
- to print the sign use `%+d`
- (not all) combinations are possible
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance

Table 3: Integer formatting examples

format string	description	example output[2] (for value 123)
`%5d`	minimum width 5	`|  123|`
`%05d`	leading zeros	`|00123|`
`%+5d`	print sign	`| +123|`
`%-5d`	left aligned	`|123  |`
`%x`	in hexadecimal	`|  0x7b|`
`%X`	in hexadecimal	`|  0x7B|`

Formatting Strings

StringFormattingDemo

```
1  public class StringFormattingDemo {
2      public static void main(String[] args) {
3
4          System.out.printf("|%s|\n", "Freitag");
5          System.out.printf("|%10s|\n", "Freitag");
6          System.out.printf("|%-10s|\n", "Freitag");
7          System.out.printf("|%S|\n", "Freitag");
8          System.out.printf("|%2.3S|\n", "Freitag");
```

```
9
10      }
11  }
```

StringFormattingDemo - Output

```
|Freitag|
|   Freitag|
|Freitag   |
|FREITAG|
|FRE|
```

StringFormattingDemo - Summary

- declare placeholders for strings with `%s`
- define a (minimum) **width** for a string by adding a number, e.g. `%6s` (print a string with minimum width 6)
- if the string has less characters we can left align with `-` , e.g. `%-6s`
- define a maximum width for a string by adding a **precision**, e.g. `%.6s` (print a string with maximum width 6), longer strings are truncated
- combine width and precision, e.g. `%6.6s` (print a string exactly 6 characters wide)
- use `%S` to convert to upper case
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance
- (not all) combinations are possible

Table 4: String formating options

format string	description	example output[3] (for `lorem ipsum`)
`%15s`	minimum width 15	`|    lorem ipsum|`
`%-15s`	left aligned	`|lorem ipsum    |`
`%10s`	minimum width 10	`|lorem ipsum|`
`%S`	capital letters	`|LOREM IPSUM|`
`%.3s`	maximum width 3	`|lor|`
`%5.5s`	exactly 5 wide	`|lorem|`

Formatting Floating Point Values

FloatingPointFormattingDemo

```
1  package FormattedOutput;
2
3  public class FloatingPointFormattingDemo {
4
5      public static void main(String[] args) {
6
7          System.out.printf("(19.2°C)\n");
```

```
8           System.out.printf("(%f°C)\n", 19.2);
9           System.out.printf("(%.2f°C)\n", 19.2);
10          System.out.printf("(%10.2f°C)\n", 19.2);
11          System.out.printf("(%+010.2f°C)\n", 19.2);
12      }
13  }
```

FloatingPointFormattingDemo - Output

```
(19.2°C)
(19.200000°C)
(19.20°C)
(     19.20°C)
(+000019.20°C)
```

FloatingPointFormattingDemo - Summary

- declare placeholders for floating point values with `%f`
- define a (minimum) **width** for a floating point value by adding a number, e.g. `%6f` (print a floating point value with minimum width 6)
- if the value needs less characters we can left align with `-` , e.g. `%-6f`
- if the value needs less characters we can add leading zeros with `0` , e.g. `%06f`
- define a number of decimal places by adding a **precision**, e.g. `%.6f` (print a floating point value with 6 decimal places), values are rounded
- combine width and precision, e.g. `%10.6f` (print a floating point value with minimum 10 characters, 6 of which are decimal places)
- we can have several placeholders in a format string
- values are mapped to placeholders by sequence of appearance
- (not all) combinations are possible

Table 5: Floating point values formatting examples

format string	description	example output[4] (value: 19.2)
`%.2f`	2 decimal places	`|19.20 |`
`%6.2f`	minimum width 6	`| 19.20|`
`%06.2f`	leading zeros	`|019.20|`
`%+6.2f`	print sign	`|+19.20|`

Formatted Output - Summary

Format Sepcifications

`printf` Syntax:

```
System.out.printf(FORMAT_STRING, ARGUMENTS)
```

- “place holders” in format string
- place holder starts with `%`

- first place holder for first argument
- second place holder for second argument
- n-th place holder for n-th argument
- arguments are comma-separated

Format string Syntax:

```
%[argument_index$][flags][width][.precision]conversion
```

Conversions

Table 6: Format string conversions

conversion	explanation
`%d`	integer values in **d**ecimal format
`%f`	double (and **f**loat) values
`%s`	**S**trings
`%x`	integer values in he**x**adecimal format
`%n`	platform-specific line separator
`%%`	a percent sign (`%`)

Width and Precision

- **minimum** width
- precision
  - **maximum** number of characters when used with `%s`
  - **exact** number of character after decimal separator when used with `%f`

Flags

Table 7: Format strings flags

Flag	Description
`-`	left justified
`0`	zero padded (numbers)
(space)	space padded (numbers)
`+`	always show sign (numbers)

For more (and more detailed information) see the Java documentation.

Calculations

Integer Arithmetic

IntegerArithmeticDemo Code

```
1  public class IntegerArithmeticDemo {
2
3      public static void main(String[] args) {
4          System.out.printf("%d + %d = %d\n", 19, 4, 19 + 4);
5          System.out.printf("%d - %d = %d\n", 19, 4, 19 - 4);
6          System.out.printf("%d * %d = %d\n", 19, 4, 19 * 4);
7          System.out.printf("%d / %d = %d\n", 19, 4, 19 / 4);
8          System.out.printf("%d %% %d = %d\n", 19, 4, 19 % 4);
9          //System.out.printf("%d / %d = %d\n", 19, 0, 19 / 0);
10
11         System.out.printf("%d\n", Integer.MAX_VALUE);
12         System.out.printf("%d\n", Integer.MIN_VALUE);
13         System.out.printf("%d\n", Integer.MAX_VALUE + 1);
14         System.out.printf("%d\n", 19 + 2 * 3);
15         System.out.printf("%d\n", (19 + 2) * 3);
16     }
17 }
```

IntegerArithmeticDemo Output

```
19 + 4 = 23
19 - 4 = 15
19 * 4 = 76
19 / 4 = 4
19 % 4 = 3
2147483647
-2147483648
-2147483648
25
63
```

IntegerArithmetic Summary

- integer division returns integer result
- modulo (`%`) is the remainder after division
- division by zero produces an error (an exception)

```
Exception in thread "main" java.lang.ArithmeticException: / by zero
 at Expressions.IntegerArithmeticDemo.main(IntegerArithmeticDemo.java:14
```

- minimum and maximum `int` values (`Integer.MIN_VALUE` and `Integer.MAX_VALUE`)
- over/underflow error (see Two's Complement)
- multiplication and division are evaluated before addition/subtraction in longer expressions
- Use parenthesis to control evaluation sequence

Double Arithmetic

DoubleArithmeticDemo Code

```
1  public class DoubleArithmeticDemo {
2
3      public static void main(String[] args) {
4          System.out.printf("%f + %f = %f\n", 19., 4., 19. + 4.);
5          System.out.printf("%f - %f = %f\n", 19., 4., 19. - 4.);
6          System.out.printf("%f * %f = %f\n", 19., 4., 19. * 4.);
7          System.out.printf("%f / %f = %f\n", 19D, 4D, 19D / 4D);
8
9          System.out.printf("%g\n", Double.MAX_VALUE);
10         System.out.printf("%g\n", Double.MIN_VALUE);
11
12         System.out.printf("%g\n", Double.MAX_VALUE - 1);
13
14         System.out.printf("%f / %f = %f\n", 19., 0., 19. / 0.);
15         System.out.printf("%f / %f = %f\n", 0., 0., 0. / 0.);
16
17         System.out.printf("%f\n", Double.POSITIVE_INFINITY * 42);
18         System.out.printf("%f\n", Double.NaN * 42);
19         System.out.printf("%f\n", Double.POSITIVE_INFINITY * Double.NaN);
20     }
21 }
```

DoubleArithmeticDemo Output

```
19.000000 + 4.000000 = 23.000000
19.000000 - 4.000000 = 15.000000
19.000000 * 4.000000 = 76.000000
19.000000 / 4.000000 = 4.750000
1.79769e+308
4.90000e-324
1.79769e+308
19.000000 / 0.000000 = Infinity
0.000000 / 0.000000 = NaN
Infinity
NaN
NaN
```

DoubleArithmetic Summary

- use a dot (`.`) or a `D` to generate a double value
- use %g to format large `double` values
- adding/subtracting (rather) small values to/from very large values does not necessarily give new values
- Division by zero gives `Infinity`
- most calculations involving `Infinity` give `Infinity` (except when combined with `NaN`)
- `0./0.` gives “Not a number” (`NaN`)
- any calculation involving `NaN` gives `NaN`
- numbers are internally represented by **mantissa** and **exponent**
- mantissa saves (the first) 12-14 digits

- exponent saves “how many places to shift the decimal separator”
- double values have a certain **precision**, calculation results are generally not exact

Mixed Arithmetic Expressions

MixedArithmeticDemo Code

```
1  public class MixedArithmeticDemo {
2
3      public static void main(String[] args) {
4          System.out.printf("%f%n", 1. + 3 /4 );
5          System.out.printf("%f%n", 1  + 3./4 );
6          System.out.printf("%f%n", 1  + 3 /4.);
7          System.out.printf("%f%n", 1  + (double) 3 /4);
8          System.out.printf("%d%n", (int) (1  +  3. /4));
9      }
10 }
```

MixedArithmeticDemo Console

```
1.000000
1.750000
1.750000
```

Mixed Expressions Summary

- if one of the operands is `double`, the other is converted to `double` **before** the operation
- convert to a different type with the *type cast operator**
  - `(int)` convert to `int` (information loss!)
  - `(double)` convert to `double`
- type casting should be the exception, *not* the rule

Types

Primitive Types in Java

There are 8 primitive datatypes in Java:

- `byte`
- `short`
- `int`
- `long`
- `float`
- `double`
- `char`
- `boolean`

Types differ in size (=memory usage) and range.

Primitive Types Table

Table 8: Java primitive types

type	size [B]	range	use
`byte`	1	-128 to 127	very small integer number
`short`	2	-32768 to 32767	small integer number
`int`	4	-2147483648 to 2147483647	integer number
`long`	8	-9223372036854775808 to 9223372036854775807	large integer number
`float`	4	(1.4E-45 to 3.4028235E38)	floating point number
`double`	8	4.9E-324 to 1.7976931348623157E308	higher precision floating point number
`char`	2	0 to 65535	single character
`boolean`	depends	true/false	boolean (logical) value

For further details visit The Java™ Tutorials on data types.

Binary Numbers

- only has two symbols: 0 and 1
- each digits is referred to as a bit

Table 9: binary numbers

binary	decimal
0001	1
0010	2
0011	3
0100	4
0101	5
0110	6
0111	7
1000	8
1111	15

Hexadecimal Numbers

- has 16 symbols: digits from 0 to 9 plus letters A to F
- one hexadecimal symbol can be represented with 4 bits

Table 10: hexadecimal numbers

hexadecimal	decimal	binary
01	1	0000 0001
09	9	0000 1001
0A	10	0000 1010
0B	11	0000 1011

hexadecimal	decimal	binary
0F	15	0000 1111
10	16	0001 0000
11	17	0001 0001
1A	26	0001 1010
1F	31	0001 1111
FF	255	1111 1111

Two's Complement

- is used to represent signed integers
- as in the binary system each bit is a power of two
- only the most significant bit is the negative power of two
- see example table for three bit signed integers

Table 11: three bit signed integer

decimal	binary	decimal conversion
0	000	$-0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
1	001	$-0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
2	010	$-0 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
3	011	$-0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$
-4	100	$-1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$
-3	101	$-1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$
-2	110	$-1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0$
-1	111	$-1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$

Strings

- Strings are not primitive types
- represent a "String of characters"
- sequence of 0 or more characters
- delimited by double quotes (`'"'`)
- empty String `""`
- `"a string of characters"`
- `"42"`

MiniExercise

Which type would you chose for

- a name
- an amount of money
- a count (e.g. the number of visitors)
- an ID
- a color
- a calendar date
- a time of the day
- a day of the week (one of MON-SUN)

- an estimate of the number of protons in the universe
- the mass of the earth

Variables

Variables

- are objects in memory
- have
  - type
  - value
  - name
  - scope
  - physical location in memory
- must be declared before use
- must be initialized before first use of their value
- can appear on the left side of an assignment (L-value)
- variable name stands for current value of the variable (except when used as L-value)
- can change value through assignment
- variable names must be valid identifiers

Variable declaration

```
TYPE NAME;
```

```
int a; //declared, but not initialized
int b = 4; //declared and initialized
int c, d, e;
int f = 4, g, h = b, i = b + 4;

//a, c, d, e, g are still not initialized
```

(Re-) assignment

```
int a =42;
//...
a = 23; //a new value is stored in a
```

Scope of a variable

- The scope of a variable describes the sections of code in which the variable is *known*.
- A variable is *in scope* in the block within which it is declared.

```
{
    int a = 23;
        { //open a new scope
            int b = 42;
```

```
            //a and b are in scope
        }
    //b is no longer in scope
}
```

Identifiers

Overview

- Identifiers are “names” we chose for things like variables, functions and types.
- An identifier can consist of
  - letters [`A` - `Za` - `z`]
  - digits [`0` - `9`]
  - underscores [`_`]
- An identifier cannot start with a digit.
- identifiers are case sensitive

Reserved keywords

The following Java reserved keywords cannot be used as identifiers:

```
abstract    continue    for         new         switch
assert      default     goto        package     synchronized
boolean     do          if          private     this
break       double      implements  protected   throw
byte        else        import      public      throws
case        enum        instanceof  return      transient
catch       extends     int         short       try
char        final       interface   static      void
class       finally     long        strictfp    volatile
const       float       native      super       while
```

Java Naming convention

Overview

- use CamelCase
- choose short, descriptive names
- variable names
  - start with a lower case letter
  - describe the value the variable holds
  - are often nouns
- function names
  - start with a lower case letter
  - are often verbs (or contain verbs)
- class names
  - start in upper case
  - are often nouns

- boolean variables and/or functions
  - often start with “is” or “has”
  - should make sense (i.e. almost sound English) after an “if” or “while” (e.g. “if isEmpy” or “if hasNext”)
- constants are named in all upper case separated by underscores
  - `DAYS_PER_WEEK`
  - `SECONDS_PER_MINUTE`

Naming tips

Table 12: Variable Naming

name	comment
`n`	good for integers, when meaning is clear from context
`i`	often used as loop counter
`age`	good, clear and short. probably numeric value
`name`	good, clear and short. probably a String
`state`	state of a system. good, clear and short. probably an `int`
`dateOfBirth`	good, clear and short and camelCase
`isValid`	good, clear and short. probably a boolean.
`variable`	we know it’s a variable. Describe the value!
`johnny`	bad. meaning unclear.
`value`	bad. meaning unclear.
`test003_4_secondTry_DoesNotWork`	getting frustrated?
`theFirstLetterOfTheLastName`	probably too long.

Expressions

Expressions - Overview

- An expression describes the calculation of a **value**.
- An expression (`EXPR`) can be
  - a literal
  - a variable
  - `EXPR BINARY_OPERATOR EXPR`
  - `UNARY_OPERATOR EXPR`
  - a function call

Literals

Literals - Overview

- literals represent a specific value
- we have used literals from the very beginning on
- depending on type there are often several ways of representing values with literals

```
System.out.printf("%d", 42); //42 is an int literal
System.out.printf("%d", 0x2A); //0x2A is an int literal in hex
System.out.printf("%f", 3.14159); //3.14159 is a double literal
System.out.printf("%s", "hello"); //"hello" is a String literal
```

`int` literals (examples)

decimal

- `0`
- `-42`
- `255`

hexadecimal

- `0x0`
- `0x2A` or `0x2A`
- `0xff` or `0xFF`

oktal

- `0`
- `052`
- `0377`

`long` literals (examples)

- `0L`
- `42L`
- `999_999_999L`

`double` literals (examples)

- `3.14159`
- `42.0`
- `42.`
- `42D`
- `0.42E2`
- `0.42e2`

`float` literals (examples)

- `42.0F` or `42.0f`
- `42.F` or `42.f`
- `42F` or `42F`
- `0.42E2f` or- `0.42e2F`

`char` literals (examples)

- `'H'`
- `'\n'`
- `'\u03C0'` (unicode escape sequence)

`boolean` literals (examples)

`boolean` :

- `true`
- `false`

Operators and Expressions

Operators - Overview

- An operator computes a value from one or more values, called *operands*.
- Properties of Operators
  - Arity
  - Precedence
  - Associativity
- Operators in Java
  - Arithmetic
  - Relational
  - Conditional
  - Assignment
  - Bitwise operators
  - Ternary

Arity

- The number of operands an operator takes

Table 13: Arity of operators

number of operands	Arity
1	unary
2	binary
3	ternary
n	n-ary

Precedence

- Determines the order of evaluation of expressions with several operators
- Operators with higher precedence are evaluated before operators with lower precedence

Precedence Table

Table 14: Java Operator Precedence (highest at the top) and Associativity

Precedence Group	Operators	Associativity
postfix	`expr++ expr--`	left
unary	`++expr --expr +expr -expr ~ !`	right
multiplicative	`* / %`	left
additive	`+ -`	left
shift	`<< >> >>>`	left
relational	`< > <= >= instanceof`	left
equality	`== !=`	left
bitwise AND	`&`	left
bitwise exclusive OR	`^`	left
bitwise inclusive OR	`|`	left
logical AND	`&&`	left
logical OR	`||`	left
ternary	`? :`	left
assignment	`= += -= *= /= %= &= ^= |= <<= >>= >>>=`	right

Associativity

- Determines how operators of the same precedence are grouped
- possible associativity values for binary operators are
  - left-associative
  - right-associative

```
//left-associative OPERATORS
EXPR1 OPERATOR_1 EXPR2 OPERATOR_2 EXPR3
//same as ((EXPR1 OPERATOR_1 EXPR2) OPERATOR_2 EXPR3)
```

```
//right-associative OPERATORS
EXPR1 OPERATOR_1 EXPR2 OPERATOR_2 EXPR3
//same as (EXPR1 OPERATOR_1 (EXPR2 OPERATOR_2 EXPR3))
```

Arithmetic Operators

- Arithmetic Operators operate on two (numeric) expressions and return a numeric value.
- binary
- left-associative (`k - i - j;` same as `(k - i) - j;`)

Table 15: Arithmetic Operators

Symbol	Operation
`+`	Addition
`-`	Subtraction
`*`	Multiplication
`/`	Division
`%`	Modulo

Mixed Arithmetic Expressions Rules

- The first rule in the sequence that applies, is carried out.
  - If either operand is of type `double`, the other is converted to `double` before the operation is carried out
  - If either operand is of type `float` the other is converted to `float` before the operation is carried out
  - If either operand is of type `long`, the other is converted to `long` before the operation is carried out
- If neither operand is `double`, `float`, or `long`, they must be `int`, `short`, or `byte`.
- Result of arithmetic expression is at least of type `int`

Simple Assignment Operator

- Assignment (`=`) operator stores a value in a variable
- Usage

```
L_VALUE = EXPR;
```

- The left hand side (`L_VALUE`) must be an l-value (a variable)
- The right hand side (`EXPR`) is evaluated first and the result copied to the left hand side
- Types of `L_VALUE` and `EXPR` must be compatible
- Right-associative
- returns value of `EXPR`

```
i = 42; //simple assignment
i = k * 42;

i = j = k = 0; //same as i = (j = (k = 0));
```

Type Cast Operator

- explicitely converts the type of an expression
- beware: data might be lost
- usage

```
(TYPE_NAME) EXPR
```

- example

```
(int) 3.14159 //"cast" a double value to int
```

Compound Assignment Operators

- Update a variable
- Usage

```
L_VALUE op= EXPR
// same as L_VALUE = (type of L_VALUE) (LVALUE op EXPR)
```

- beware: casts to type of `L_VALUE`
- Operators: `*=` , `/=` , `+=` , `-=` , `%=` , `&=` , `^=` , `|=` , `<<=` , `>>=` , `>>>=`
- example

```
int i;
//...
i += 2; //same as i = (int) (i + 2);
```

Increment/Decrement Operators

- in/decrease an l-value by one
- unary
- two flavors
  - prefix form
  - postfix form
- usage

```
L_VALUE++  //postfix
++L_VALUE  //prefix
L_VALUE--
--L_VALUE
```

- prefix form returns the *new* vale
- postfix form returns the *old* value (and then in/decrements)

Increment example

```
int i = 1;
int j = 2;
int k = ++i + j++; // i==2 j==3 k==4
i = 1;
j = 2;
k = i++ + j++; // i==2 j==3 k==3
```

Formatted Input

Reading Integer Values

Reading Integer Values - Code

```
1  package FormattedInput;
2
3  import java.util.Scanner;
4
5  public class ReadingIntegerValuesDemo {
6
7      public static void main(String[] args) {
8          Scanner sc = new Scanner(System.in);
9          System.out.printf(": ");     //prompt for input
10         int time = sc.nextInt();     //read an integer value
11         System.out.printf("time = %d\n", time); //print it again
12         sc.close();
13     }
14
15 }
```

Reading Integer Values - Output

(text in red is user input)

```
: 1245
time = 1245
```

Reading Integer Values - Explanation

- A `Scanner` is used to read input from `System.in`
- `System.in` is a *stream* from which the program *reads* input. For now, think of it as the keyboard.
- The `Scanner` *class* is *imported*, i.e. it is made available in our program.

```
3  import java.util.Scanner;
```

- A `Scanner` object named `sc` is created and connected to `System.in`

```
8          Scanner sc = new Scanner(System.in);
```

- The `Scanner` is instructed to read an integer value (which is *stored* in a *variable* named `time`)

```
10         int time = sc.nextInt();     //read an integer value
```

- The *variable* named `time` now *holds* the value that was entered
- the *value* of variable `time` is printed

```
11          System.out.printf("time = %d\n", time); //print it again
```

- The `Scanner` is closed at the end of the program

```
12          sc.close();
```

Reading Double Values

Reading Double Values - Code

```
1  package FormattedInput;
2
3  import java.util.Scanner;
4
5  public class ReadingDoubleValuesDemo {
6
7      public static void main(String[] args) {
8          Scanner sc = new Scanner(System.in);
9          System.out.printf(": ");
10         double temperature = sc.nextDouble();
11         System.out.printf("temperature = %.1f%n", temperature); //print it again
12         sc.close();
13     }
14
15 }
```

Reading Double Values - Console

```
: 22.8
temperature = 22.8
```

Reading Double Values - Explanation

- The `Scanner` is instructed to read a `double` value (which is *stored* in a *variable* named `temperature`)
- the variable is of type `double`

```
10          double temperature = sc.nextDouble();
```

Reading Strings

Reading Strings - Code

```
1  package FormattedInput;
2
3  import java.util.Scanner;
```

```
4
5  public class ReadingStringsDemo {
6
7      public static void main(String[] args) {
8          Scanner sc = new Scanner(System.in);
9          System.out.printf(": ");
10         String lastname = sc.next();
11         String firstname = sc.next();
12         String greeting = sc.nextLine();
13         System.out.printf("%s %s %s\n", greeting, firstname, lastname);
14         sc.close();
15     }
16
17 }
```

Reading Strings - Console

```
: Mustermann Max Guten Morgen
Guten Morgen Max Mustermann
```

Reading Strings - Explanation

- with sc.next() everthing till the next space or line break is read
- with sc.nextLine() a complete line is read (everthing till the next line break)
- it is read as a String

```
10         String lastname = sc.next();
11         String firstname = sc.next();
12         String greeting = sc.next();
```

Reading Strings - MiniExercise

Reading Strings - PersonDetails

Read the following details of a person on the console

- firstname
- lastname
- date of birth
  - year
  - month
  - day
- weight (in kg)
- height (in m)

and pretty-print them on the console (names are truncated to a max of 7 characters, height and weight are displayed with one digit after the decimal separator)

Example:

```
: Arnold Schwarzenegger 1947 7 30 113.5 1.88
Arnold Schwarz 30.07.1947 1.9m 113.5kg
```

Selection Statements

Conditional Statement

Overview

- some statements in a program should only be executed under certain conditions
- the conditions can be checked in an if statement

ConditionalStatementDemo - Code

```
1  public class ConditionalStatementDemo {
2
3      public static void main(String[] args) {
4          Scanner sc = new Scanner(System.in);
5          System.out.printf("grade: ");
6          int grade = sc.nextInt();
7
8          if (grade == 5) {
9              System.out.printf("failed.\n");
10         }
11
12         System.out.printf("Thank you.");
13         sc.close();
14     }
15 }
```

ConditionalStatementDemo - Flowchart

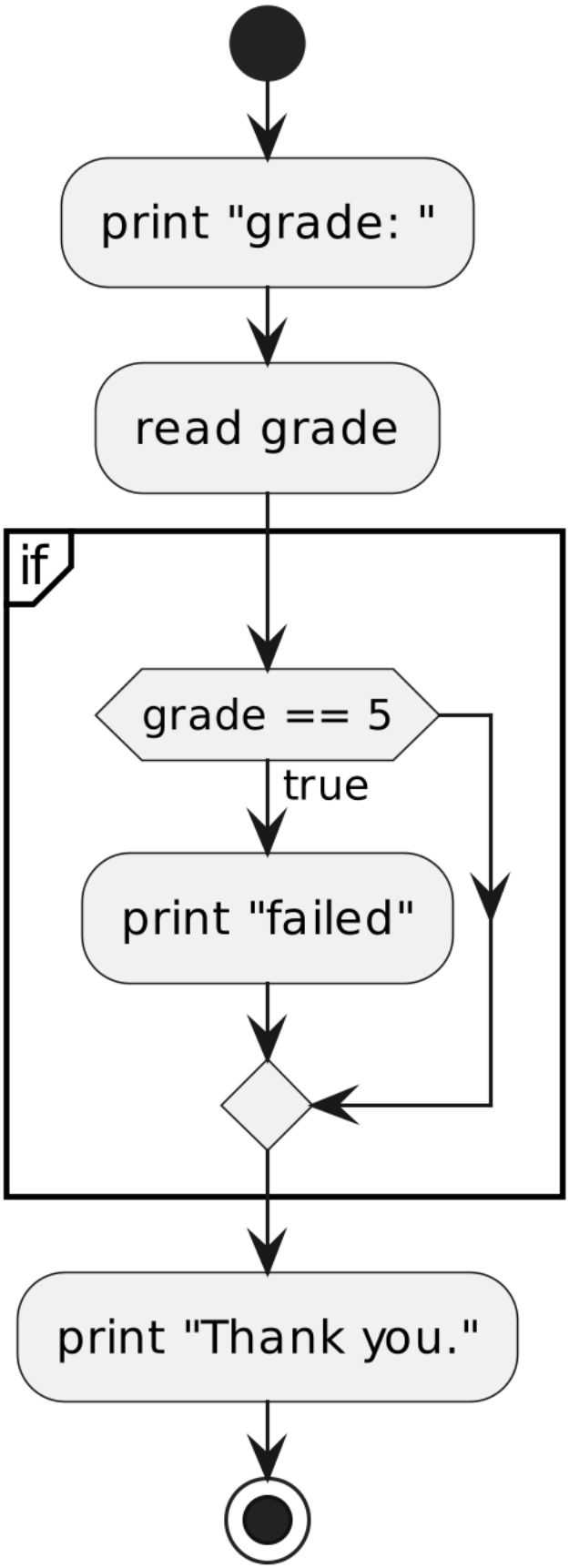


Figure 3: Flowchart ConditionalStatementDemo

ConditionalStatementDemo - Output

- failed case

```
grade: 5
failed.
Thank you.
```

- non-failed case

```
grade: 3
Thank you.
```

ConditionalStatementDemo - Summary

- equality operator ==
  - tests for equality
  - returns `true` if the two values are equal
  - returns `false` if the two values are *not* equal
- more comparisons are `<=` , `<` , `>` , `>=` , `!=` (see Relational Operators)

Conditional Statement - Syntax & Semantic I

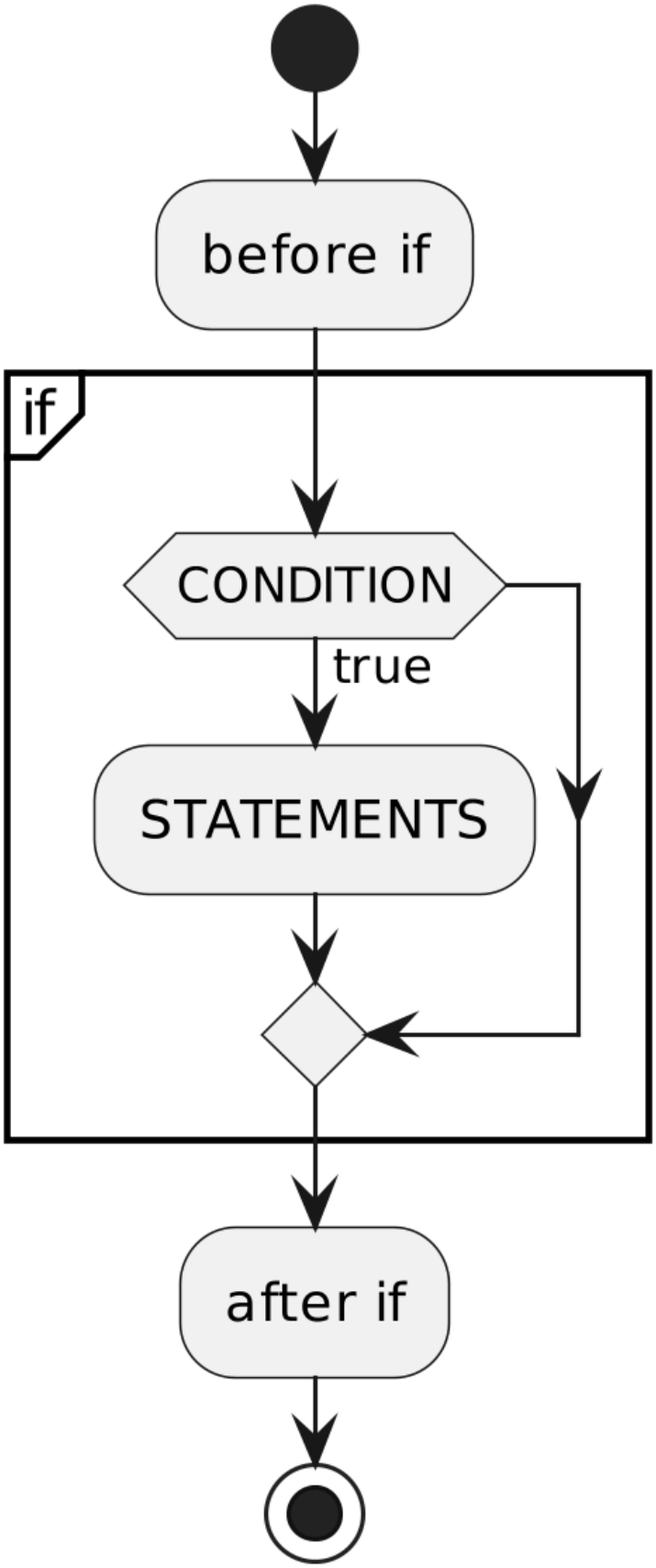


Figure 4: Flowchart ConditionalStatement

```
//before if

if (CONDITION) {
    STATEMENTS
}

//after if
```

Conditional Statement - Syntax & Semantic II

- `CONDITION` is a `boolean` expression
- `CONDITION` is always evaluated
- only if `CONDITION` is `true` the `STATEMENTS` are executed

Code Formatting

- a block opens (`{`) after `if` and closes (`}`) in the same column
- `STATEMENTS` are indented

Single Alternative

Single Alternative - Code

```
1
2  public class SingleAlternativeDemo {
3
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          System.out.printf("grade: ");
7          int grade = sc.nextInt();
8
9          if (grade == 5) {
10             System.out.printf("failed.\n");
11         } else {
12             System.out.printf("passed.\n");
13         }
14
15         System.out.printf("Thank you.");
16         sc.close();
17     }
18
19 }
```

Single Alternative - Console

- failed case

```
grade: 5
failed.
Thank you.
```

- passed case

```
grade: 3
passed.
Thank you.
```

Single Alternative - Summary

- We can specify what shall happen if `CONDITION` is `false` in the `else` block

```
1 //before if/else
2
3 if (CONDITION) {
4     STATEMENTS_1
5 } else {
6     STATEMENTS_2
7 }
8
9 //after if/else
```

- Semantic:
  - `CONDITION` is always evaluated
  - only if `CONDITION` is `true` the `STATEMENTS_1` are executed
  - otherwise (if `CONDITION` is `false`) the `STATEMENTS_2` are executed
- use single alternative if a problem/system has exactly two possible cases/states, one of which must be picked/realized

Single Alternative - Flowchart

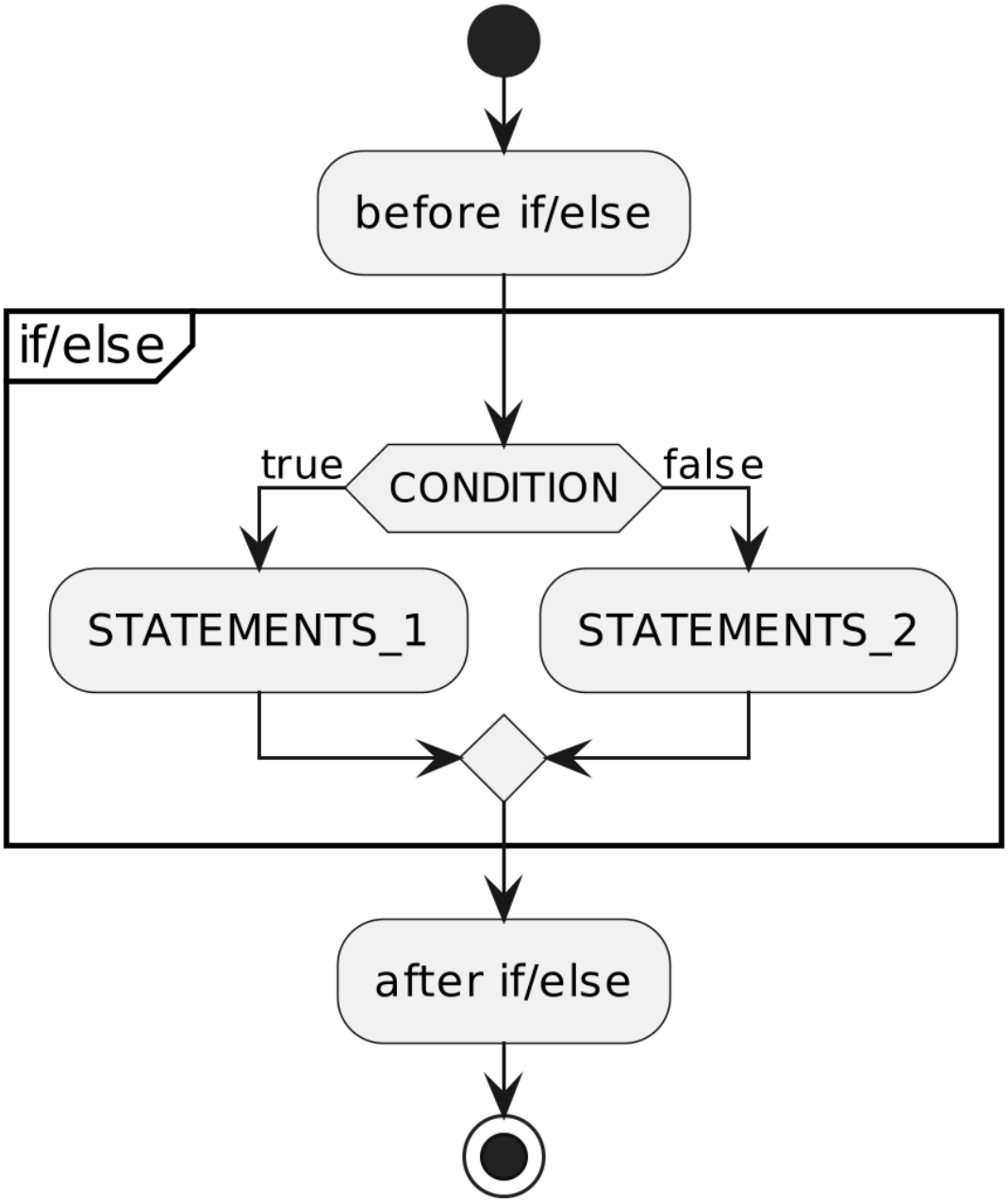


Figure 5: Flowchart SingleAlternative

Single Alternative - Code Formatting

- a block opens (`{`) after `if` and closes (`}`) in the same column
- a block opens (`{`) after `else` and closes (`}`) in the same column
- `STATEMENTS_1` and `STATEMENTS_2` are indented

Single Alternative - MiniExercise

Modify `SingleAlternativeDemo` such that the `if` case covers *passing* grades.

```
    if (/* find the correct condition*/) {
        System.out.printf("passed.\n");
    } else {
        System.out.printf("failed.\n");
    }
```

Consecutive ifs

Consecutive ifs - Code

```
    Scanner sc = new Scanner(System.in);
    System.out.printf("grade: ");
    int grade = sc.nextInt();

    if (grade < 6 && grade > 0) {
        System.out.printf("valid\n");
    }else {
        System.out.printf("invalid\n");
    }

    if (grade == 5) {
        System.out.printf("failed.\n");
    } else {
        System.out.printf("passed.\n");
    }

    System.out.printf("Thank you.");
    sc.close();
```

Consecutive ifs - Console

- valid and failed case

```
grade: 5
valid
failed.
Thank you.
```

- vaild and passed case

```
grade: 3
valid
passed.
Thank you.
```

- invalid and passed case (makes no sense - will be fixed)

```
grade: 10
invalid
passed.
Thank you.
```

Consecutive ifs - Summary

- logical AND operator `&&`
  - yields `true` if and only if both sides are `true`
- other logical operators
  - OR `||` yields `true` if at least on side is `true`
  - NOT `!` "inverts" the value of its right side (`true` -> `false` , `false` -> `true`)
- Condition of second if (line 11) is evaluated, regardless of the value of condition of first if (line 5)
- In this problem, however, there is a logical connection (given by the problem and implemented in the conditions)
- Use consecutive ifs when the two conditions are independent of each other

Consecutive ifs - Flowchart

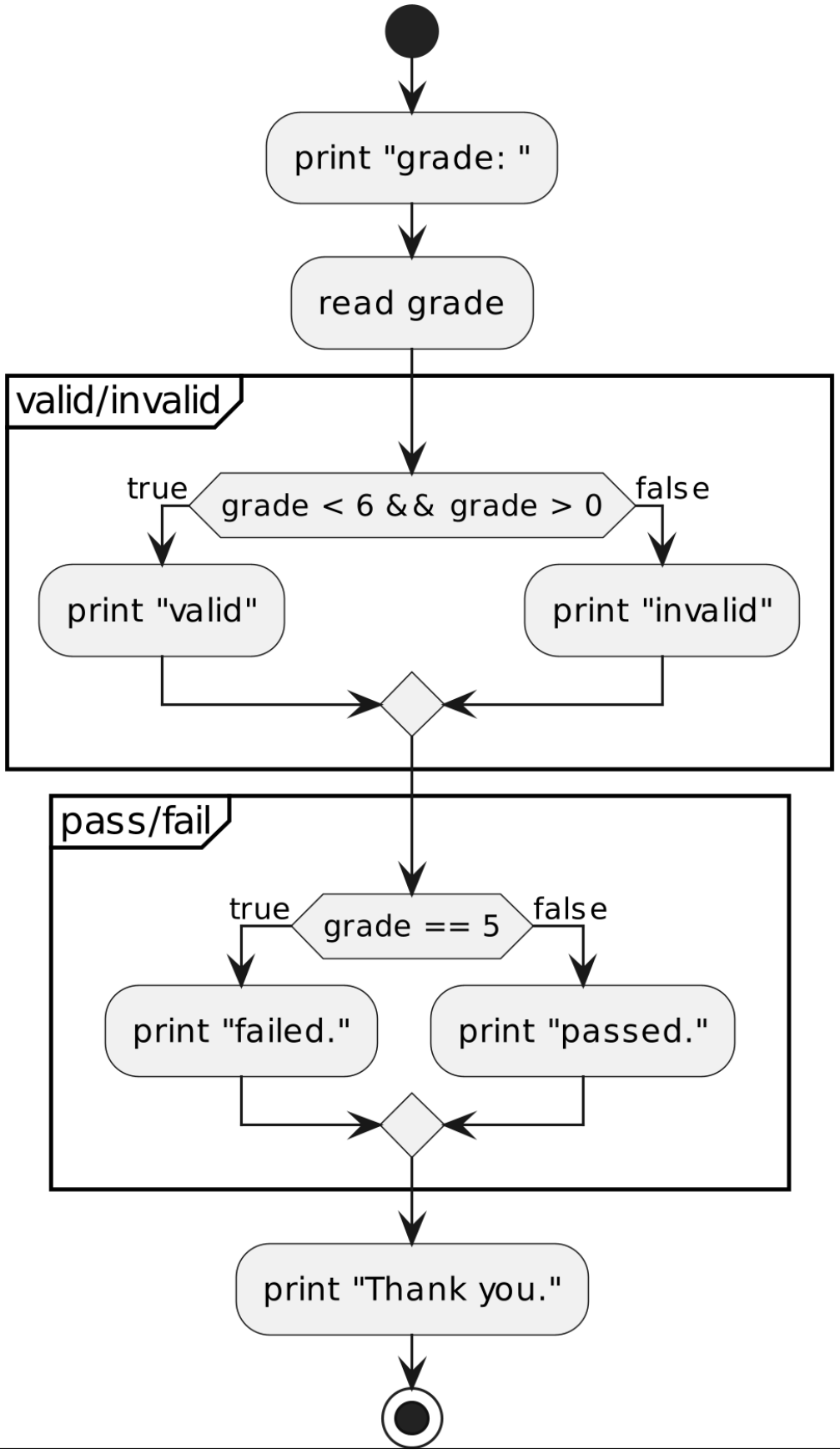


Figure 6: Flowchart Consecutive Ifs (Example)

Nested Ifs

Nested Ifs - Code

```
1         Scanner sc = new Scanner(System.in);
2         System.out.printf("grade: ");
3         int grade = sc.nextInt();
4
5         if (grade < 6 && grade > 0) {
6             System.out.printf("valid\n");
7             if (grade == 5) {
8                 System.out.printf("failed.\n");
9             } else {
10                System.out.printf("passed.\n");
11            }
12        } else {
13            System.out.printf("invalid\n");
14        }
15
16        System.out.printf("Thank you.");
17        sc.close();
```

Nested Ifs - Console

- valid and failed case

```
grade: 5
valid
failed.
Thank you.
```

- valid and passed case

```
grade: 3
valid
passed.
Thank you.
```

- invalid case

```
grade: 10
invalid
Thank you.
```

Nested Ifs - Summary

- outer if/else structure is the top-level distinction valid/invalid
- nested if/else structure is only executed if outer condition was `true` (we have a valid grade)
- conditions are logically connected
- full inner condition is `(grade < 6 && grade > 0) && (grade == 5)`

Nested Ifs - Flowchart

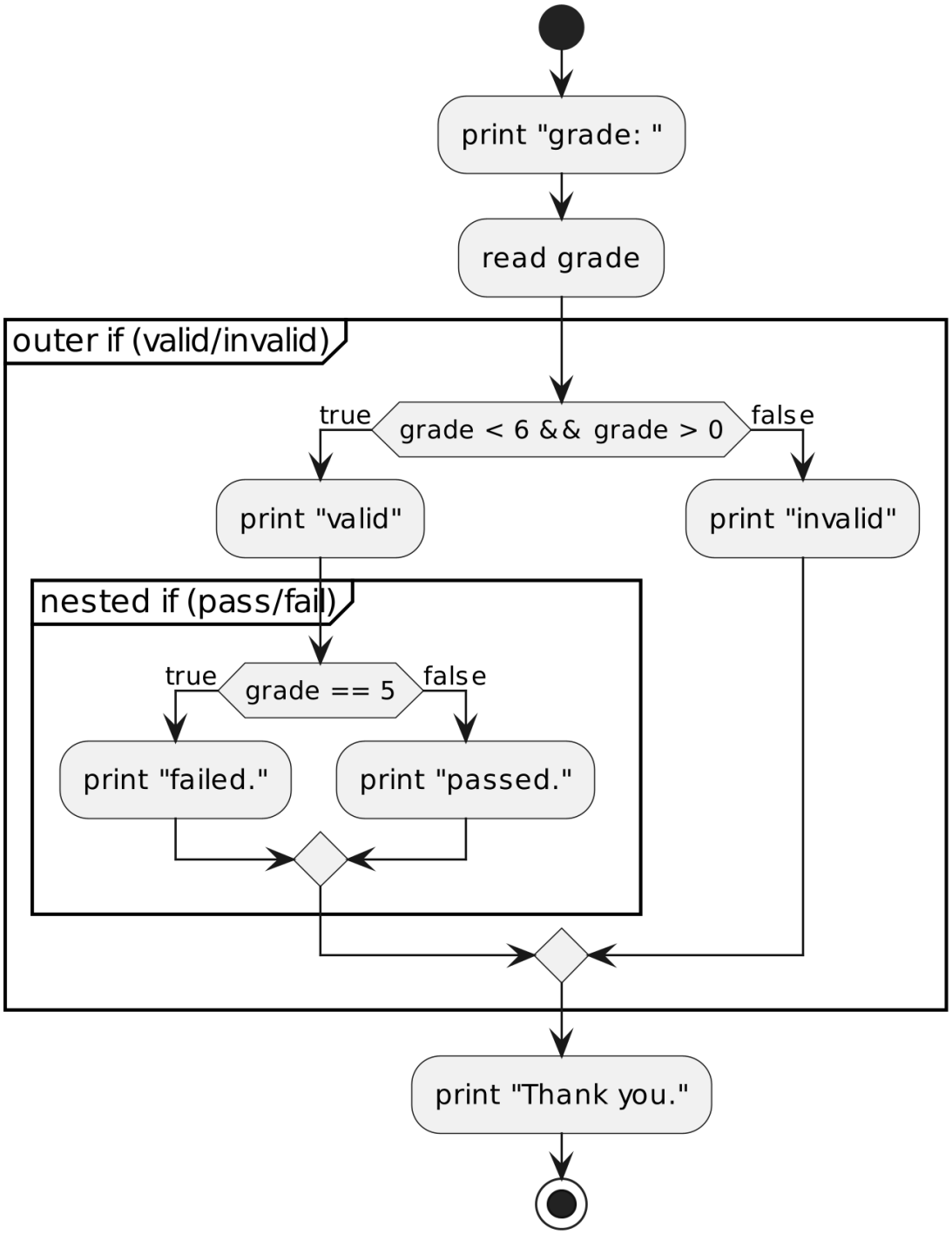


Figure 7: Flowchart Nested Ifs (Example)

Multiple Alternative

Multiple Alternative - Code

```
1         Scanner sc = new Scanner(System.in);
2         System.out.printf("grade: ");
3         int grade = sc.nextInt();
4
5         if (!(grade < 6 && grade > 0)) {
6             System.out.printf("invalid\n");
7         } else if (grade == 5) {
8             System.out.printf("failed.\n");
9         } else if (grade == 4) {
10            System.out.printf("close fail.\n");
11        } else if (grade == 3) {
12            System.out.printf("minimum pass.\n");
13        } else if (grade == 2) {
14            System.out.printf("very good.\n");
15        } else {
16            System.out.printf("excellent.\n");
17        }
18
19        System.out.printf("Thank you.");
20        sc.close();
```

Multiple Alternative - Console

- failed case

```
grade: 5
failed.
Thank you.
```

- minimum passed case

```
grade: 3
minimum pass.
Thank you.
```

- invalid case

```
grade: 10
invalid
Thank you.
```

Multiple Alternative - Summary

- Semantics
  - conditions are evaluated top to bottom
  - statements of first true condition are executed
  - and control is tranferred to the end of the *complete* structure (after `else`)

  - if no condition is true, the else branch is executed

- else is optional
- use multiple alternative **with** an else branch when a problem/system has several possible cases/states, and **exactly one** of them must be picked/realized
- use multiple alternative **without** an else branch when a problem/system has several possible cases/states, and **at most one** of them must be picked/realized

Multiple Alternative - Flowchart

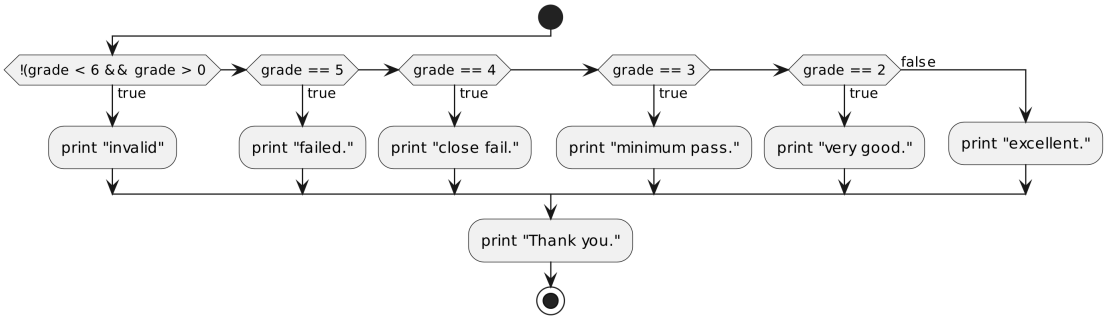


Figure 8: Flowchart Multiple Alternative (Example)

Switch/Case

Switch/Case - Overview

- a `switch` statement can be used to check different cases of an integer or a String against several constants
- it is sometimes a more readable alternative to the `if`/`else` - `if` logic

Example Simple Text Menu

```
Scanner sc = new Scanner(System.in);
System.out.println("Please enter 1-3: ");
int a = sc.nextInt();

switch(a) {
    case 1:
        System.out.println("Selection 1\n");
        break;
    case 2:
        System.out.println("Selection 2\n");
        break;
    case 3:
        System.out.println("Selection 3\n");
        break;
    default:
        System.out.println("Wrong input.\n");
} /*End of switch*/
```

- the value in a will be compared with the cases
- if none of the cases is equal to the value in a the default case will be executed
- the default case is optional (it similar to “else” in the if statement)
- the “break;” statement is used to jump to the end of the switch block as soon as a case is true, if there is no break all following cases will be executed after the first one is true (this is called “fallthrough”)

Example Number Days

```
Scanner sc = new Scanner(System.in);
int month = sc.nextInt();

switch (month) {
    case 1:
    case 3:
    case 5:
    case 7:
    case 8:
    case 10:
    case 12:
        System.out.println("31 days");
        break;
    case 4:
    case 6:
    case 9:
    case 11:
        System.out.println("30 days");
        break;
    case 2:
        System.out.println("28 days (or 29 if leap year)");
        break;
    default:
        System.out.println("Invalid month.");
}
```

Relational and Conditional Operators

Relational and Conditional Operators - Overview

- `if` tests a condition
- the condition must be a `boolean` expression
- **relational operators** like `<` (less than) compare two (numeric) values and return a boolean value
- **conditional** operators like `||` (logical OR) operate on boolean values and return a boolean value

- with the types of operators we can construct logical expressions and conditions

Relational Operators

Relational Operators compare two (numeric) expressions and return a `boolean` value.

Table 16: Relational Operators

Symbol	Operation
`<`	Less Than
`<=`	Less Than Or Equal
`>`	Greater Than
`>=`	Greater Than Or Equal
`==`	Equal
`!=`	Not Equal

Conditional Operators

Conditional Operators operate on `boolean` expressions and return a `boolean` value.

Table 17: Conditional Operators

Symbol	Operation
`&&`	AND
`||`	OR
`!`	NOT (unary operator)

Table 18: Truth Table (OR, AND, NOT)

`A`	`B`	`A || B`	`A && B`	`!A`
`true`	`true`	`true`	`true`	`false`
`true`	`false`	`true`	`false`	`false`
`false`	`true`	`true`	`false`	`true`
`false`	`false`	`false`	`false`	`true`

Iteration Statements

Iteration Introduction

Iteration Motivation

A way to deal with invalid input

```
1     Scanner sc = new Scanner(System.in);
2     int grade = sc.nextInt();
3     if (grade < 1 || grade > 5) {
4         System.out.printf("invalid.\n");
5         grade = sc.nextInt();
6     }
7     // classify grades
8     System.out.printf("Thank you.");
9     sc.close();
```

Iteration Motivation: Valid Input

```
2
Thank you.
```

Iteration Motivation: Invalid Input

```
9
invalid.
2
Thank you.
```

Iteration Motivation: FlowChart

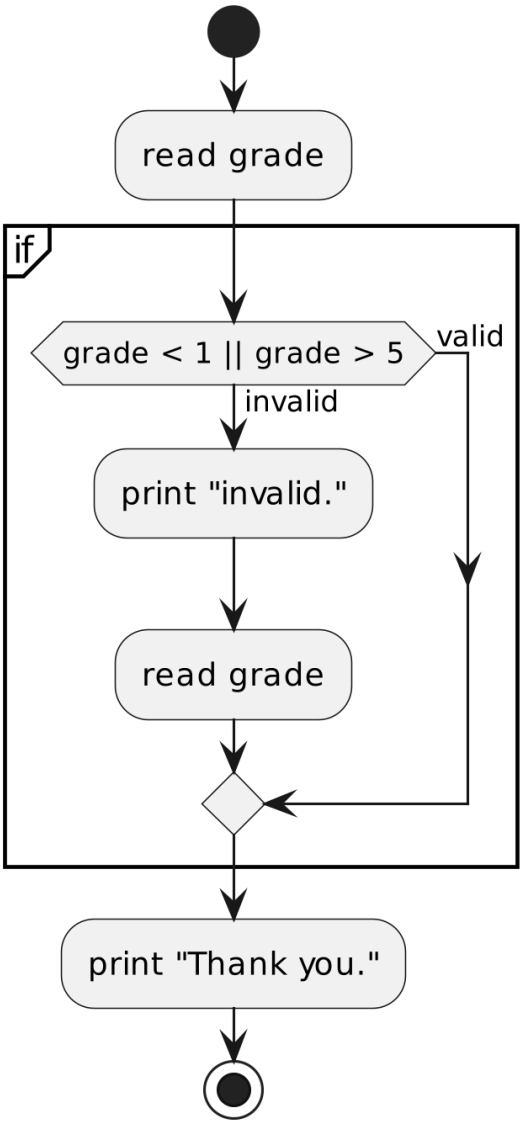


Figure 9: Flowchart of input validation with `if`

Iteration Motivation: Summary

- Only one invalid input can be handled
- We want to be able to **repeat** input (and validation) **until** input is **valid**
- We want to be able to **repeat** input (and validation) **while** input is **invalid**

`while` statement

WhileDemo - Code

```
    Scanner sc = new Scanner(System.in);
    int grade = sc.nextInt();
    while (grade < 1 || grade > 5) {
        System.out.printf("invalid.\n");
        grade = sc.nextInt();
    }
    // classify grades
    System.out.printf("Thank you.");
    sc.close();
```

WhileDemo - valid input

```
2
Thank you.
```

WhileDemo - one invalid input

```
9
invalid.
2
Thank you.
```

WhileDemo - several invalid inputs

```
9
invalid.
12
invalid.
19
invalid.
102
invalid.
2
Thank you.
```

WhileDemo - flowchart

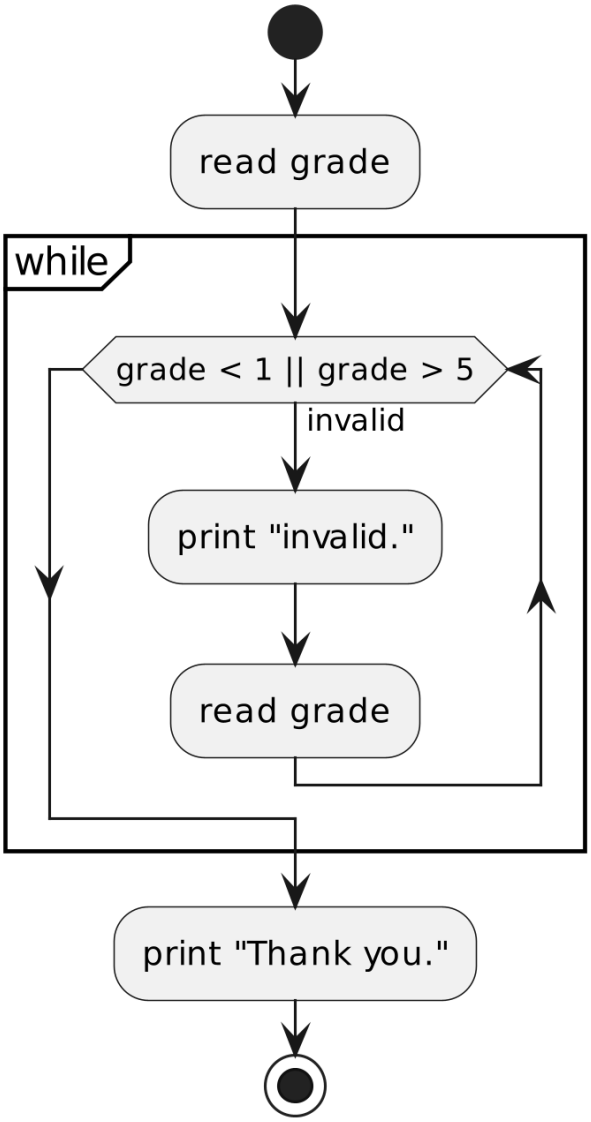


Figure 10: Flowchart of input loop. Note the difference to validation with `if` .

- Input is repeated **as long as** it is invalid
- Input is repeated **while** it is invalid

`while` statement - summary I

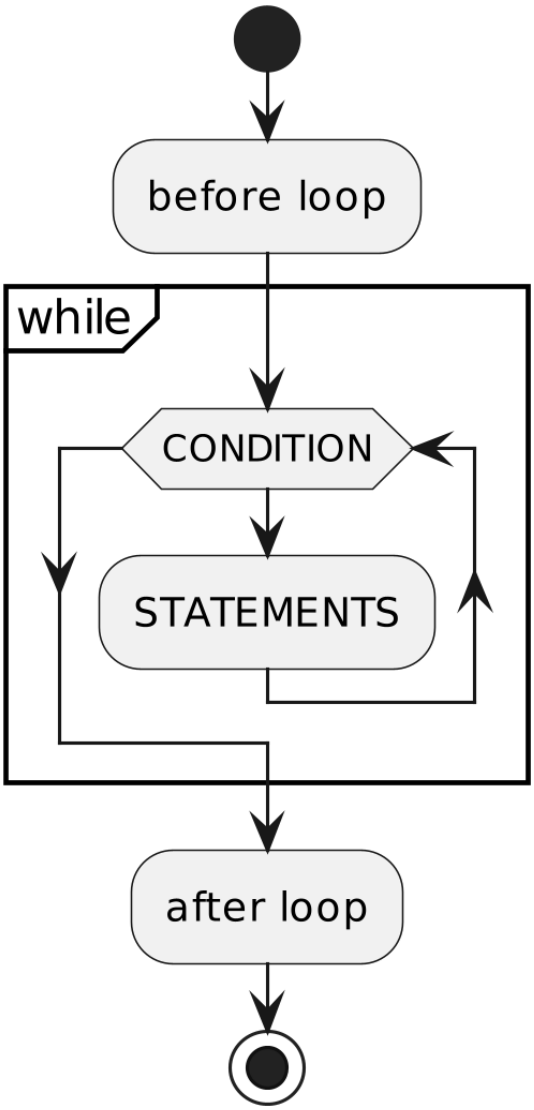


Figure 11: Flowchart of the `while` loop

```
//before loop

while (CONDITION) {
    STATEMENTS
}

//after loop
```

`while` statement - summary II

- Semantic:

  - `CONDITION` is always evaluated
  - if `CONDITION` is `true` the `STATEMENTS` are executed
  - and `CONDITION` is evaluated again
  - this cycle repeats *as long as* `CONDITION` evaluates to `true`
- `CONDITION` must be a `boolean` expression
- `CONDITION` is a **run condition**
- immediately after the loop `CONDITION` is `false` (otherwise we would still be *inside* the loop)

`while` statement - summary III

- `while` statement repeats a block of code
- `CONDITION` is evaluated at least once
- `STATEMENTS` are executed zero or more times
- **head-controlled** since `CONDITION` is evaluated **before** the **loop body** is executed

`for` statement

`for` statement - Motivation

A counter controlled while loop.

```
1     int n = 10;
2     while (n > 0) {
3         //many lines of complex code
4         System.out.printf("%d\n", n);
5         n--;
6     }
```

The loop

- starts at 10 (`int n = 10;`)
- ends at 1 (`n > 0`)
- in steps of 1 (`n--;`)

This information is not (or need not be) in the same line or in lines close to each other.

ForDemo - Code

```
1     for (int n = 10; n > 0; n--) {
2         //many lines of complex code
3         System.out.printf("%d\n", n);
4     }
```

The loop

- starts at 10 (`int n = 10;`)
- ends at 1 (`n > 0`)
- in steps of 1 (`n--;`)

This information is on the same line.

ForDemo - Console

```
10
9
8
7
6
5
4
3
2
1
```

ForDemo - Flowchart

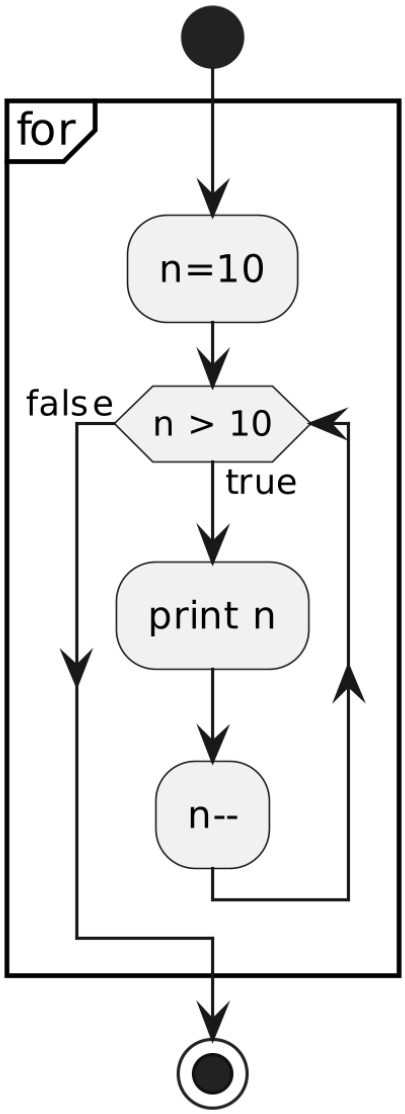


Figure 12: For Example

`for` statement - summary I

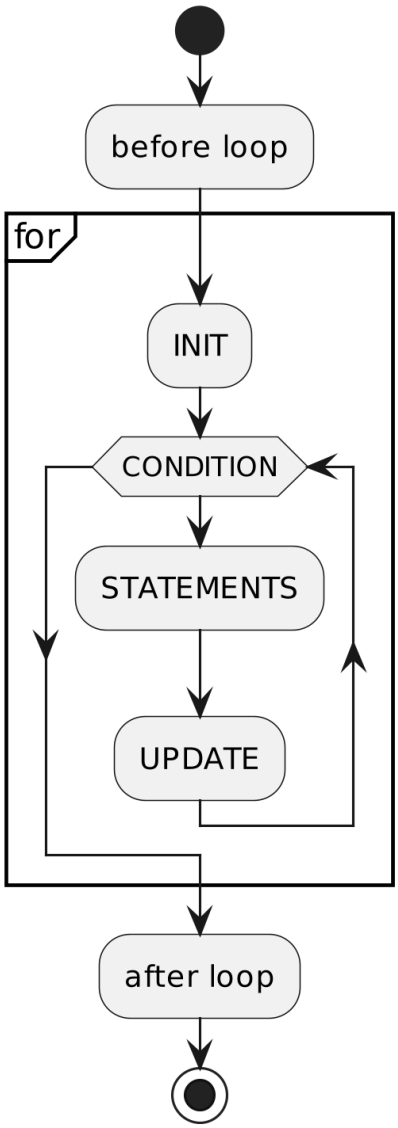


Figure 13: Flowchart of the `for` loop

```
//before loop

for (INIT; CONDITION; UPDATE) {
    STATEMENTS
}

//after loop
```

`for` statement - Summary II

- Semantic:

  - `INIT` is evaluated once
  - then `CONDITION` is evaluated
  - if `CONDITION` is `true` the `STATEMENTS` are executed and `UPDATE` is evaluated
  - then `CONDITION` is evaluated again
  - this cycle (`CONDITION` -> `STATEMENTS` -> `UPDATE` -> `CONDITION` -> ...) repeats *as long as* `CONDITION` evaluates to `true`

- `CONDITION` is a **run condition**
- `CONDITION` must be a `boolean` expression
- variables declared in `INIT` are local to the loop (out of scope after the loop)

`for` - Summary III

- `for` statement repeats a block of code
- it is typically used as a counter controlled loop
- `INIT` is evaluated exactly once
- `CONDITION` is evaluated at least once
- `STATEMENTS` are evaluated zero or more times
- `UPDATE` is evaluated zero or more times
- **head-controlled** since `CONDITION` is evaluated **before** the **loop body** is executed

`do` statement

DoDemo - Code

```
1     Scanner sc = new Scanner(System.in);
2     int grade;
3     do {
4         System.out.print(": ");
5         grade = sc.nextInt();
6     } while (grade < 1 || grade > 5);
7     // classify grades
8     System.out.printf("Thank you.");
9     sc.close();
```

DoDemo - valid input

```
: 2
Thank you.
```

DoDemo - one invalid input

```
: 9
: 2
Thank you.
```

DoDemo - several invalid inputs

```
: 9
: 12
: 19
: 102
: 2
Thank you.
```

DoDemo - flowchart

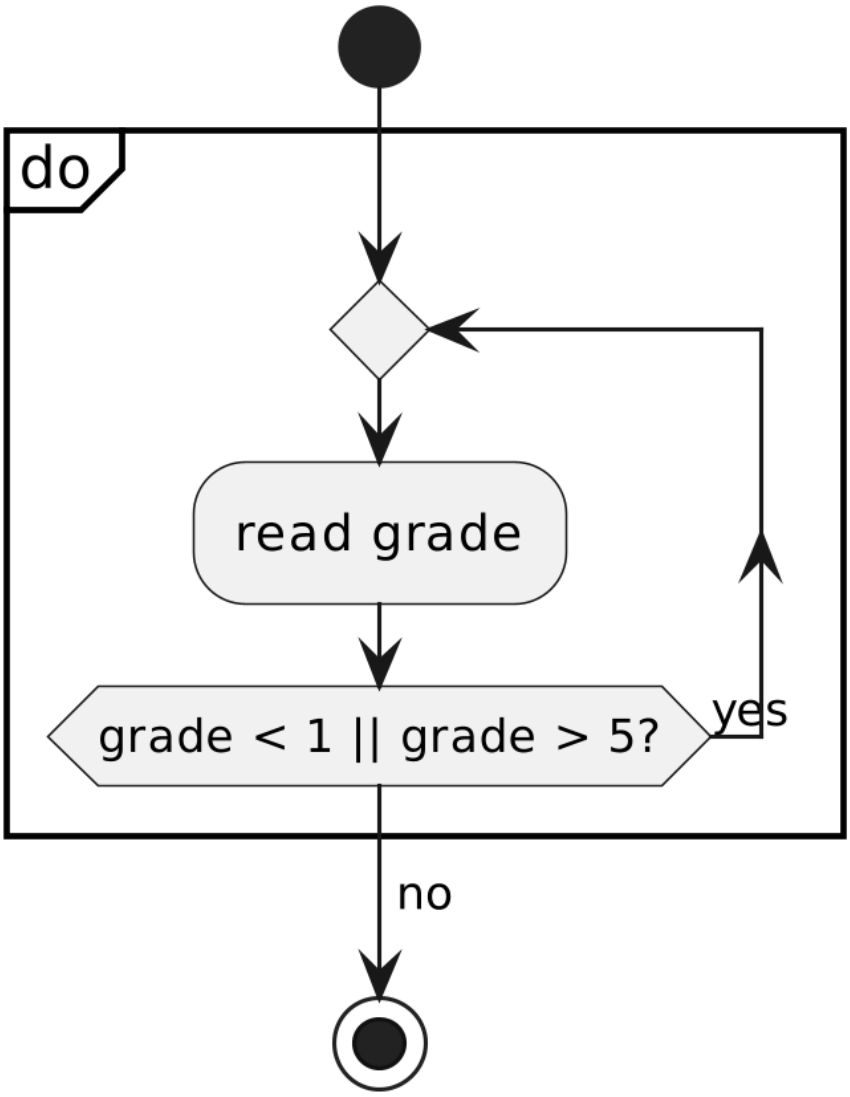


Figure 14: Flowchart of input loop with `do/while` .

- Input is repeated **as long as** it is invalid
- Input is repeated **while** it is invalid

`do` statement - summary I

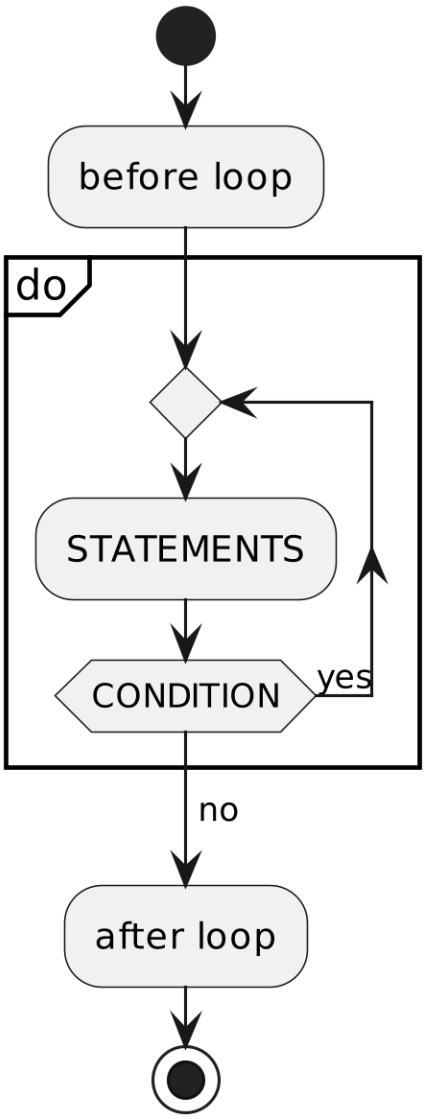


Figure 15: Flowchart of the `do` loop

```
1  //before loop
2
3  do {
4      STATEMENTS
5  } while (CONDITION); //note the ;
6
7  //after loop
```

`do` statement - summary II

- Semantic:

  - `STATEMENTS` is executed at least once
  - `CONDITION` is evaluated at least once
  - if `CONDITION` is `true` the `STATEMENTS` are executed again
  - and then `CONDITION` is evaluated again
  - this cycle repeats *as long as* `CONDITION` evaluates to `true`
- `CONDITION` must be a `boolean` expression
- `CONDITION` is a **run condition**
- immediately after the loop `CONDITION` is `false` (otherwise we would still be *inside* the loop)

`do` statement - summary III

- `do` statement repeats a block of code
- `CONDITION` is evaluated at least once
- `STATEMENTS` are executed one or more times
- **foot-controlled** since `CONDITION` is evaluated **after** the **loop body** is executed

Special Cases

Infinite loops

```
while(true)
    ;
```

- the loop will never end (in a practical sense)
- similar constructs are in principle possible with the other iteration statements
- the above example is very simple - watch out for and avoid disguised infinite loops, like below

```
int a = 0;
while(a<10){
  //do smth
  a = 0; //typically some complicated calculation that always yields 0
}
```

Incomplete `for` I

```
int i=0;
for(; i<10; i++)
    ;
```

- `INIT` can be left blank

Incomplete `for` II

```
int i=0;
for(; i<10; )
    i++;
```

- `UPDATE` can also be left blank

Incomplete `for` III

```
for(;;)
    ;
```

- `CONDITION` can also be left blank
- it is the same as `for(; true; )` or `while(true)`
- this is an infinite loop

Jump commands

Break and Continue

- `break;`
  - can be used to jump out of a loop
  - is also used in `switch` to prevent "fallthrough" (see Selection Statements)
- `continue;`
  - Control remains inside the loop, transfers control just before the end of the loop body
  - can be seen as "skipping" the current run and "continuing" with the next run

```
    for (int i = 0; i < 10; i++) {
        if (i%2 == 1) continue; //skip odd numbers
        System.out.printf("%d\n", i);
    }
```

Infinite loop with break

```
while (true) {
    statements;
    if(TERMINATION_CONDITION) break;
}
```

- `while(true)` would be an infinite loop, but the break command in the loop will end the loop
- control was transferred to the body of the loop
- it is possible to have several break commands at different positions in the loop body

Iteration summary

Iteration Overview

- There are three iteration statements in Java
  - `while`
  - `for`
  - `do` (aka `do`/`while`)
- There is nothing one can do that the others cannot (all three have equal expressive power)
- It mght be easier to express something with one loop than with another

- for counter controlled loops (especially with Arrays) `for` loops are often preferred
- enclose the loop body in a block (`{ /* loop-body*/ }`)
- indent the body for clarity of structure and improved readability
- the loop body may consist of any sequence of statements, including
  - expression statements
  - input/output
  - selections (`if`/`else`)
  - iterations (loops)
- jump statements let you tranfer control in non-standard ways
  - use `break` to "break out of a loop"
  - use `continue` to skip over the current run but stay in the loop
  - use them carefully and scarcely (only if you have a very good reason)

Strings

Introduction

Overview

- by “string” we mean “string of characters”, e.g. a text
- Java has a string type: `String`
- `String` is not a primitive type
- use `String` variables for text
- do **not** use `String` variables when better types exist (numbers, enumerations, etc.)

String Operations

String concatenation

```
1 String str1 = "This is a text."; // a string variable
2 System.out.printf("str1 = \\"%s\\"\\n", str1);
3
4 String str2 = "This is another text.";
5 System.out.printf("\\n%s\\n", str1 + str2); //concatenation
```

- Concatenate strings with the concatenation operator (`+`)

```
str1 : "This is a text."
concatenated: "This is a text. This is another text."
```

String equality

```
1 if (str1.equals(str2))   //this is how to check for equality!!
2     System.out.printf("\"%s\" equals \"%s\"\n", str1, str2);
3 else
4     System.out.printf("\"%s\" not equals \"%s\"\n", str1, str2);
```

- **always** use `equals` to test for equality of strings

```
"This is a text." not equals "This is another text."
```

String comparison

```
1 String name1 = "hans";
2 String name2 = "hAns";
3 if (name1.compareToIgnoreCase(name2) < 0) { //compare two strings
4     System.out.printf("%s < %s\n", name1, name2);
5 } else
6     System.out.printf("%s >= %s\n", name1, name2);
```

- returns “a measure of the distance” between the two strings
- `compareTo` if case is to be considered

```
"hans" >= "hAns"
```

String trimming

```
String trailing = "\t\n   te xt   \t\n";
System.out.printf("untrimmed: \"%s\"\n", trailing);
System.out.printf("trimmed: \"%s\"\n", trailing.trim()); //remove leading and trailing whitespace
```

- `trim` removes both leading and trailing whitespace

```
untrimmed: "
   te xt
"
trimmed: "te xt"
```

Characters

CharacterDemo

```
char c = 'H'; // a single character
System.out.printf("c: '%c' (%d)\n", c, c + 0);

System.out.printf("added: '%c'\n", c + ('a' - 'A')); //convert to lower case
```

- `char` is the type for a single character
- we can calculate with `char`
- see ASCII Table (Fig. **??**) for (some) values
- Java supports unicode

```
c: 'H' (72)
added: 'h'
```

Reading a single Character with `Scanner`

```
Scanner sc = new Scanner(System.in);
sc.useDelimiter("");    //set up to read single char
String s = sc.next();   //one char read, returned as String
c = s.charAt(0);        //extract the char from String
System.out.printf("s: \"%s\"\n", s);
System.out.printf("c: '%c'\n", c);
sc.reset();             //reset to standard behavior

sc.close();
```

- `Scanner.next` always returns a string, but we can make it read single-character strings
- extract the character as `char` with `charAt(0)` (if needed)

```
X
s: "X"
c: 'X'
```

ASCII Table

Dec	Hx	Oct	Char	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr	Dec	Hx	Oct	Html	Chr
0	0	000	NUL (null)	32	20	040		Space	64	40	100	@	@	96	60	140	`	`
1	1	001	SOH (start of heading)	33	21	041	!	!	65	41	101	A	A	97	61	141	a	a
2	2	002	STX (start of text)	34	22	042	"	"	66	42	102	B	B	98	62	142	b	b
3	3	003	ETX (end of text)	35	23	043	#	#	67	43	103	C	C	99	63	143	c	c
4	4	004	EOT (end of transmission)	36	24	044	$	$	68	44	104	D	D	100	64	144	d	d
5	5	005	ENQ (enquiry)	37	25	045	%	%	69	45	105	E	E	101	65	145	e	e
6	6	006	ACK (acknowledge)	38	26	046	&	&	70	46	106	F	F	102	66	146	f	f
7	7	007	BEL (bell)	39	27	047	'	'	71	47	107	G	G	103	67	147	g	g
8	8	010	BS (backspace)	40	28	050	(	(	72	48	110	H	H	104	68	150	h	h
9	9	011	TAB (horizontal tab)	41	29	051	)	)	73	49	111	I	I	105	69	151	i	i
10	A	012	LF (NL line feed, new line)	42	2A	052	*	*	74	4A	112	J	J	106	6A	152	j	j
11	B	013	VT (vertical tab)	43	2B	053	+	+	75	4B	113	K	K	107	6B	153	k	k
12	C	014	FF (NP form feed, new page)	44	2C	054	,	,	76	4C	114	L	L	108	6C	154	l	l
13	D	015	CR (carriage return)	45	2D	055	-	-	77	4D	115	M	M	109	6D	155	m	m
14	E	016	SO (shift out)	46	2E	056	.	.	78	4E	116	N	N	110	6E	156	n	n
15	F	017	SI (shift in)	47	2F	057	/	/	79	4F	117	O	O	111	6F	157	o	o
16	10	020	DLE (data link escape)	48	30	060	0	0	80	50	120	P	P	112	70	160	p	p
17	11	021	DC1 (device control 1)	49	31	061	1	1	81	51	121	Q	Q	113	71	161	q	q
18	12	022	DC2 (device control 2)	50	32	062	2	2	82	52	122	R	R	114	72	162	r	r
19	13	023	DC3 (device control 3)	51	33	063	3	3	83	53	123	S	S	115	73	163	s	s
20	14	024	DC4 (device control 4)	52	34	064	4	4	84	54	124	T	T	116	74	164	t	t
21	15	025	NAK (negative acknowledge)	53	35	065	5	5	85	55	125	U	U	117	75	165	u	u
22	16	026	SYN (synchronous idle)	54	36	066	6	6	86	56	126	V	V	118	76	166	v	v
23	17	027	ETB (end of trans. block)	55	37	067	7	7	87	57	127	W	W	119	77	167	w	w
24	18	030	CAN (cancel)	56	38	070	8	8	88	58	130	X	X	120	78	170	x	x
25	19	031	EM (end of medium)	57	39	071	9	9	89	59	131	Y	Y	121	79	171	y	y
26	1A	032	SUB (substitute)	58	3A	072	:	:	90	5A	132	Z	Z	122	7A	172	z	z
27	1B	033	ESC (escape)	59	3B	073	;	;	91	5B	133	[	[	123	7B	173	{	{
28	1C	034	FS (file separator)	60	3C	074	<	<	92	5C	134	\	\	124	7C	174			|
29	1D	035	GS (group separator)	61	3D	075	=	=	93	5D	135	]	]	125	7D	175	}	}
30	1E	036	RS (record separator)	62	3E	076	>	>	94	5E	136	^	^	126	7E	176	~	~
31	1F	037	US (unit separator)	63	3F	077	?	?	95	5F	137	_	_	127	7F	177	&#127;	DEL

Source: www.LookupTables.com

Figure 16: ASCII-Table

SimpleFunctions

Introduction

Overview

- The purpose of functions is to group (encapsulate) some computations and use (call) them without worrying about the implementation details
- With functions it is possible to ignore how something is done as long as you know what is done

Syntax

```
1  public class ClassName{
2
3      public static void main(String[] args){
4          //call function in other functions
5          returnType variable = functionName(parameter-list);
6      }
7
8      public static returnType functionName(parameter-list){
9          //declarations and statements
10         //ends with return statement
11         return returnTypeValue;
12     }
13 }
```

Declaration

- a function is declared within a class but outside the main() function
- function name should be descriptive of what the function does

Return type

- returnType is the type of the value returned by the function
- the returnType is void if the function returns nothing
- only one value can be returned by a function

Parameter list

- a list of parameters can be given to a function
- the number and data types of the parameters is fixed
- it is possible to have a function with 0 parameters

Return statement

- the return statement ends the function
- it gives the returnTypeValue back to the caller
- the return statement can also be used to end a void function without returning any value.

Examples

findMin function

```java
public class Main{

    public static void main(String[] args){
        int a = 5;
        int b = 3;
        //values of a and b are passed to findMin
        //after the execution of findMin the value of second is returned
        int min = findMin(a, b);
        // "3" is printed
        System.out.print(min);
    }

    public static int findMin(int first, int second){
        //first and second receiving copies of a and b
        //in this example first = 5 and second = 3
        if(first < second){
            return first;
        }
        else{
            //in this example the second value is returned
            return second;
        }
    }
}
```

printGreeting function

```java
public class Main{

    public static void main(String[] args){
        printGreeting();
        System.out.print("Hello World!");
    }

    //the return type of this function is void,
    //because this function doesn't return any value.
    public static void printGreeting(){
        System.out.print("Hello User!");
    }
}
```

```
Hello User!
Hello World!
```

Function parameters and variables

power function

```
1  public class Main{
2
3      public static void main(String[] args){
4          int number = 2;
5          int n = 4;
6          int p = power(number, n);
7          System.out.printf("%d^%d=%d", number, n, p);
8      }
9
10     // number and n in power() are initialized
11     // with copies of the passed arguments
12     public static int power(int number, int n){
13         int p = 1;
14         while(n>0){
15             p = p * number;
16             // n changes in this function but the original value
17             // in the main function remains unchanged
18             n--;
19         }
20         return p;
21     }
22 }
```

```
2^4=16
```

Parameter passing

- number, order and type of parameters in the function call must match those of formal parameters in definition (and declaration)
- arguments are passed by value to functions (as a copy of the original)
- modification of the (local, temporary) copies do not affect original values in calling function

Scope of variables

- scope of a variable: the section in which it is visible
- a variable is visible within the block it is declared
- variables declared in a function are local to the function (cannot be accessed from outside the function)

Function overloading

- in Java, it is possible to define several functions of the same name. They are called overloaded functions.
- they MUST differ in parameterlist (number of parameters and/or parameter type)
- the return type can bethe same
- Java decides which of the overloaded versions to call based on the paramterlist.
- a different return type with the same parameter list is a compilation error

Function Overloading Examples

- use overloaded functions for same tasks with different arguments or argument types

```java
public class Main{
    public static void main(String[] args){
        int width = 2;
        int height = 3;
        double width2 = 2.0;
        double height2 = 3.5;
        double radius = 3.0;

        //calls the first function because width and height are int
        double area = calculateArea(width, height);

        //calls the second function because width2 and height2 are double
        area = calculateArea(width2, height2);

        //calls the third function because only one double argument is passed
        area = calculateArea(radius);
    }

    public static double calculateArea(int width, int height){
        return (double) width * height;
    }
    public static double calculateArea(double width, double height){
        return width * height;
    }
    public static double calculateArea(double radius){
        return 2 * radius * Math.PI;
    }
}
```

Summary

Advantages

- Decomposition
- complex problems are broken into smaller, less complex problems
- increase in readability, clarity of structure
- parallel development possible
- possibility to reuse solutions already implemented
- information encapsulation

SimpleRecursion

Introduction

Smart catchphrase

To understand recursion you must first understand recursion!

Recursion in Math

- A problem or function is **recursive**, when it has self-reference.
- Example of a recursive definitions:
  - Fibonacci-Sequence
    The next element is the sum of its two predecessors in the sequence.
    $a_n := a_{n-1} + a_{n-2}, n \in N, n \geq 2$
  - Factorial
    $n! := n \cdot (n-1)!$
- Both definitions contain a base case, where the recursion stops
  - Fib: first two numbers in sequence are 1 ($a_0 = a_1 := 1$)
  - Fac: by definiton facotiral of zero is one (0! := 1$$)
- Without base cases, the recursion is endless!

Recursion in programming

Before we begin

WARNING The following section is about studying the technical (programming) aspects of recursion. The examples are actually anti-examples as the recursive solution is rather inefficient – all of them have a much better iterative solution

Recursive Function(s)

- A function is called **recursive** if is calls itself
- See a recursive implementation of factorial

```
public static int fac(int n) {
    if ( n == 0 ) {
        return 1;
    }else {
        return n * fac(n-1);
    }
}
```

Call stack

```
public static void main(String[] args) {
    int fac_4 = fac(4);
}
```

- `fac()` calls itself repeatedly (recursively)

```
main() -> fac(4) -> fac(3) -> fac(2) -> fac(1) -> fac(0)
       <- 24     <- 6      <- 2      <- 1      <- 1

24 =     (4      * (3     * (2       * (1      * (1)))))
```

- All function calls are active until `fac(0)` produces no further recursive call

```
- `main()` calls and waits for `fac(4)`
  - `fac(4)` calls and waits for `fac(3)`
    - `fac(3)` calls and waits for `fac(2)`
      - `fac(2)` calls and waits for `fac(1)`
        - `fac(1)` calls and waits for `fac(0)`
          - `fac(0)` returns 1 and closes
        - `fac(1)` receives 1, returns 1 and closes
      - `fac(2)` receives 1, returns 2 and closes
    - `fac(3)` receives 2, returns 6 and closes
  - `fac(4)` receives 6, returns 24 and closes
- `main()`receives 24
```

- when recursion breaks there are 5 active calls to `fac()`
- we say **recursion depth** is 5
- each call has its own copy of parameter `n`
- paramater `n` has different values in all of them
- recursion must end at some point, otherwise it is **infinite recursion**
- with finite resources (e.g. memory) infinite recursion is not possible - in practice, it leads to **Stack overflow** (try it out!)

SimpleRecursion_MiniExercise

- implement a recursive function `countdown(int n)` that counts from `n` to zero and prints "boom!" in the end

```
5
4
3
2
1
0
boom!
```

- implement a recursive function `thereAndBackAgain(int n)` that counts from `n` to zero and back to `n`

```
2
1
0
1
2
```

Recursive Fibonacci

Code and Output

```
private static int fib(int n) {
    if (n<=2) return 1;
    return fib(n-1) + fib(n-2);
}

public static void main(String[] args) {
    for (int i=1; i<10; i++)
        System.out.print(""+fib(i)+" ");
}
```

```
1 1 2 3 5 8 13 21 34
```

Analysis

- assume `fib(8)` , then the following calls are created

```
- fib(8)
  - fib(7)
    - fib(6)
      - ...
      - ...
    - fib(5)
      - ...
      - ...
  - fib(6)
    - fib(5)
      - ...
      - ...
    - fib(4)
      - ...
      - ...
```

- Make a table which call is performed how often

Table 19: call count for `fib(8)`

call	count
`fib(8)`	1
`fib(7)`	1
`fib(6)`	2
...	...
`fib(0)`	???

- How many calls to `fib()` are created in total?
- What is the maximum recursion depth?

Towers of Hanoi

Introduction

- Towers of Hanoi is a puzzle with an extremely concise recursive solution.
- We will study the problem first, then implement the solution



Figure 17: Towers of Hanoi with 8 discs on the middle peg

- 3 pegs (named A, B, C)
- a number of discs of different sizes
- initially all discs are on peg A, sorted by size, largest on the bottom, the smallest (disc 1) on top
- the task is to move all discs to peg C, however
  - you may only move one disc at a time
  - you may only move the topmost disc
  - you may place discs only on larger discs or empty pegs

Try it out! [5]

[5] Rod Pierce. "Play Tower of Hanoi" Math Is Fun. Ed. Rod Pierce. 27 Jun 2018. 6 Aug 2020 http://www.mathsisfun.com/games/towerofhanoi.html

Towers of Hanoi with a single disc

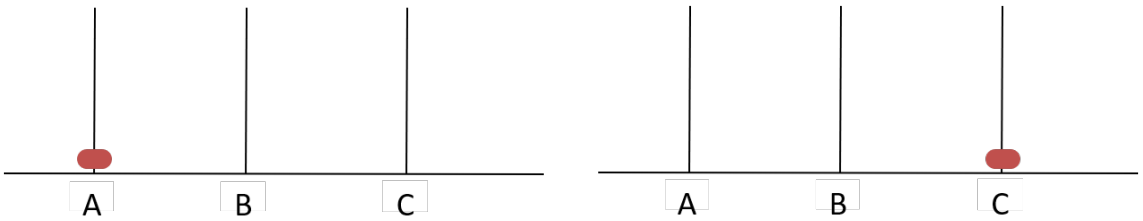


Figure 18: Playing Towers of Hanoi with a single disc is trivial

Towers of Hanoi with two discs

- Move smallest on temporary hold (B)
- Largest disk to target (C)
- Move smallest to target (C)

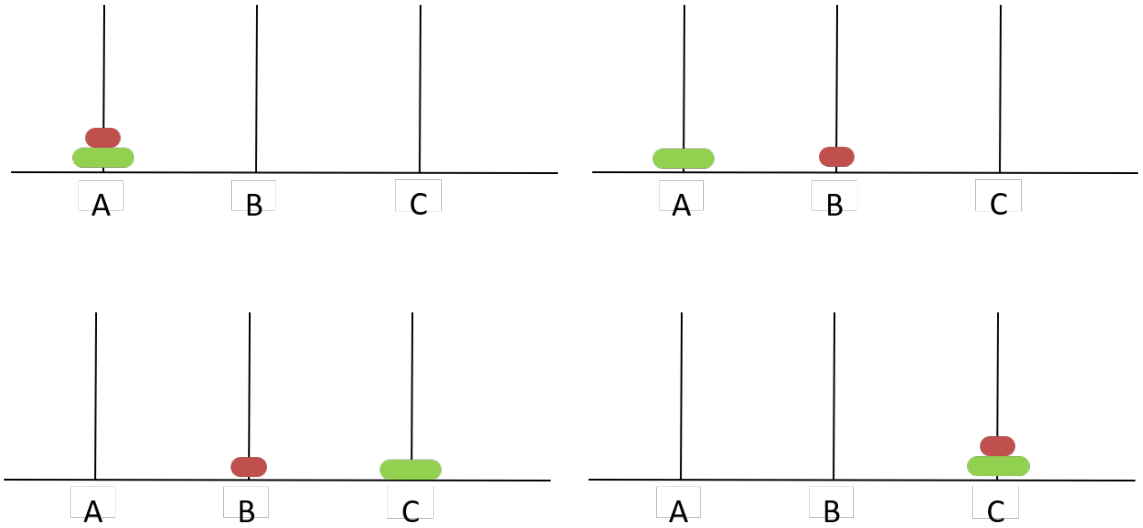


Figure 19: Playing Towers of Hanoi with two discs.

Towers of Hanoi with three discs

- Move the two smallest on temporary hold (B) (and solve this (smaller) problem with the same principle)
- Largest disk to target (C)
- Move the two smallest to target(C)

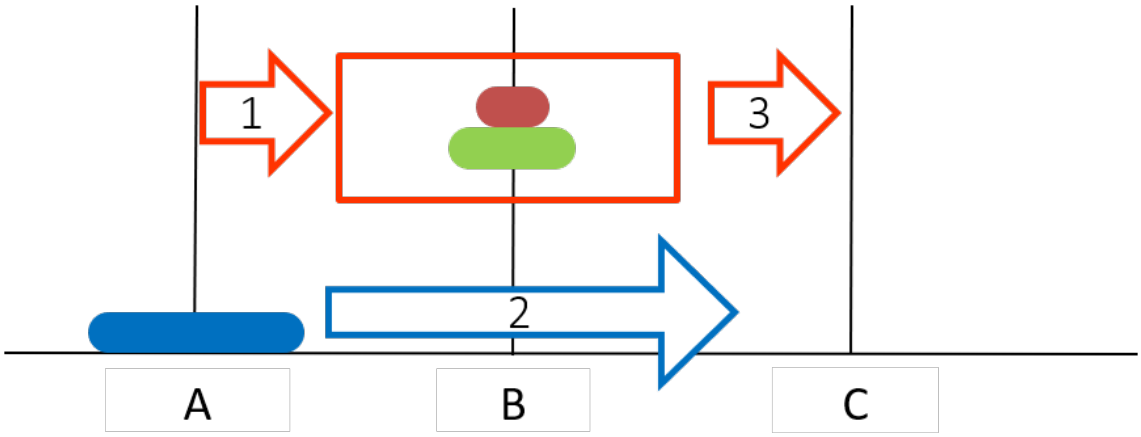


Figure 20: Playing Towers of Hanoi with three discs. Overall strategy

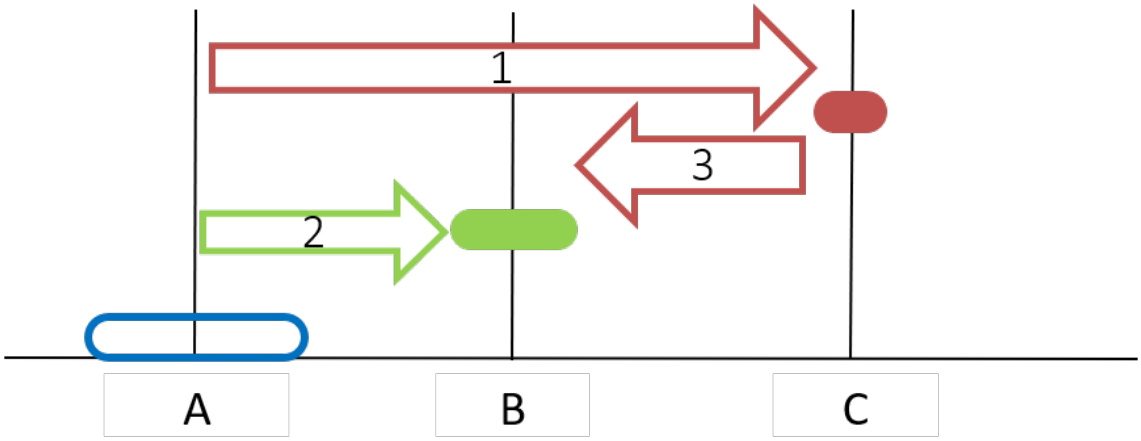


Figure 21: Playing Towers of Hanoi with three discs. Solving the smaller problem

Towers of Hanoi with more discs

Q: Can Towers of Hanoi be solved for any number of discs?

A: yes!

- we will inductively prove in two parts
  - Part 1: if we can solve for n discs, we can also solve for n+1 discs
  - Proof:
    - move n discs to the third peg (possible by assumption, only pegs switch role)
    - move largest disc to target peg
    - move n discs from third peg to target peg (possible by assumption, the largest disc makes no problems, since all other discs are smaller)
  - Part 2: we can move 1 disc (trivial)
  - by induction we conclude that we can solve for any (natural) number of discs

How to solve for n discs

- move n-1 discs to temporaray peg
- move largest to target peg
- move n-1 discs from temporary to target

How to solve for n-1 discs

- move n-2 discs to temporary peg
- move largest (of the n-1) to target peg
- move n-2 discs from temporary to target

How to solve for n-2 discs

- move n-3 . . .
- . . .

and so on

Towers of Hanoi Code

```
public static void towersOfHanoi(int n,
                                 String from,
                                 String to,
                                 String temp) {
    if (n == 1) {
        System.out.printf("Moving disc 1 from %s to %s\n", from, to);
        return;
    }
    towersOfHanoi(n - 1, from, temp, to);
    System.out.printf("Moving disc %d from %s to %s\n", n, from, to);
    towersOfHanoi(n - 1, temp, to, from);
}
```

```
public static void main(String[] args) {
    towersOfHanoi(3, "A", "C", "B");
}
```

```
Moving disc 1 from A to C
Moving disc 2 from A to B
Moving disc 1 from C to B
Moving disc 3 from A to C
Moving disc 1 from B to A
Moving disc 2 from B to C
Moving disc 1 from A to C
```

Towers of Hanoi Analysis

Table 20: number of moves to solve Towers of Hanoi with n discs

n	moves
1	1
2	3
3	7
4	15
5	31
6	63
...	...
n	$2^n - 1$

- assuming 1s time to move one disc, then moving 64 discs takes $1.8 \cdot 10^{19} s$
- the age of the universe is about $10^{18} s$
- the solution is optimal

Arrays

Introduction

Motivation

- read and store an arbitrary number of user-input values

Example read10Values

```
1  public static void main(String[] args){
2      Scanner sc = new Scanner(System.in);
3      double mean = 0;
4
5      int[] v = new int[10];  //declare an array for 10 int values
6
7      //fill the array with user input values
8      for(int i=0; i<10; i++){
9          System.out.println("enter a number please (" + (i+1) + "/10)");
10         v[i] = sc.nextInt(); //assign a value to the i-th position in the array v
11     }
12
13     for(int i=0; i<10; i++){
14         // get the value at the i-th position
15         // of the array v and add it to mean
16         mean += v[i];
17     }
18     mean /= 10.0;
19     System.out.println("mean= "+ mean);
20     sc.close();
21 }
```

- in this example an integer array is created (`new int[10]`) with space for 10 integer values
- values can be assigned or read by indexing (`arrayName[index]`)

Example readXValues

```
1  public static void main(String[] args){
2      Scanner sc = new Scanner(System.in);
3      double mean = 0;
4
5      System.out.println("enter number of values to be entered");
6      int nValues = sc.nextInt();
7      int[] v = new int[nValues];  //declare an array of the size entered by the user
8
9      //v.length returns the size of the array
10     for(int i=0; i<v.length; i++){
11         System.out.println("enter a number please (" + (i+1) + "/" + nValues + ")");
12         v[i] = sc.nextInt();  //assign a value to the i-th position in the array v
13     }
```

```
14
15      for(int i=0; i<v.length; i++){
16          mean += v[i]; //get the value at the i-th position of the array v and add it to mean
17      }
18      mean /= (double)v.length;
19      System.out.println("mean= "+ mean);
20      sc.close();
21  }
```

- in this example the size of the array is determined during runtime

Theory

Overview

- an array is a data structure that combines several values (of the same type!)
- elements of an array are arranged one after another (they form a sequence)
- elements in the array are accessed by their position (index)

Declaration

- arrays can only contain values of one data type
- the type of the array is specified with its declaration:
  - `int[] numbers;`
  - `char[] alphabet;`
  - `Person[] contacts;`

Initialization

- `numbers = new int[10];`
- an array is created with the new keyword
- the size of the array must be known during creation and of type int
- intialization creates a reference to the array and it is stored in a variable (e.g. in numbers)
- all values in an array are initialized with the default value of the data type (0, false, null)
- an array can also be created with values: `int[] list = {1, 3, 5};`

Indexing

- position counting starts at 0
- the i-th value in an array has the index i-1 (e.g. the 12th element is accessed is at index-position 11)
- Compiler doesn't check if an index is in the bounds of an array, if it is not an ArrayIndexOutOfBoundsException occurs

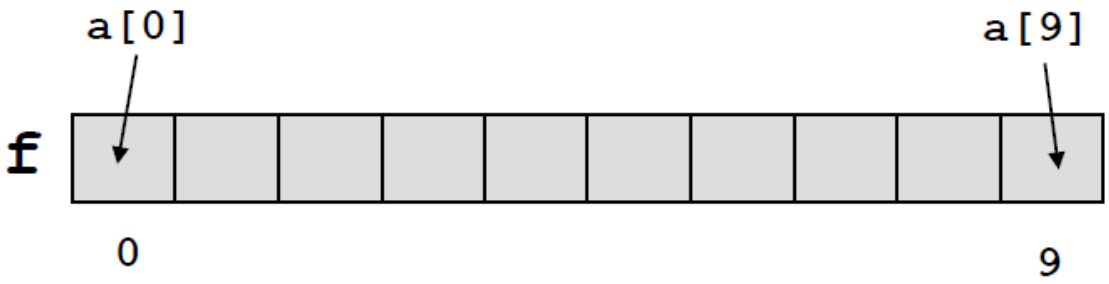


Figure 22: Array Indexing

Summary

Iterating over an array

```java
public static void main(String[] args){
    int[] a = {1, 3, 5, 7};

    //from left to right
    for(int i = 0; i < a.length; i++){
        System.out.printf("%2d", a[i]);
    }
    System.out.println();

    //from right to left
    for(int i = a.length - 1; i >= 0; i--){
        System.out.printf("%2d", a[i]);
    }
}
```

```
 1 3 5 7
 7 5 3 1
```

Short Summary

- Array creation:
  - `double[] arr = new [size];`
  - `int[] arr = { 1, 3, 5};`
- get size of an array:
  - `int a = arr.length;`
- Access elements with []:
  - First index: `0`
  - Last index: `length - 1`
- reading a value:
  - `a = arr[i];`
- setting a value:
  - `arr[i] = value;`
  - `arr[i]++;`

ArraysandFunctions

Arrays as function parameters

Arrays are reference types

- Java passes arguments as copies of the originals (pass-by-copy)
  - primitive variables are copied, when used as an argument in a function
  - the copy is a local variable in the function
  - original value can't be changed in the function
  - Arrays are passing a copy of their reference, all changes made to the elements of an array passed to a function affect the original array
  - Java is pass-by-copy, i.e. references are copied
  - a copied reference is still a handle to the original data

Example addTwo I

```
1  public class Main {
2      public static void main(String[] args){
3          int[] myArray = {23, -3, 5, 7, 11};
4
5          addTwo(myArray);
6
7          for(int i=0; i<myArray.length; i++)
8              System.out.printf("%3d", myArray[i]);
9      }
10
11     public static void addTwo(int[] parArray){
12         for(int i=0; i<parArray.length; i++)
13             parArray[i] += 2;
14     }
15 }
```

```
25 -1  7  9 13
```

Example addTwo II

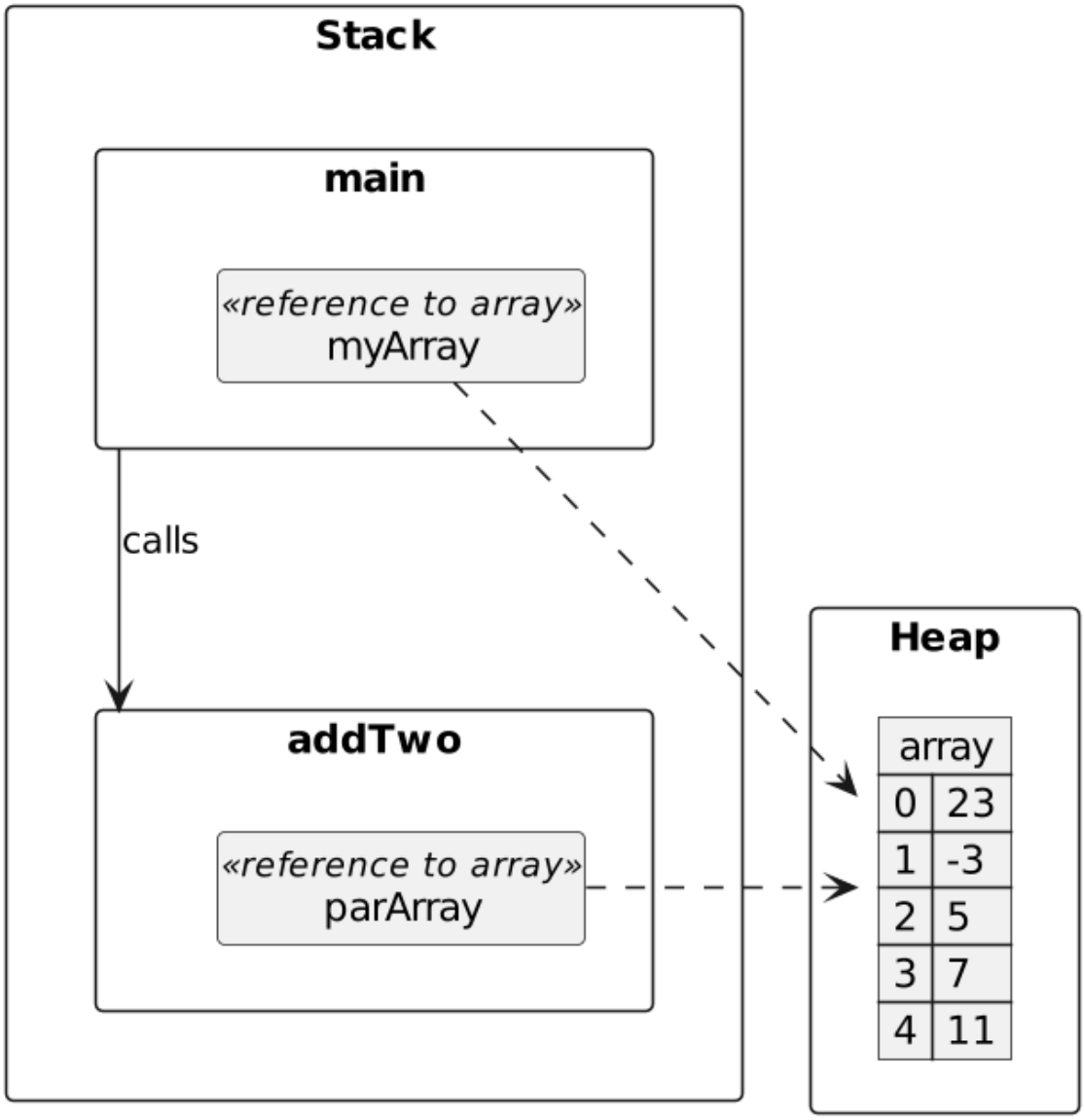


Figure 23: The copy of the reference to an array is passed to a function

Arrays as return type

- Arrays can be returned by functions

Example average3 I

```
1 public class Main {
2     public static void main(String[] args){
3         double[] myArray = {23, -3, 5, 7, 46, 11};
4
```

```
5           display(myArray);
6           display(average3(myArray));
7       }
8
9       public static void display(double[] a) {
10          for(int i=0; i<a.length; i++)
11              System.out.printf("%6.1f", a[i]);
12          System.out.println();
13      }
```

Example average3 II

```
15      public static double[] average3(double[] a){
16          if(a.length == 0)
17              /*empty array*/
18              return null;
19
20          double[] b = new double[a.length];
21
22          if(a.length == 1) {
23              /*one element only*/
24              b[0] = a[0];
25              return b;
26          }
27
28          /* special case: begin*/
29          b[0] = (a[0] + a[1]) / 2.0;
30
31          for(int i=1; i < a.length - 1; i++) {
32              /* general case*/
33              b[i] = (a[i-1] + a[i] + a[i+1]) / 3.0;
34          }
35
36          /* special case : end*/
37          b[a.length-1] = (a[a.length-2] + a[a.length-1]) / 2.0;
38
39          return b;
40      }
41  }
```

Output average3

```
23.0  -3.0   5.0   7.0  46.0  11.0
10.0   8.3   3.0  19.3  21.3  28.5
```

For-each

For-each I

- compact, type safe syntax, if index information is not required
- “for each element in a”

```
public static void display(double[] arr) {
    for(double element : arr){
        System.out.printf("%6.1f", element);
    }
    System.out.println();
}
```

- read-only access
- elements in arr are not changed

For-each II

```
public class Main {
    public static void main(String[] args){
        int[] arr = {23, -3, 5, 7, 46, 11};

        for(int e : arr){
            e++; //only e changes not the element in arr
            System.out.printf("%3d", e);
        }
        System.out.println();
        for(int e : arr){
            System.out.printf("%3d", e);
        }
    }
}
```

```
24 -2  6  8 47 12
23 -3  5  7 46 11
```

MultidimensionalArrays

2D Arrays

Concept

- conceptually, a two-dimensional array can be thought of as a table
- a value is specified by two indices

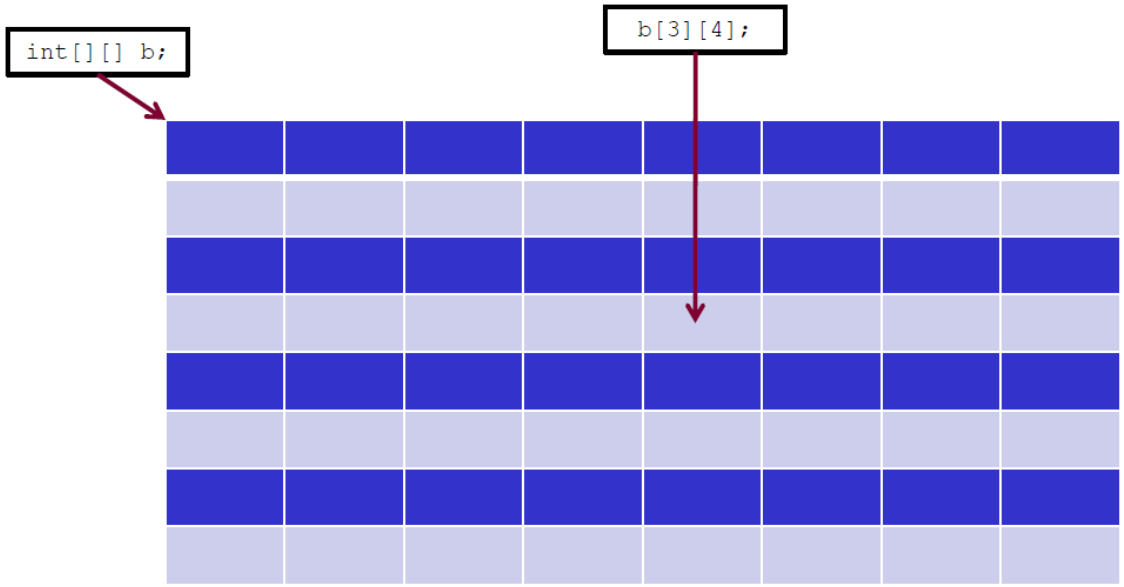


Figure 24: 2D array as table

Concept

- declaration
  - `typename[][];`
  - `int table[][];`
- declaration + initialization

```
char chessBoard[][]={
 {'r','s','l','q','k','l','s','r'}, \\
 {'p','p','p','p','p','p','p','p'}, \\
 {' ',' ',' ',' ',' ',' ',' ',' '}, \\
 {' ',' ',' ',' ',' ',' ',' ',' '}, \\
 {' ',' ',' ',' ',' ',' ',' ',' '}, \\
 {' ',' ',' ',' ',' ',' ',' ',' '}, \\
 {'P','P','P','P','P','P','P','P'}, \\
 {'R','S','L','Q','K','L','S','R'}};
```

- creation
  - `int table[][] = new int[10][10];`

- access
  - `table[rowIdx][colIdx]`
  - `chessboard[8][8];`
- Java does not support 2-dimensional arrays directly
- They are realized as a 1-d array of 1-d arrays (or as a 1-d array of references to 1-d arrays of a certain type)

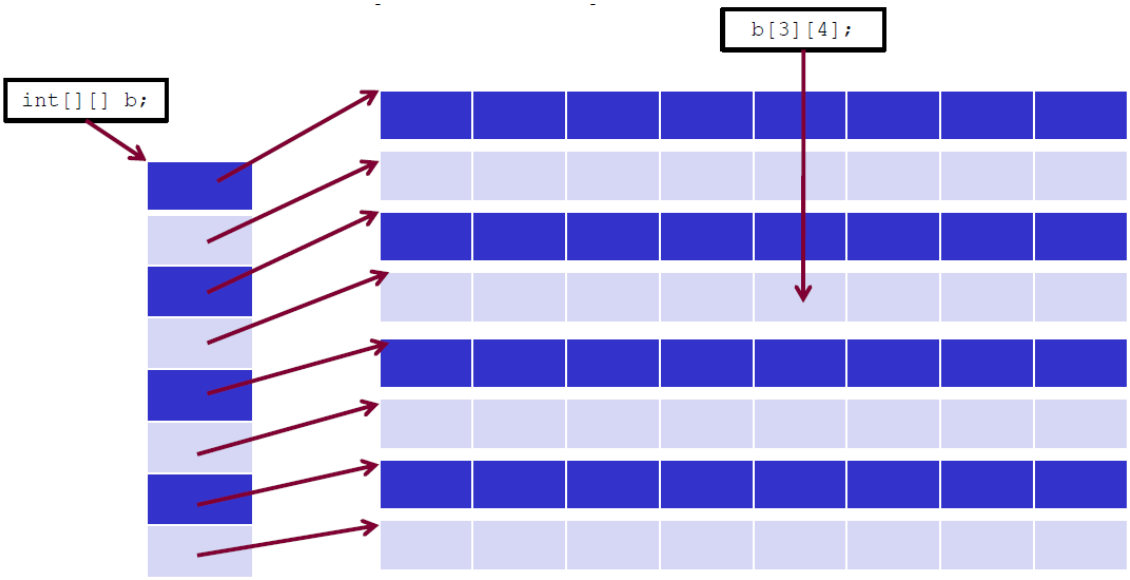


Figure 25: 2D array as one array of eight int arrays

Iterating over 2D array

```
public class MultiplicationTable {
    public static void main(String[] args){
        int[][] table = new int[5][10];

        for(int i = 0; i < table.length; i++) {
            for(int j = 0; j < table[0].length; j++) {
                table[i][j] = (i+1)*(j+1);
            }
        }

        display(table);
    }

    private static void display(int[][] a) {
        for(int i = 0; i < a.length; i++) {
            for(int j = 0; j < a[0].length; j++) {
                System.out.printf("%3d", a[i][j]);
            }
            System.out.println();
```

```
        }
    }
}
```

MultiplicationTable Output

```
 1  2  3  4  5  6  7  8  9 10
 2  4  6  8 10 12 14 16 18 20
 3  6  9 12 15 18 21 24 27 30
 4  8 12 16 20 24 28 32 36 40
 5 10 15 20 25 30 35 40 45 50
```

Multidimensional Arrays

More than 2D

- more then 2 dimensions are possible:
  - `int[][][] cube;`
  - `int[]...[] hypercube;`
- the same concept as for 2D arrays applies
- a 3D int array is a 1d-array of references to 1d-arrays of references to 1d-arrays of ints

Sort

Introduction

Motivation

- sorting enables advanced search algorithms like binary search (complexity of $\mathcal{O}(log(N))$ that increase data access in a list of elements drastically
- without a sorted list every element has to be compared (complexity $\mathcal{O}(N)$)
- sorting a collection of elements is a one time effort, that speeds up all future searches
- sorting an unsorted collection of elements is not trivial therefore different sorting algorithms exists to optimise this process

Prerequisites

- the following Java implementations of sorting algorithms assumes:
  - data is numerical
  - data is to be sorted ascending (smaller first)
  - data is stored in an array
  - the algorithms use a swap operation to change the position of two elements in an array:

```
public static void swap(int arr[], int i, int j) {
    // save the value at position i in tmp variable
    int tmp = arr[i];
    // override the value at position i with the value at position j
    arr[i] = arr[j];
    // override the value at position j with the tmp value
    arr[j] = tmp;
}
```

BubbleSort

BubbleSort Algorithm

- compare neighbors and swap if necessary

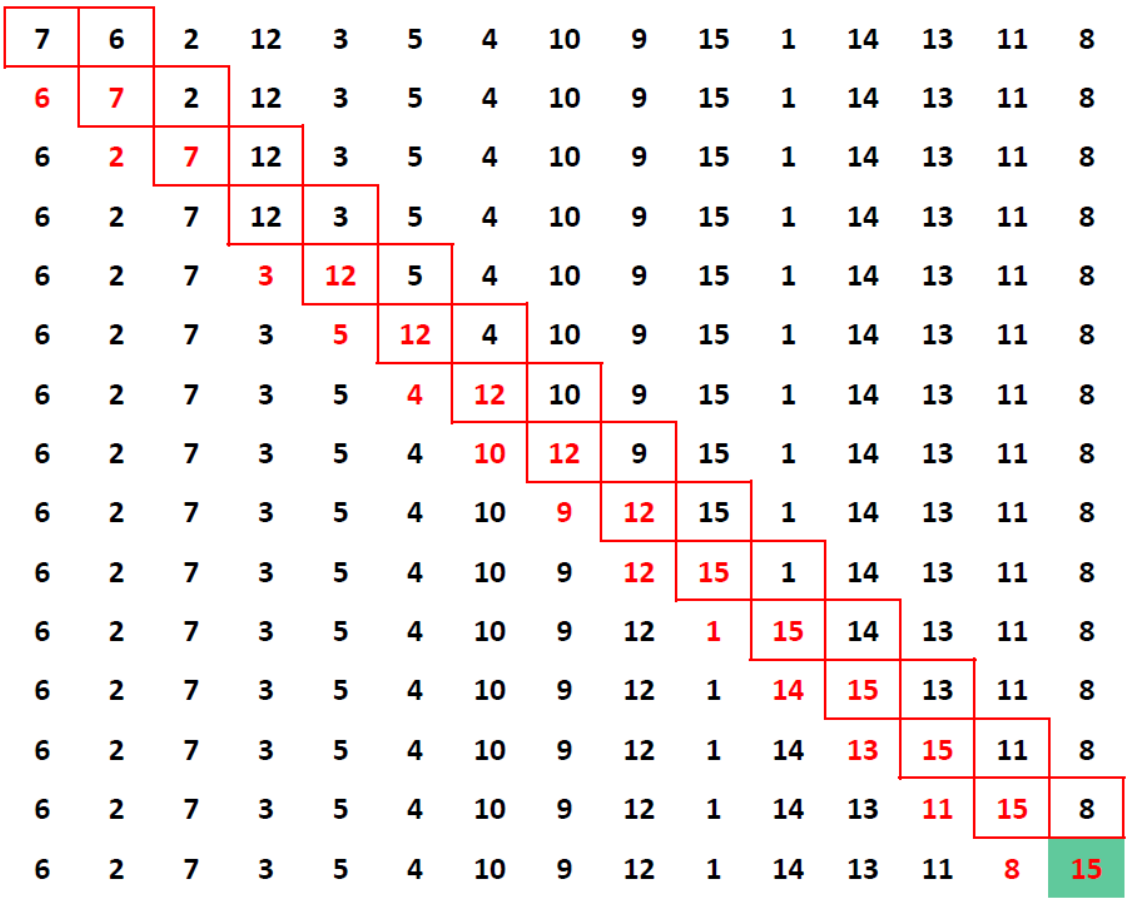


Figure 26: BubbleSort single run

- after the first run it is guaranteed that the last element is the biggest element
- the second run therefore only has to go to the second last element and the two biggest elements are at the end
- every following run has to compare one element less
- this process is repeated until only one element is left
- the bubble sort algorithm can be stopped early once there was no swap operation performed during one complete run, since this means all elements are already in order

---

6	2	7	3	5	4	10	9	12	1	14	13	11	8	15
2	6	3	5	4	7	9	10	1	12	13	11	8	14	15
2	3	5	4	6	7	9	1	10	12	11	8	13	14	15
2	3	4	5	6	7	1	9	10	11	8	12	13	14	15
2	3	4	5	6	1	7	9	10	8	11	12	13	14	15
2	3	4	5	1	6	7	9	8	10	11	12	13	14	15
2	3	4	1	5	6	7	8	9	10	11	12	13	14	15
2	3	1	4	5	6	7	8	9	10	11	12	13	14	15
2	1	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
							...							
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 27: BubbleSort complete run

BubbleSort Implementation

```
public static void bubble(int numbers[]) {
    boolean swapped;

    /* number of completed runs */
    int i=0;

    /* this loop is executed as long as there    */
    /* was at least one swap operation performed */
    /* at maximum the loop is executed N-1 times */
    do {

        /* no swap operation was performed at the beginning of a run */
        swapped = false;

        /* with each run one more number is fixed and */
        /* has not to be checked in the next run (-i) */
        for (int j = 1; j < numbers.length - i; j++){

            /* if the left number is smaller then swap */
            if (numbers[j - 1] > numbers[j]) {
                swap(numbers, j - 1, j);
                swapped = true;
            }
        }
        i++;
```

```
    } while (swapped);
}
```

BubbleSort Analysis

- what is the number of comparisons?
- best case:
  - the list is already sorted, the algorithm would have to complete one complete run to be sure that no swaps are necessary
  - in the best case $N - 1$ operations are necessary
  - the complexity is $\mathcal{O}(N)$
- worst case:
  - the list is inverted
  - the complexity is $\mathcal{O}(N^2)$
- average case:
  - the complexity is still $\mathcal{O}(N^2)$
- the algorithm is stable (if the list contains duplicate values the original order of these equal values is guaranteed)
- the algorithm can be performed in-place (it doesn't need additional memory to store intermediate results)

InsertionSort

InsertionSort Algorithm

- insert the first element of the unsorted partition at the correct position in the sorted partition

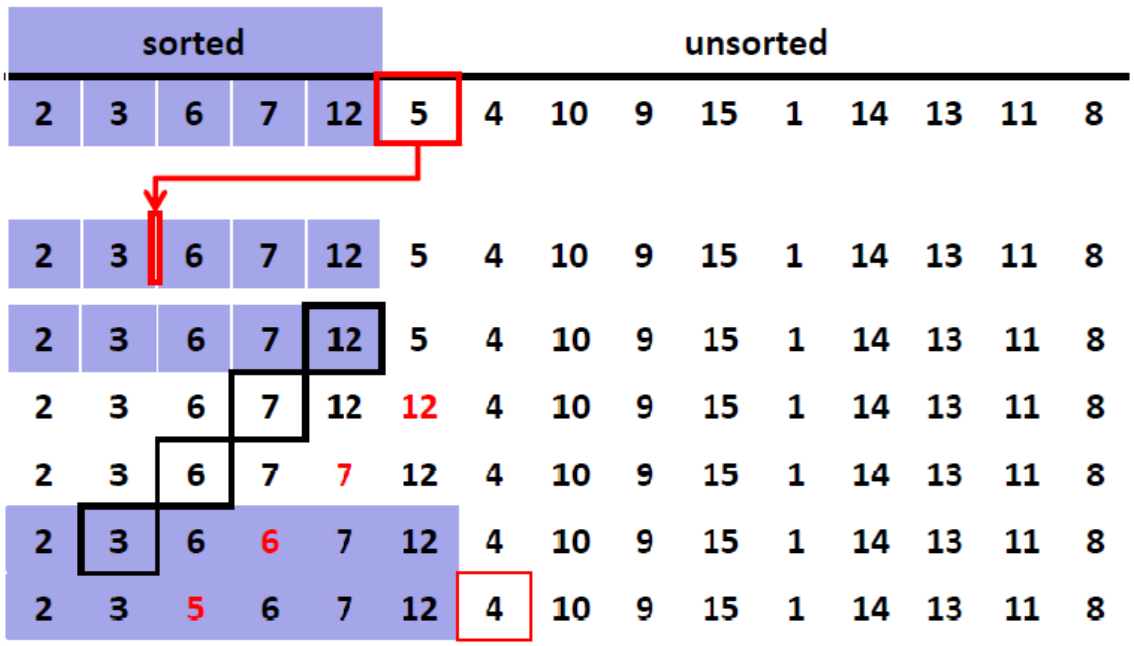


Figure 28: InsertionSort - insertion of one element

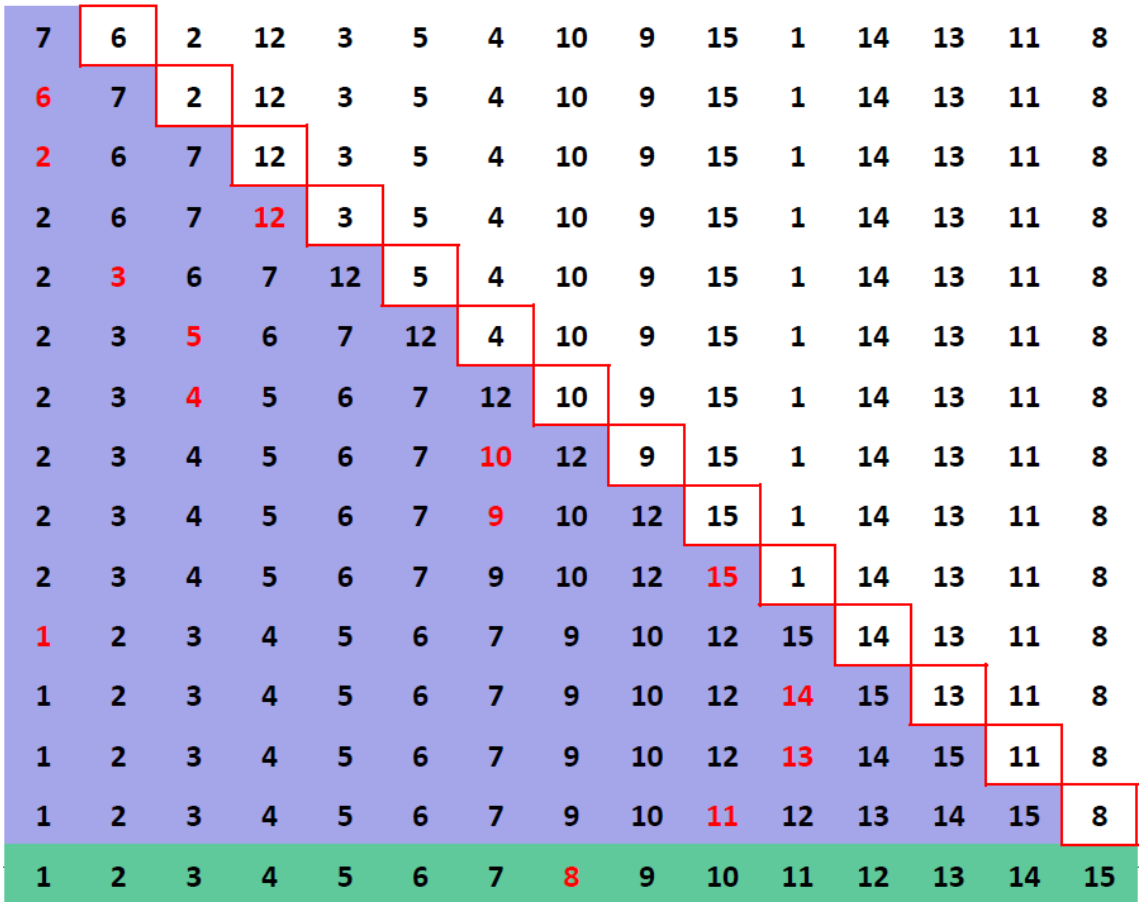


Figure 29: InsertionSort - complete run

InsertionSort Implementation

```
public static void insertion(int numbers[]) {

    /* i is the first idx of unsorted partition */
    for (int i=1; i<numbers.length; i++) {
        int j=i;

        /* tmp holds value of next element to insert */
        int tmp=numbers[i];

        /* find correct position for tmp */
        while (j>0 && tmp<numbers[j-1]) {

            /* on the way there shift right */
            numbers[j]=numbers[j-1]; j--;
        }
        numbers[j]=tmp;
```

```
    }
}
```

InsertionSort Analysis

- what is the number of comparisons?
- best case (sorted): $\mathcal{O}(N)$
- worst case (inverted): $\mathcal{O}(N^2)$
- average case: $\mathcal{O}(N^2)$
- the algorithm is stable
- the algorithm can be performed in-place

SelectionSort

SelectionSort Algorithm

- select the smallest element of the unsorted partition and swap it the with first element of unsorted partition

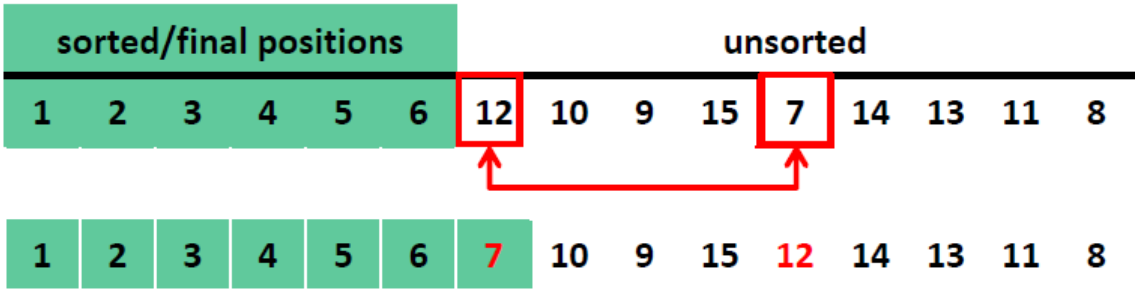


Figure 30: SelectionSort - sorting one element

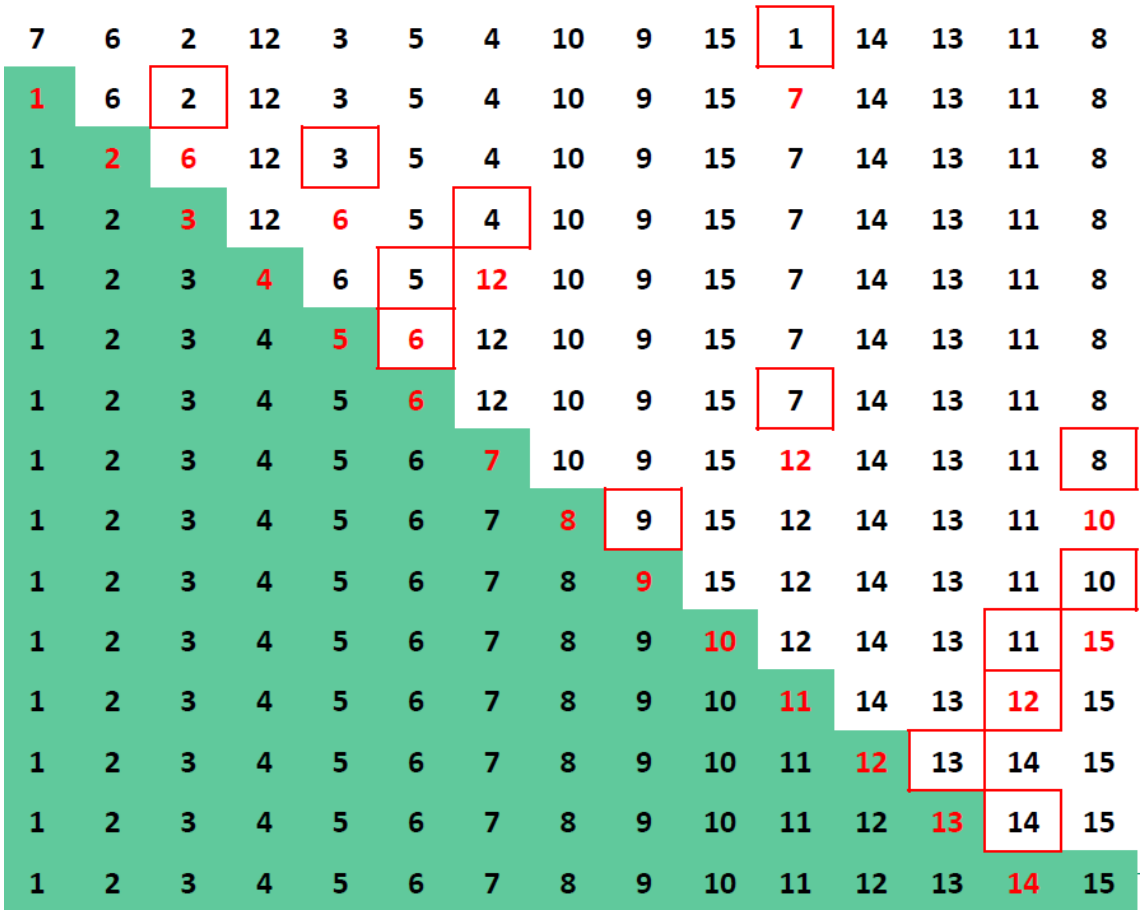


Figure 31: SelectionSort - complete run

SelectionSort Implementation

```
public static void selection(int numbers[]) {
    int i, j;

    for (i = 0; i < numbers.length - 1; i++) {
        int min_idx = i;

        /* find idx of minimum of unsorted partition */
        for (j = i + 1; j < numbers.length; j++) {
            if (numbers[j] < numbers[min_idx])
                min_idx = j;
        }

        /* swap minimum to correct position */
        swap(numbers, i, min_idx);
    }
}
```

SelectionSort Analysis

- what is the number of comparisons?
- best case (sorted): $\mathcal{O}(N^2)$
- worst case (inverted): $\mathcal{O}(N^2)$
- average case: $\mathcal{O}(N^2)$
- the algorithm is stable
- the algorithm can be performed in-place

MergeSort

MergeSort Algorithm

- split the data into two subsets and sort them (again with mergesort)
- merge the sorted subsets into one

7	6	2	12	3	5	4	10	9	15	1	14	13	11	8
6	7	2	12	3	5	4	10	9	15	1	14	13	11	8
6	7	2	12	3	5	4	10	9	15	1	14	13	11	8
2	6	7	12	3	5	4	10	9	15	1	14	13	11	8
2	6	7	12	3	5	4	10	9	15	1	14	13	11	8
2	6	7	12	3	5	4	10	9	15	1	14	13	11	8
2	6	7	12	3	4	5	10	9	15	1	14	13	11	8
2	3	4	5	6	7	10	12	9	15	1	14	13	11	8
2	3	4	5	6	7	10	12	9	15	1	14	13	11	8
2	3	4	5	6	7	10	12	9	15	1	14	13	11	8
2	3	4	5	6	7	10	12	1	9	14	15	13	11	8
2	3	4	5	6	7	10	12	1	9	14	15	11	13	8
2	3	4	5	6	7	10	12	1	9	14	15	11	13	8
2	3	4	5	6	7	10	12	1	9	14	15	8	11	13
2	3	4	5	6	7	10	12	1	8	9	11	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 32: MergeSort - complete run

MergeSort Implementation I

- the algorithm is implemented recursive, mSort calls it self with smaller splits until the splits are to small to split again.
- then the smaller sorted splits are merged

MergeSort Implementation II

```
public static void mergesort(int[] a) {
    /* create an auxilliar array that is used be the merge function */
    int[] aux = new int[a.length];

    /* starts the mergesort by calling the recursive function mSort */
    mSort(a, 0, a.length - 1, aux);
}

public static void mSort(int numbers[], int left, int right, int[] aux) {
    /* stops the recursion when portion can't be splited further */
    if (right <= left) return;

    /* call mSort recursively with the left and the right split */
    int middle = (right + left) / 2;
    mSort(numbers, left, middle, aux);
    mSort(numbers, middle + 1, right, aux);

    /* merge is first executed for the smallest splits */
    merge(numbers, left, middle, right, aux);
}
```

MergeSort Implementation III

```
public static void merge(int numbers[], int left, int middle, int right, int[] aux) {
    /* fill the helper array with the numbers between left and right */
    for (int i = left; i <= right; i++)
        aux[i] = numbers[i];
    /* the l counter starts at the left position and goes till the middle */
    int l = left;
    /* the r counter starts at the position middle+1 and goes till right position*/
    int r = middle+1;

    /* sorts the elements in the numbers array */
    for (int i = left; i <= right; i++){
        /* if the left element is smaller take it otherwise take */
        /* the right one and increase the respective counter     */
        if (r > right || (l <= middle && aux[l] < aux[r]))
            numbers[i] = aux[l++];
        else
            numbers[i] = aux[r++];
    }
}
```

MergeSort Analysis

- what is the number of comparisons?
- best case (sorted): $\mathcal{O}(Nlog(N))$
- worst case (inverted): $\mathcal{O}(Nlog(N))$
- average case: $\mathcal{O}(Nlog(N))$
- the algorithm is stable
- the algorithm needs additional memory in the size of the array that is to be sorted
- the algorithm is implemented recursive (“divide and conquer” strategy)

QuickSort

QuickSort Algorithm

- select an element and move it to its correct place
- sort all data left of it and right of it in the same fashion

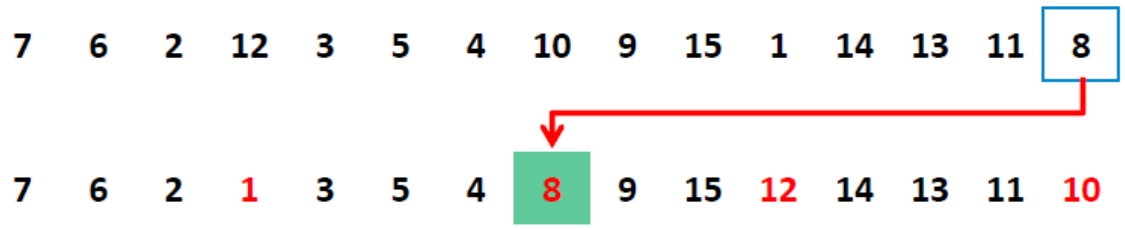


Figure 33: QuickSort - sort one element

- moving the pivot element to the correct place is called partition step

QuickSort Partition step

- a selected pivot element (for expmaple the last element) is moved to the correct position
- the correct position is found when all elements to the left are smaller and all elements to the right are bigger (although still unsorted)

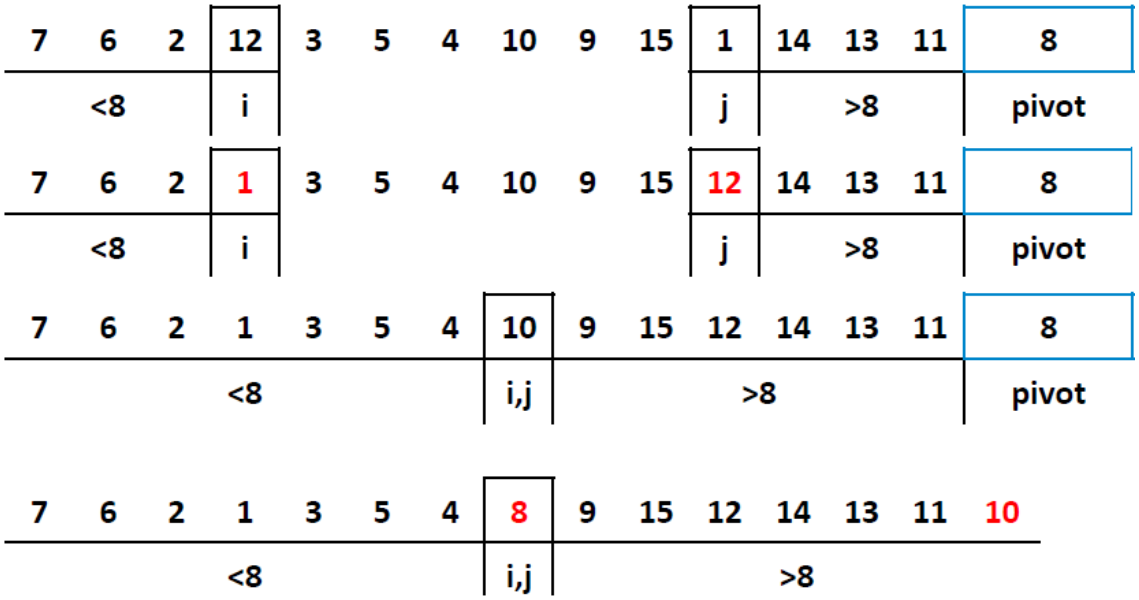


Figure 34: QuickSort - sorting elements left and right from pivot element

---

- sorting with pivot element:
  - one index starts from the left, stops on first element>pivot
  - one index starts from the right, stops on first element<pivot
  - the two numbers are swapped
  - the correct position of the pivot is found, when indices run into each other. pivot is then swapped with there
  - partition is complete (all left<pivot, all right>pivot)
- the two partitions (left and right) are then sorted with the same algorithm

---

7	6	2	1	3	5	4	8	9	15	12	14	13	11	10
3	1	2	4	7	5	6	8	9	15	12	14	13	11	10
1	2	3	4	7	5	6	8	9	15	12	14	13	11	10
1	2	3	4	7	5	6	8	9	15	12	14	13	11	10
1	2	3	4	5	6	7	8	9	15	12	14	13	11	10
1	2	3	4	5	6	7	8	9	15	12	14	13	11	10
1	2	3	4	5	6	7	8	9	15	12	14	13	11	10
1	2	3	4	5	6	7	8	9	10	12	14	13	11	15
1	2	3	4	5	6	7	8	9	10	12	14	13	11	15
1	2	3	4	5	6	7	8	9	10	12	14	13	11	15
1	2	3	4	5	6	7	8	9	10	11	14	13	12	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Figure 35: QuickSort - complete run

QuickSort Implementation: partition

```
public static int partition(int numbers[], int left, int right) {
    /* sorting an array of length 1 is trivial */
    if (left == right) return left;

    /* the right element is selected as pivot */
    int pivot = numbers[right];
    int i = left - 1, j = right; /* search indices from left and right */

    while (true) {
        /* search for the first element bigger then pivot from the left */
        while (numbers[++i] < pivot)
            ;

        /* search for the first element smaller then pivot from the right */
        while (pivot < numbers[--j])
            if (j == left) /* stop at left search index */
                break;

        /* pivot positon is found and the loop is stopped */
        if (i >= j)
            break;

        /* swap the bigger element on the left side with */
        /* the smaller element on the right side         */
```

```
            swap(numbers, i, j);
        }
        /* swap the pivot element */
        swap(numbers, i, right);
        /* return the new position of the pivot element */
        return i;
}
```

QuickSort Implementation: quicksort

- the algorithm is implemented recursive, `quicksort()` calls it self with smaller splits until the splits are too small to split again.
- the split is determined by a preceding call to `partintion()`

```
public static void quicksort(int numbers[]) {
    /* starts the quicksort algorithm */
    quicksort(numbers, 0, numbers.length-1);
}

public static void quicksort(int numbers[], int left, int right) {
    if (right<=left)
        return;

    /* partition returns the new position of the pivot element */
    int i=partition(numbers, left, right);

    /* quicksort is then executed again for the left part ... */
    quicksort(numbers, left, i-1);

    /* ... and for the right part */
    quicksort(numbers, i+1, right);
}
```

QuickSort Analysis

- what is the number of comparisons?
- best case (pivot splits range in equal halves): $\mathcal{O}(Nlog(N))$
- worst case (pivot is always largest or smallest; sorted): $\mathcal{O}(N^2)$
- average case: $\mathcal{O}(Nlog(N))$
- the algorithm is unstable
- the algorithm can be performed in-place
- the algorithm is implemented recursive (“divide and conquer” strategy)

Search

Linear/Binary search

Motivation

- Think of looking up a person in a telephone book
- (the name of) the person we are looking for is called key
- The telephone book is a sequence of entries (sorted by name!)

- there are (at least) two ways of finding the phone number (the value) for the person (the key):
  - starting with the first entry, each entry is compared with the key until the key is found or the end of the book is reached (which means the person is not listed)
  - starting from the the middle page and the first entry on that page is compared with the key
    - if it is the key, we are done
    - if the key is before that entry, the procedure is repeated with the first half of the book
    - if the key is after that entry, the procedure is repeated with the second half of the book

---

- the first method will for most cases take much longer than the second method
- the time complexity can be descibed mathematically
  - count or estimate the number of steps (operations) required to find the key (or to be sure it is not in the sequence)

Linear/Binary Time complexity

- Assume we have to find a person in a telephone book with twice as many entries than the first one
- the first method takes about twice as long
- the second method takes one more step
  - we open the middle page and continue with one half, which means that after one step we are searching a sequence of same length as the first book!
- The Big-O concept captures that notion mathematically

Linear search

Example

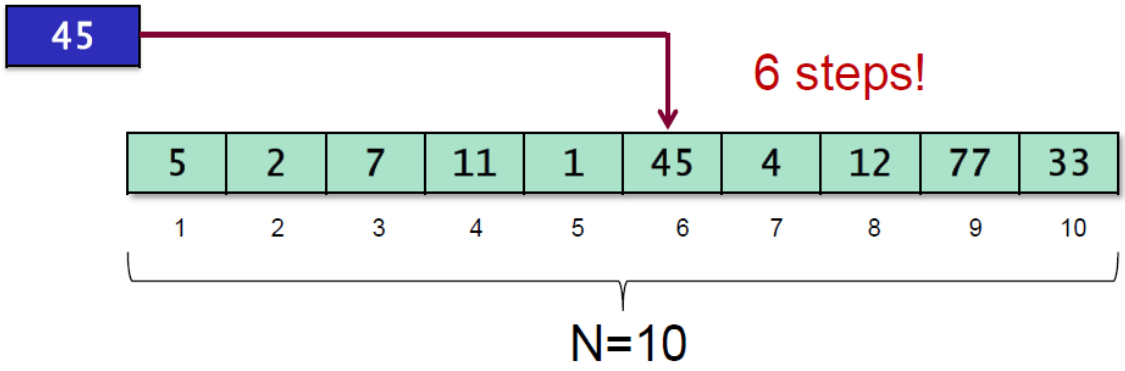


Figure 36: Example linear search with N=10 unsorted elements

Analysis

- Algorithm: iterate over elements until key found or end of sequence reached
- best case
  - first element tested is key
- worst case
  - all (N) elements must be tested
  - this is the case when the key is the last element or the key is not found
- average case
  - $\sum_{i=1}^{N} p_i \cdot i$
  - sum off the probabilty of each case times the cost for that case
  - applying this formular the expected number of steps is: $\frac{N+1}{2}$
  - for large N, the expected number of steps is about N/2
- doubling the number of elements (roughly) doubles the number of steps

Binary search

Binary search Analysis

- can only be used with sorted sequences
- algorithm:
  - compare key K with middle element m
  - if K=m: found (exit)
  - if K>m: repeat algorithm for right half
  - if K<m: repeat algorithm for left half

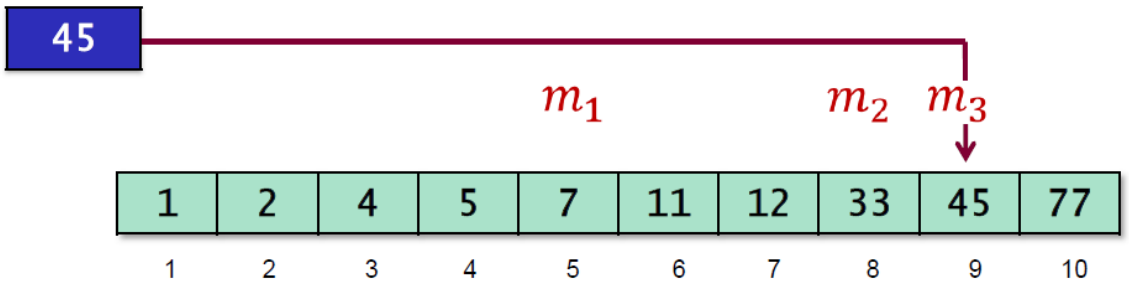


Figure 37: Example binary search with 10 sorted elements

- best case
  - first element tested is key
- worst case
  - key is last element tested or not found
  - interval must be halved repeatedly until one single element is left
  - maximum number of steps: $log_2(N)$
- average case
  - expected number of steps is one less than maximum number of steps
- doubling the number of elements increases the number of steps by one

Implementation in Java

```
public int binarySearch(int sortedArray[], int x) {
    int l = 0;
    int r = sortedArray.length - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;

        // Check if value x is found at position m
        if (sortedArray[m] == x)
            return m;

        // If x is greater, ignore left half
        if (sortedArray[m] < x){
            l = m + 1;
        }
        // If x is smaller, ignore right half
        else{
            r = m - 1;
        }
    }

    // if the value x is not in the array we end up here
    return -1;
}
```

Big-O Notation

Time complexity

- for large values of N it is sufficient to estimate the order of time complexity (rather than the exact number of steps)
- the time complexity of an algorithm is expressed in the Big-O notation
- constant factors are not considered in this notation since they have a relatively low impact

---

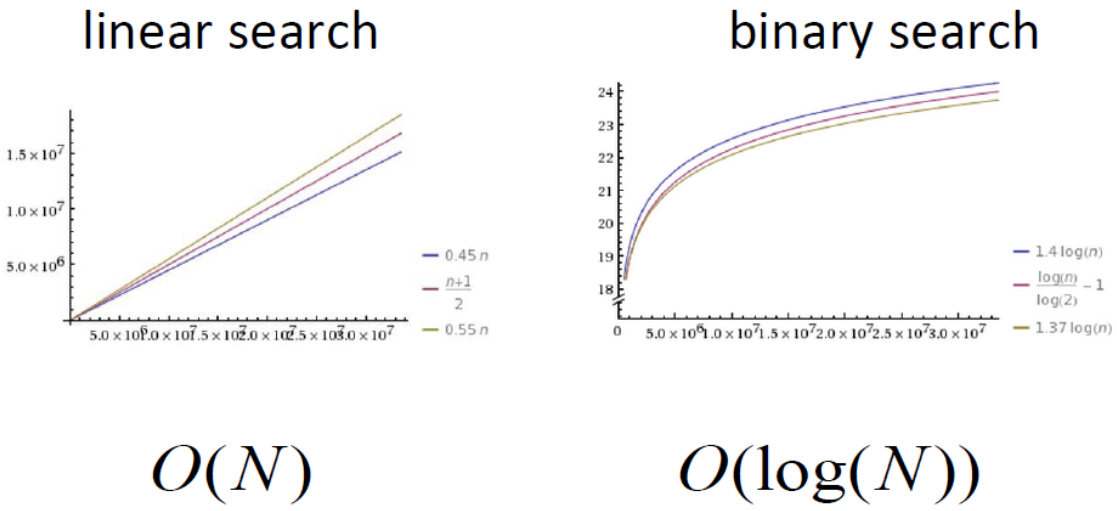


Figure 38: Time complexity of linear and binary search. Note the difference in scaling of the y-axes!

Time complexity overview

Table : complexity classes

Big-O	class	example(s)	assessment
$\mathcal{O}(1)$	constant	Access an element of an array	very good
$\mathcal{O}(log(N))$	logarithmic	binary search	
$\mathcal{O}(\sqrt{N})$	root		
$\mathcal{O}(N)$	linear	linear search	good
$\mathcal{O}(Nlog(N))$	log-linear	Quicksort, Mergesort	
$\mathcal{O}(N^2)$	quadratic	Bubblesort InsertionSort SelectionSort	acceptable
$\mathcal{O}(N^p)$	polynomial	Matrix multiplication (p > 1) (p=3)	
$\mathcal{O}(p^N)$	exponential	compute power set of N elements (p=2)	very bad
$\mathcal{O}(N!)$	factorial	permutations brute-force TSP	catastrophic

CompoundDatatypes

Motivation

Data of a person

Assume we want to store some data of a person

- first name
- last name
- sex
- date ob birth

we could simply define variables

```
String fn = "John", ln = "Doe";
char sex = 'm';
int dobDay = 1, dobMonth = 1, dobYear = 1970;
```

Data of two persons

If we wanted to store data of two persons we would have to create more variables, e.g.

```
String fn2 = "Jane", ln2 = "Doe";
char sex2 = 'w';
int dobDay2 = 31, dobMonth2 = 12, dobYear2 = 1972;
```

- With more persons this would become a mess
- think of arrays of persons (actually 1 array of first names, 1 of lastnames, . . .)
- the data is actually **part of** a person
- we will collect all data of a person and create something like a person record
- we will make a new data type for person, which **HAS-A** firstname and **HAS-A** lastname etc

Simple Classes

Person class

Define a Person class

```
1 class Person{
2     String fn;
3     String ln;
4     char sex;
5     int dobDay, dobMonth, dobYear;
6 }
```

This is a description what data a person is “made of”.



Figure 39: Person class diagram

create a Person

```
Person p = new Person(); //create a person object
p.fn = "John";     //the person object's first name is set
p.ln = "Doe";
p.sex = 'm';
p.dobDay = 1;
p.dobMonth = 1;
p.dobYear = 1970;
```

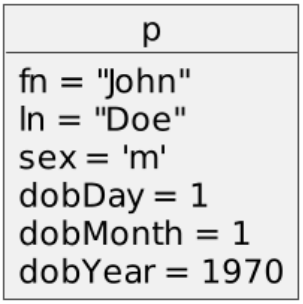


Figure 40: Person object diagram with one Person object p

create a second Person

```
Person p2 = new Person(); //create another person object. Note the different name
p2.fn = "Jane";     //the person object's first name is set
p2.ln = "Doe";
p2.sex = 'w';
p2.dobDay = 31;
p2.dobMonth = 12;
p2.dobYear = 1972;
```

- we have two Person objects
- they have the same attributes but different values in (at least some) of the attributes.

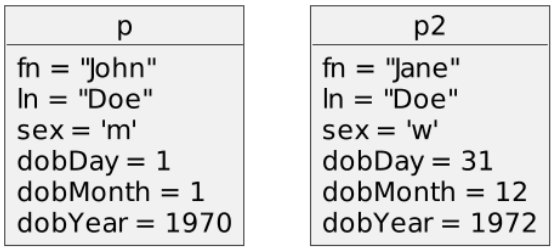


Figure 41: Person object diagram with two Person objects (p, p2)

Summary I

- a class defines a new data type with **attributes**
- each attribute has a **name** and a **type**
- we can create objects of that new data type with the `new` keyword
- the attributes in the new object are initialized with zero (or `null` or `false`)
- we can access the attributes of an object by specifying its object- and attribute name
- with the `objName.attributeName` syntax we can both read and set the value

MiniExercise

- print the Person details
- create and initialize a Person object

Classes and references

Classes are reference types

```
//assume objects p, p2
Person p3 = p;
p3.sex = 'd';

//what is the value of p.sex?
```

- the variable of a Person object is actually just a **handle** to access the object
- `p3` is an additional handle to **the same** object as `p`
- assignment **copies references**

Reference vs. Object I

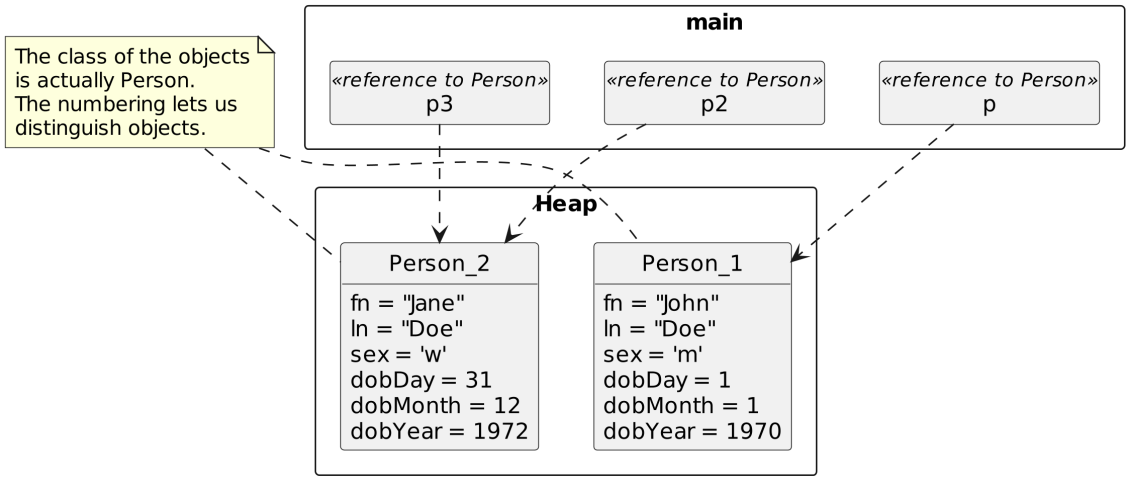


Figure 42: Two `Person` objects on the heap and three references (`p` , `p2` , `p3`) on the stack. Note how both `p2` and `p3` reference the same object.

Reference vs. Object II

- as of now let us distinguish between a reference and the object
- references live on the stack in functions (here in `main`)
- objects live on the heap
- it makes no difference whether we use `p2` or `p3` to access `Person_2`

Deleting reference types

- in Java, reference types are automatically deleted by the **garbage collector**
- when it happens that no more references to an object exist, it is deleted (eventually)
- we can explicitely nullify a reference by assigning it `null`
- `null` represents the non-existing object
- `null` cannot be dereferenced, that means we cannot access any attributes (of the non-existing object)

```
Foo a = new Foo(); a.f = 0;

a = null; //the Foo object formerly known as 'a' can no longer be reached
a.f++; // this causes problems. null.f does not exist
```

Classes and equality

Testing equality

- equality operator tests **reference equality**
- the values of the referenced objects are not checked

```
class Foo { int f;}
```

```
// now in main
Foo a = new Foo(); a.f = 0;
Foo b = new Foo(); b.f = 1;
Foo c = new Foo(); c.f = 1;
Foo d = a;

a == b;  //F
a == c;  //F
a == d;  //T
```

```
b == c;  //F even though the values are the same
b == d;  //F
c == d;  //F
d == d;  //T
```

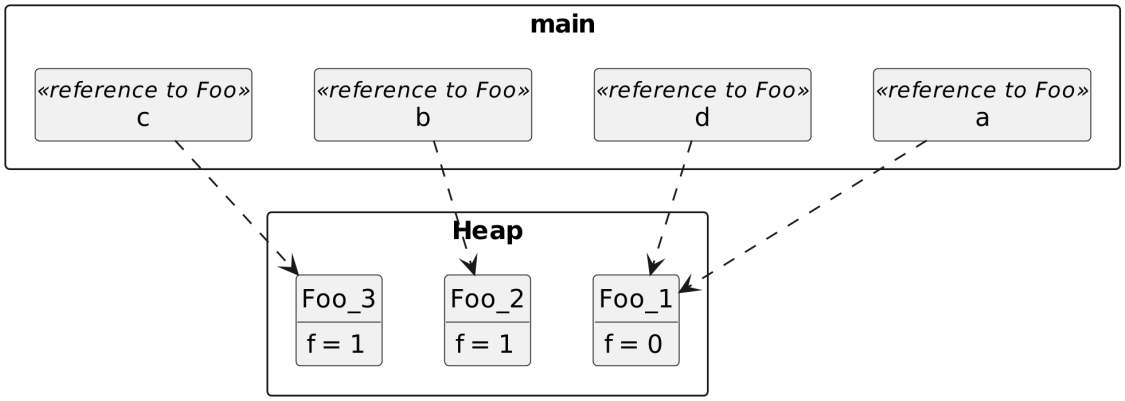


Figure 43: References `a` , `b` , `c` , `d` . Note that `d` is just another handle to ‘Foo_1“.

to test equality of two objects, we have to test equality of all attributes

Classes and functions

Reference types and functions

- when passing a reference type to a function **only the reference is copied**
- therefore, the function has access to the original data

```
public static void bar(Foo p){
    p.f++;
}

public static void main(String[] args){
    Foo a = new Foo(); a.f = 0;

    bar(a);

    //a.f is now 1
}
```

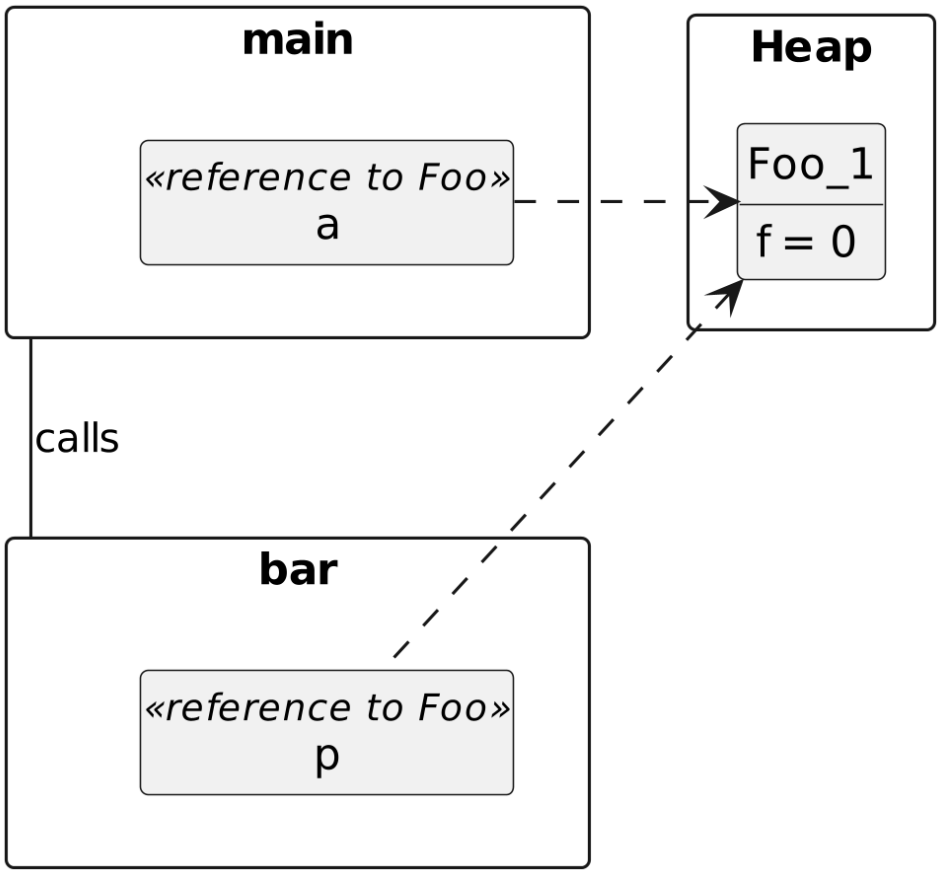


Figure 44: References `a` and `p` reference the same object.

Reference types as return values

- functions can create objects and return references
- the returned reference is a **copy of the original reference**
- when a function ends the objects created inside the function are not deleted, only the local references

```
public static Foo spam(){
    Foo n = new Foo();
    n.f = 42;
    return n;
}

public static void main(String[] args){
    Foo m = spam();
}
```

Classes as attributes

Reference type attributes

- reference types can be attributes of classes
- let us extract a date class

```
class Date{
    int day, month, year;
}

class Person{
    String fn;
    String ln;
```

```
    char sex;
    Date dob;
}
```

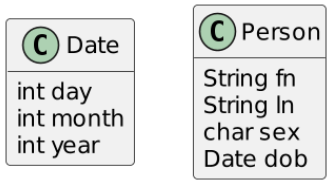


Figure 45: Person and Date class diagram

- Create a person with an attached date object

```
Person p = new Person(); //create a person object
p.fn = "John"
p.ln = "Doe";
p.sex = 'm';
p.dob = new Date();   //create a new date object
p.dob.day = 1;        //note the use of .
p.dob.month = 1;
p.dob.year = 1970;
```

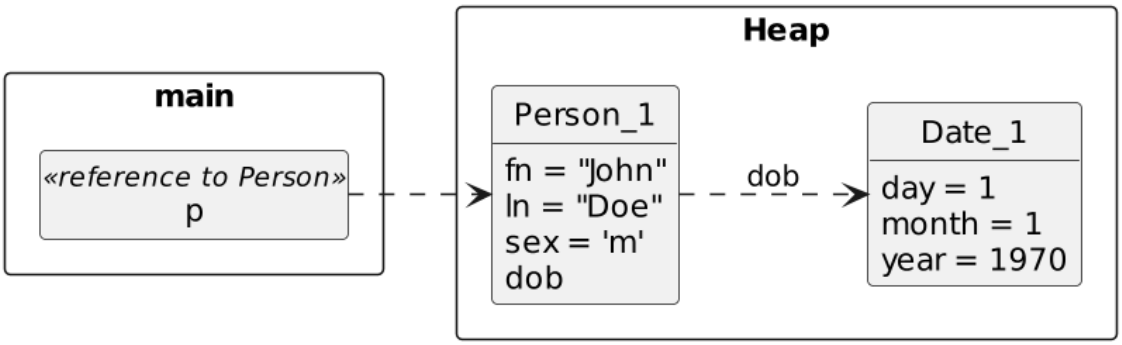


Figure 46: Object diagram. The Person object holds a reference to a Date object.

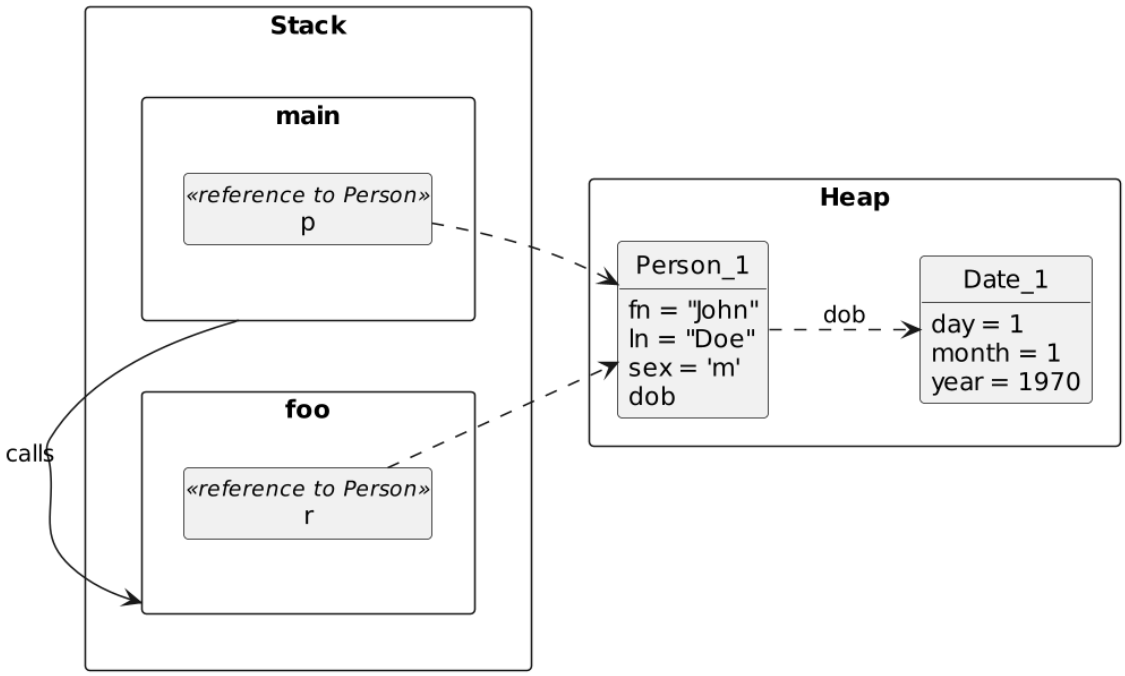


Figure 47: Object diagram. Function call with attached obejct.

Classes and initialization

Initial values for primitive type attributes

- we can provide initial values in the class definition
- the choice of good initial values dpends on the application

```
class Date {
    int day = 1;
    int month = 1;
    int year = 1970;
}
```

```
public static void main(String[] args){
    Date d = new Date(); //d is now 1.1.1979
}
```

Initial values for reference type attributes

```
class Person{
    String fn;
    String ln;
    char sex;
    Date dob = new Date();
}
```

```
public static void main(String[] args){
    Person p = new Person(); //by default the birthday is 1.1.1979
}
```

Classes and Arrays

Arrays of Objects

- we can create arrays of classes

- reference type arrays are initalized with `null`
- note that we have to create the objects seperately

```
Person[] squad = new Person[11]; //an array for 11 persons (all null)

for (int i=0; i<squad.length; i++)
    squad[i] = new Person();  //fill the array with objects

squad[4].dob.y;  //the year in which the 5th person was born
```

Arrays as attributes

- classes can have array type attributes

```
class Exmployee{
    //lots of attributes
    Project[] projects; //the current projects
}
```

- there is nothing special to it - just appply your knowledge of arrays, reference types and classes

Stack and Queue

Stack

Definition

- A **Stack** is a data structure with restricted access
- It supports two main operations
  - **push**: add an element (to the top)
  - **pop**: remove an element (from the top)
- The operations display **last-in first-out** (LIFO) behavior

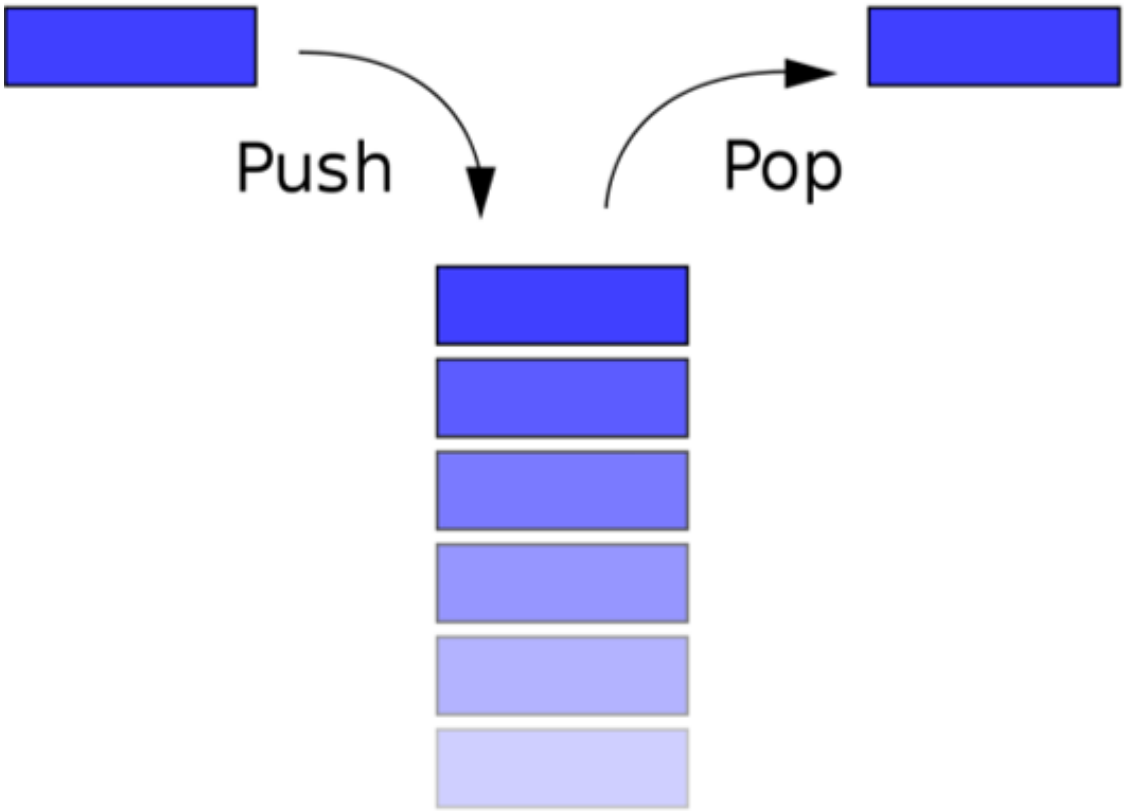


Figure 48: Stack with LIFO mechanism

Typical Stack Operations

- `s.push(a)` adds value `a` as the top element to `s`
- `s.pop()` removes and returns the top element of `s`
- `s.peek()` returns the top element of `s` (the element is *NOT* removed)
- `s.empty()` returns whether `s` is empty

Typical Stack Operations Example

```
1   s1 = ]1|2|3|4|5]       //1 is top element
2   s2 = ]]                //emtpy stack
3
4   s1.push(23)            //s1 = ]23|1|2|3|4|5] , s2 = ]]
5   while( ! s1.empty())
6     s2.push(s1.pop())
7                          //s1 = ]] , s2 = ]5|4|3|2|1|23]
8   s2.push(42)            //s1 = ]] , s2 = ]42|5|4|3|2|1|23]
9   print "s2:" s2.peek()
10  print "s1:" s1.peek()
```

Real World Examples and Applications

- (wet) trays in a self-service restaurant
- browser history
- undo/redo mechanism in many applications
- call stack
- HP-calculators

Evaluating an expressiong using a stack

evaluate `5 * (((9+8) * (4 * 6)) + 7)`

```
push(5)
push(9)
push(8)
push(pop()+pop())
push(4)
push(6)
push(pop()*pop())
push(pop()*pop())
push(7)
push(pop()+pop())
push(pop()*pop())
pop()
```

Stack Exercises

- design (with pen & paper) a browser history solution based on stacks
  - the primary data are URLs
  - when are items pushed?
  - when are items poped?
- include feature to go-back as well as to go-forward in history
  - when are items pushed?
  - when are items poped?
  - when is which stack cleared?
- implement the Tower of Hanoi game with 3 stacks
- study how the evaluation of an expression in *reverse polish notation* can be done with a stack (see also Postfix Exercise)

Queue

Queue Definition

- A **Queue** is a data structure with restricted access
- It supports two main operations
  - **enqueue**: add an element (to the 'end' of the queue)
  - **dequeue**: remove an element (from the 'front' of the queue)
- The operations display **first-in first-out** (FIFO) behavior

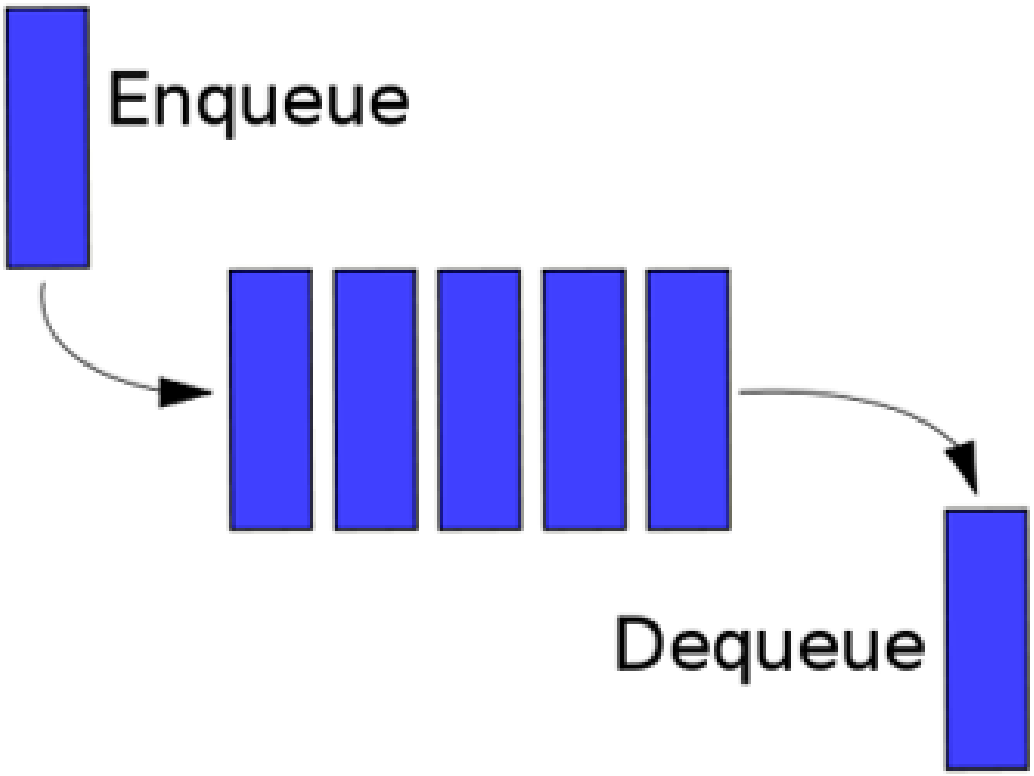


Figure 49: Queue with FIFO mechanism

Typical Queue Operations

- `q.enqueue(a)` adds value `a` to the end of queue `q`
- `q.dequeue()` removes and returns the next element in `q`
- `q.peek()` returns the next element of `q` (the element is *NOT* removed)
- `q.empty()` returns whether `q` is empty

Queue Examples and Applications

- queueing at the bus station/supermarket/...
- printer queue
- refilling supermarket shelves (newest products in the back)

Exercises

- can you reverse a stack with a queue?
- can you reverse a queue with a stack?
- can you implement a queue with two stacks?
- can you implement a stack with two queues?

Implementation

Summary

- Stacks and Queues can be implemented with
  - lists
  - arrays

- we will have a look at the array-based implementations

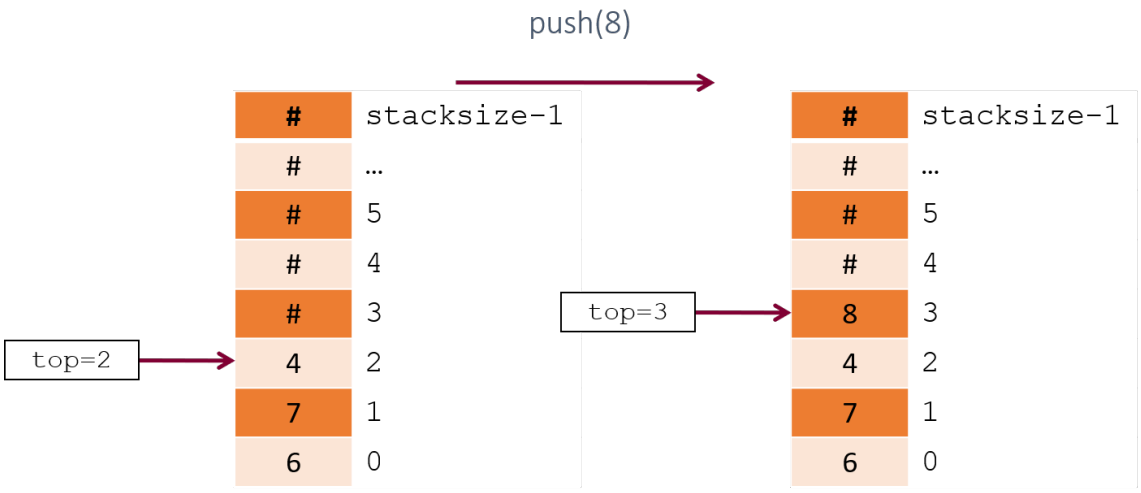


Figure 50: ArrayStack.

ArrayStack

ArrayStack class

```
1  class ArrayStack {
2      int[] data = new int[16]; //this stack has size 16
3      int top = -1;
4  }
```

The stack is backed by an array. `top` is the index of the last entry.

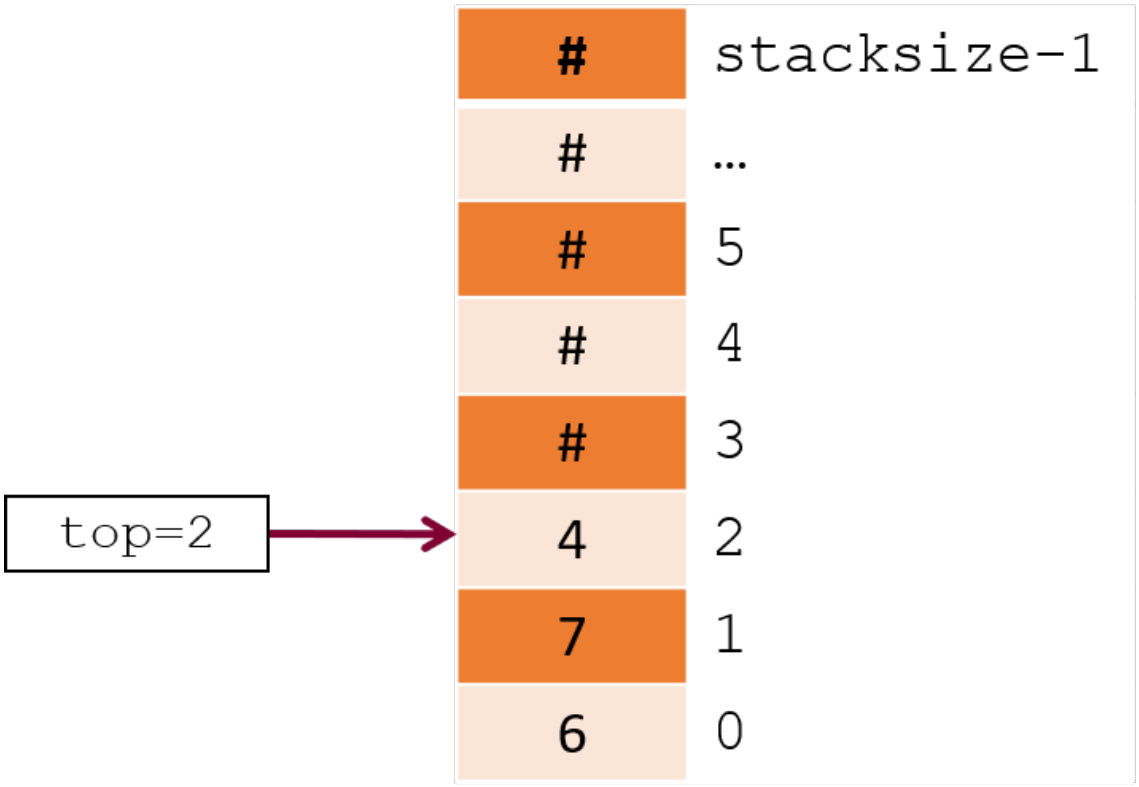


Figure 51: ArrayStack.

Array-based push

```
1      public static void push(ArrayStack s, int data) {
2          if (s.top == s.data.length - 1) {
3              System.out.println("Stack overflow");
4              return;
5          }
6          s.data[++s.top] = data;
7      }
```

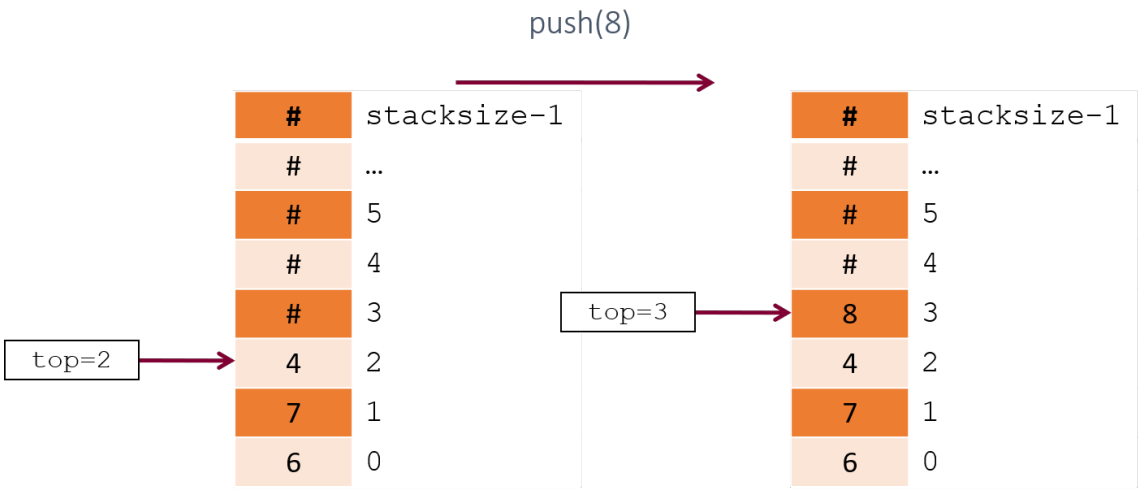


Figure 52: ArrayStack.

Array-based pop

```
1    public static int pop(ArrayStack s) {
2        if (s.top <= -1) {
3            System.out.println("Stack underflow");
4            return Integer.MIN_VALUE;
5        }
6        return s.data[s.top--];
7    }
```

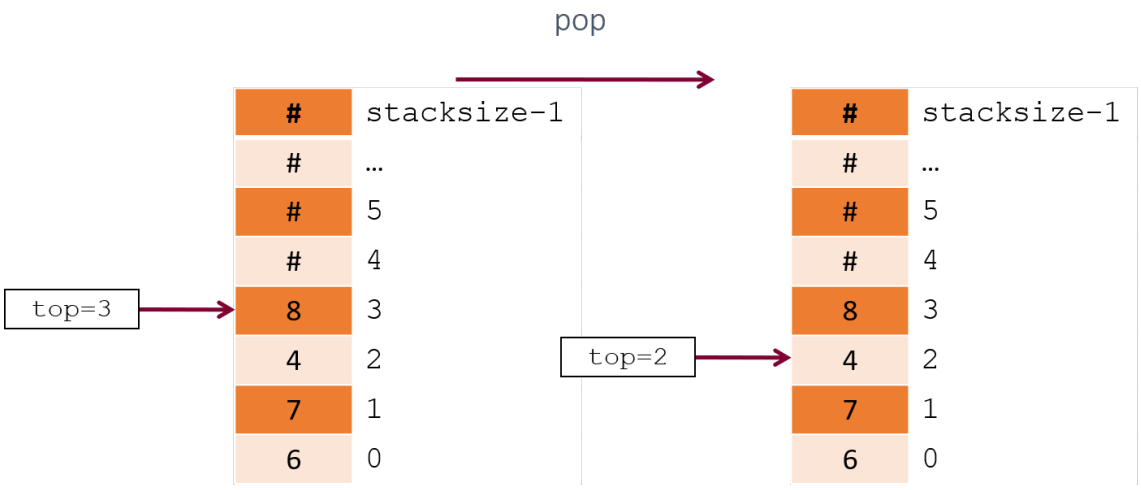


Figure 53: ArrayStack.

ArrayQueue

ArrayQueue class

```
class ArrayQueue{
    int[] data = new int[16]; //initial size
    int front = 0; //idx of front element
    int rear = -1; //index of rear element
    int size = 0;  //nr of elements in the queue
}
```

Array-based enqueue

```
public static void enqueue(ArrayQueue queue, int element) {
    if ( queue.size == queue.data.length) {
        System.out.println("Queue is full. Cannot enqueue element.");
        return;
    }
    queue.rear =
        (queue.rear + 1) % queue.data.length;   //wrap-around
    queue.data[queue.rear] = element;           //insert element
    queue.size++;                               //update size
}
```

Array-based dequeue

```
public static int dequeue(ArrayQueue queue) {
    if ( queue.size == 0) {
        System.out.println("Queue is empty. Cannot dequeue element.");
        return -1; // Returning -1 as a sentinel value
    }
    int element = queue.data[queue.front];                  //get the element
    queue.front = (queue.front + 1) % queue.data.length;    //wrap around
    queue.size--;                                           //update size
    return element;
}
```

Graphs

Motivation

A street map

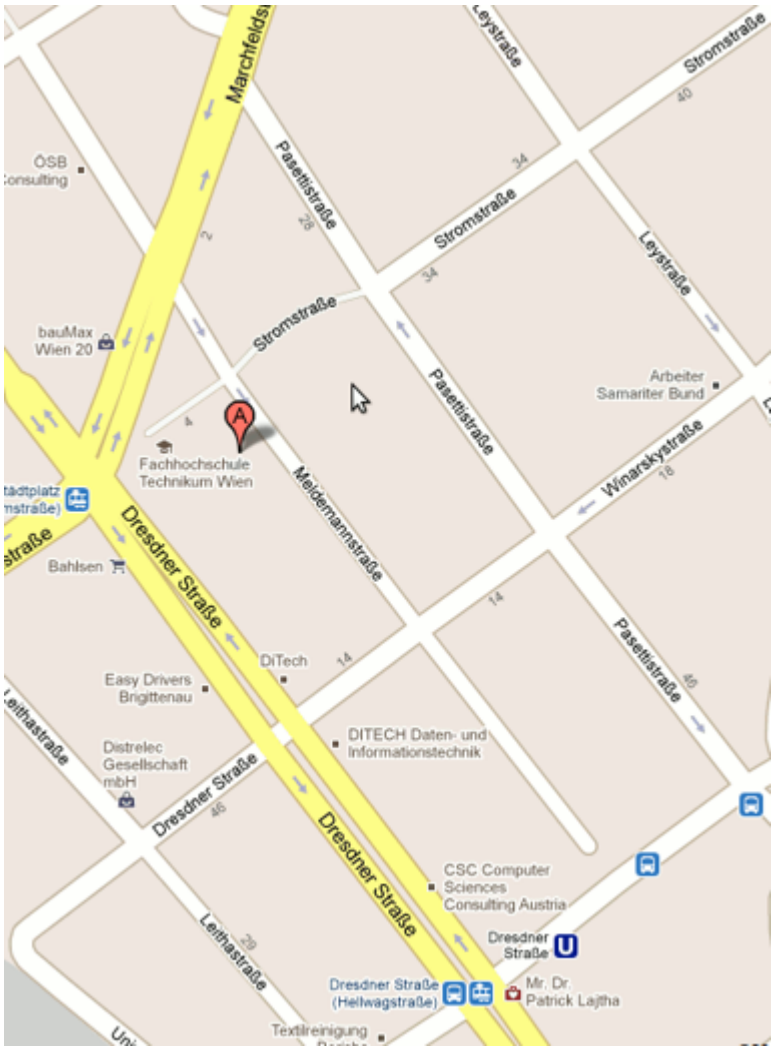


Figure 54: Map of the UAS' area.

The task

Task: find shortest driving route

- START: Marchfeldstraße/Pasettistraße
- END: Hellwagstraße/Pasettistraße
- do consider one-way streets

To do this we have to find a computer friendly representation of the problem

- mark intersctions
- include streets (connections between intersections)
- include length of connections

Intersections and connections

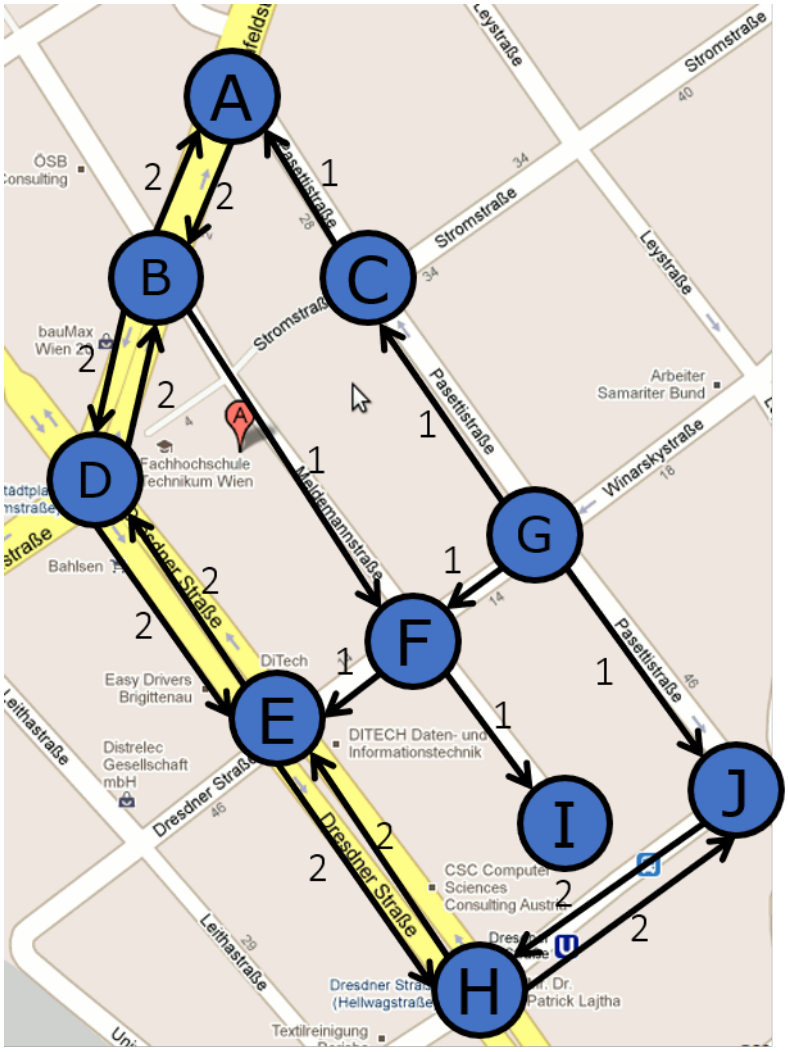


Figure 55: Map with intersections, streets, and lengths marked.

Abstract reality away

- with intersections, connections, and lengths marked we have all the relevant information
- let us free ourselves from reality…

The abstract problem

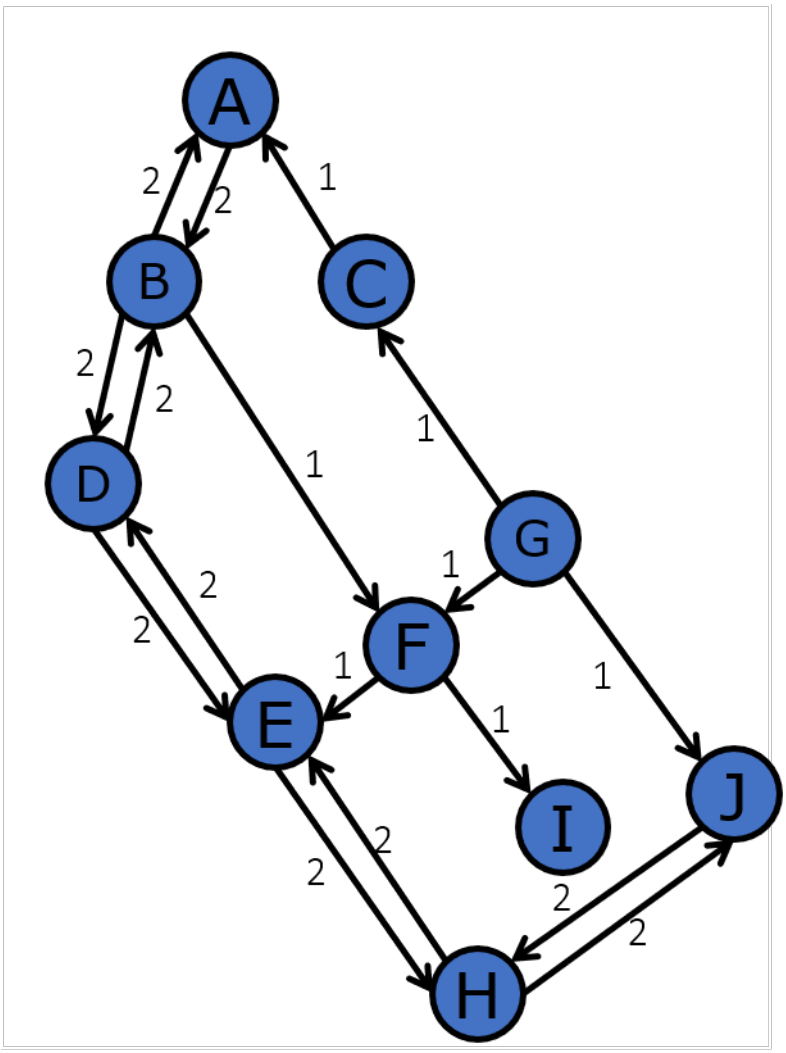


Figure 56: Intersections connections and lengths without map. Now it is a graph

Graph Terms

Definition

- Let
  $V = \{A, B, C, ...\}$
  be the **nodes**
- Let the set of pairs
  $E = \{A \to B, B \to A, C \to A, G \to C, ...\}$
  be the **edges**.
- Call the pair (V, E) a **graph**
- the **weights** are a mapping from E to the real numbers, e.g. $d(A \to B) = 2$

Going back to the original problem

- in graph terms the original problem statement is
  "**Find the shortest path from Node A to node J.**"

- there are several path finding algorithms in graphs

A few more terms

- a graph is **undirected** if edges have no direction, or more precisely if an edge between two nodes X and Y can be called $X \to Y$ or $Y \to X$
- a **path** is a sequence of nodes and edges, e.g
  $A, A \to B, B, B \to D, D, D \to E$
- a graph is **connected** if, for all nodes, there is a path to each node in the graph
- a graph is **acyclic** if there is no path from a node to itself and **cyclic** otherwise

Data structures for graphs

Adjacency list

- an array of lists that hold the (outgoing) edges and weights

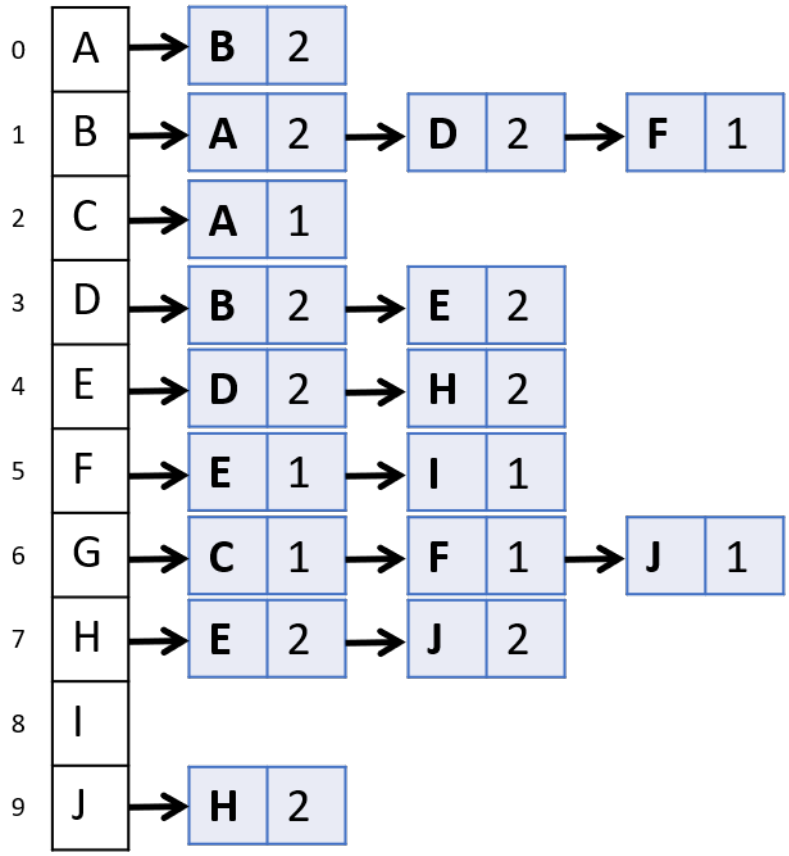


Figure 57: Adjacency list representation of our graph

Adjacency matrix I

- an $N \times N$ matrix (for a graph with N nodes)
- each row and column represents a node, each cell an edge
- the value in a cell is the weight of the edge
- values 1 or 0 for unweighted graphs
- no multiple edges between pairs of nodes possible
- the matrix of an undirected graph is symmetric

Adjacency matrix II

	A	B	C	D	E	F	G	H	I	J
A		2								
B	2			2		1				
C	1									
D		2			2					
E				2				2		
F					1				1	
G			1			1				1
H					2					2
I										
J								2		

Figure 58: Adjacency matrix representation of our graph. Values in the main diagonal are 0 (not shown) all other empty cell are ∞ or `NaN`

Dijkstra's Algorithm

Prerequisites

- Dijkstra's Algorithm finds the shortest path between two nodes in a directed, weighted graph [6]
- All weights must be positive
- The graph can be cyclic

Remark:

For an undirectd graph, substitute each edge with two directed edges

Initialization

- Let $V = \{V_1, V_2, \dots\}$ be a set of nodes, with start node V_a and end node V_e.
- Every node is assigned a distance
  - V_a is assigned 0 (final) ($m(V_a) = 0$)
  - all others are assigned ∞ (tentative)
- Let V_k be the current node: $V_k = V_a$
- Mark all nodes except V_a as unvisited

[6] E. W. Dijkstra (1930-2002)

Main routine

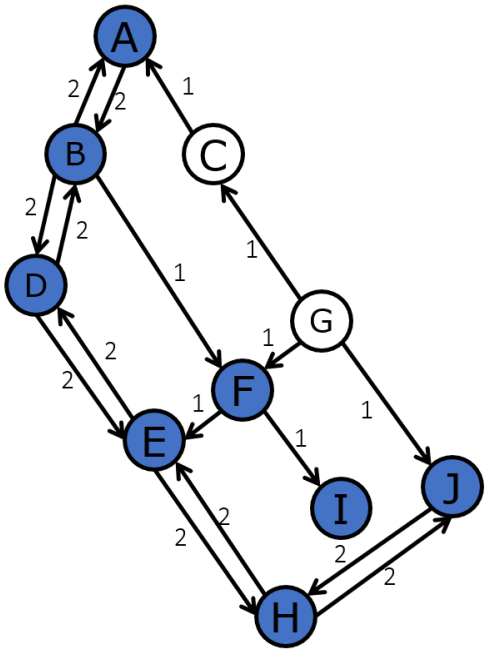
1. Consider **all** neighboring nodes V_i of V_k with a tentative distance and calculate a new distance as such: $m(V_i) = min\{m(V_i), m(V_k) + d(V_k \rightarrow V_i)\}$
2. Out of all V_j with a tentative distance, select the one with **minimal** distance, mark it **final** and make it V_k
3. If $V_k \neq V_e$: go to (1)
   else: stop;

- $m(V_e)$ ist the length of the shortest path from V_a to V_e

Path Reconstruction

- the distance assigned to a node V_i marked as final is the length of the shortest path from V_a to V_i
- The shortest path is reconstructed from end to start by considering the neighboring nodes with consistent (final) distances
- A node V_h is a consistent neighbor of V_i if
  - $m(V_h)$ is final
  - $m(V_h) + d(V_h \rightarrow V_i) = m(V_i)$

Example I

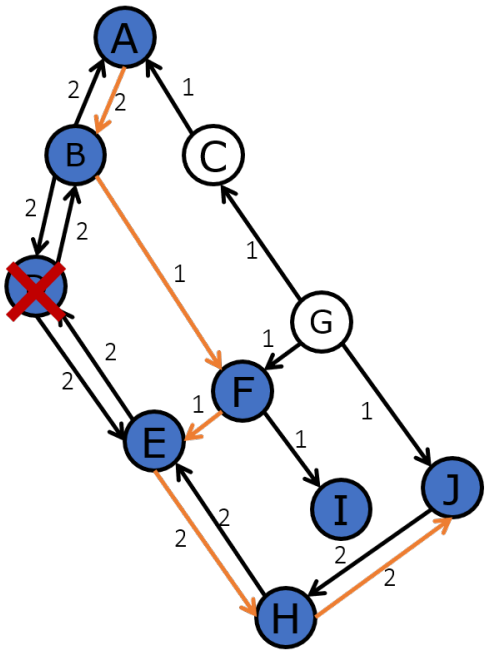


node	final?	distance
A	✓	0
B	✓	~~∞~~ 2
C		∞
D	✓	~~∞~~ 4
E	✓	~~∞~~ 4
F	✓	~~∞~~ 3
G		∞
H	✓	~~∞~~ 6
I	✓	~~∞~~ 4
J	✓	~~∞~~ 8

length of shortest path

Figure 59: After Dijkstra's Algorithm

Example II



node	final?	distance
A	✓	0
B	✓	~~∞~~ 2
C		∞
D	✓	~~∞~~ 4
E	✓	~~∞~~ 4
F	✓	~~∞~~ 3
G		∞
H	✓	~~∞~~ 6
I	✓	~~∞~~ 4
J	✓	~~∞~~ 8

length of shortest path

path: J ←H ←E ←F ←B ←A

Figure 60: Path reconstructed

Proerties and Remarks

- Dijkstra's Algorithm **marks** the nodes, assigning tentative and final values
- Dijkstra's Algorithm is a **greedy** algorithm. It searches local optima but does find a global optimum.
- Several shortest paths might exist, depending on implementation one or all are found

Exercise

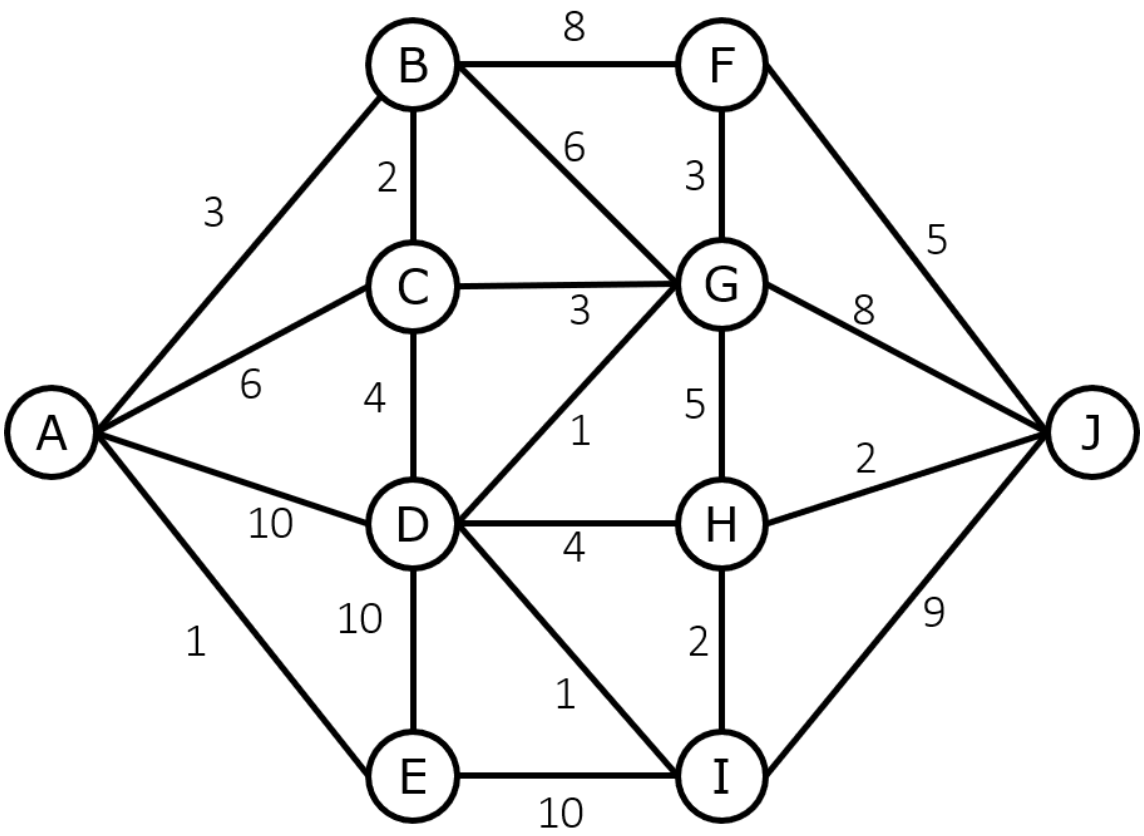


Figure 61: Find **all** shortest paths from A to J.

Implementation

Define the graph

```
static final double oo = Double.POSITIVE_INFINITY;

//...

double[][] cost = { //
        { 0, 3, 6, 10, 1, oo, oo, oo, oo, oo },   // A
        { 3, 0, 2, oo, oo, 8, 6, oo, oo, oo },    // B
        { 6, 2, 0, 4, oo, oo, 3, oo, oo, oo },    // C
        { 10, oo, 4, 0, 10, oo, 1, 4, 1, oo },    // D
        { 1, oo, oo, 10, 0, oo, oo, oo, 10, oo }, // E
        { oo, 8, oo, oo, oo, 0, 3, oo, 5, oo },   // F
        { oo, 6, 3, 1, oo, 3, 0, 5, oo, 8 },      // G
        { oo, oo, oo, 4, oo, oo, 5, 0, 2, 2 },    // H
        { oo, oo, oo, 1, 10, oo, oo, 2, 0, 9 },   // I
        { oo, oo, oo, oo, oo, 5, 8, 2, 9, 0 },    // J
        };
```

dijkstra

```
public static void dijkstra(double cost[][], int[] preced, double[] distance) {
    int[] selected = new int[cost.length];
```

```
    int current = 0, i, k = -1;
    double dc, smalldist, newdist;
       for (i = 0; i < distance.length; i++)  // init distances
        distance[i] = oo;
    // init algo
    selected[current] = 1; // first node is final
    distance[0] = 0; // distance to first node is 0
    current = 0; // selected/current node is first node ("A")
    while (!allselected(selected)) {
        smalldist = oo;
        dc = distance[current]; // cost to current node
        for (i = 0; i < selected.length; i++) {
            if (selected[i] == 0) { // search only non-final nodes
                newdist = dc + cost[current][i];
                if (newdist < distance[i]) {
                    distance[i] = newdist; // shorter path found
                    preced[i] = current; // via current node
                }
                if (distance[i] < smalldist) {
                    smalldist = distance[i]; // search for the next smallest
                    k = i;
                }
            }
        }
        current = k; // select the next smallest
        selected[current] = 1; // mark it final
    }
}
```

helper function

```
public static boolean allselected(int[] selected) {
    int i;
    for (i = 0; i < selected.length; i++)
        if (selected[i] == 0)
            return false;
    return true;
}
```

main

```
public static void main(String[] args) {

        double[][] cost = // see above
        final int MAX = cost.length;

        int i;
        int[] preced = new int[MAX];
        double[] distance = new double[MAX];
        dijkstra(cost, preced, distance);

        System.out.printf("Predecessor:\n");
        for (i = 0; i < MAX; i++)
            System.out.printf("%d\n", preced[i]);
        System.out.printf("\ndistances:\n");
        for (i = 0; i < MAX; i++)
            System.out.printf("%5.1f\n", distance[i]);

        int end = 9;
        int j = end;
        while (j != 0) {
            System.out.printf("%c <- ", j+'A');
            j = preced[j];
        }
        System.out.printf("%c\n", 0+'A');
```

Result

```
Predecessor:
0
0
1
2
```

```
0
1
2
8
3
7

distances:
  0.0
  3.0
  5.0
  9.0
  1.0
 11.0
  8.0
 12.0
 10.0
 14.0
J <- H <- I <- D <- C <- B <- A
```

More graph problems

Critical path

Task: Find the longest path in a graph.

- if elements on the critical path are delayed the project is guaranteed to be delayed.

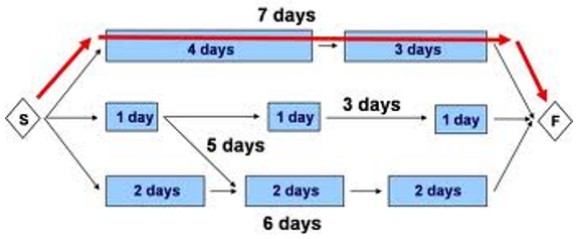


Figure 62: A project as a graph. S is the start, F the finish. Nodes are work packages with a *duration*, edges represent dependencies.

Travelling sales person

Task: Find the shortest round trip



Figure 63: Map with 10 cities the travelling sales person needs to visit.

- a hard problem. Naive approach to calculate all round trips and select the shortest only works for about 10-15 nodes (on a common office PC)
- heuristic algorithms find approximations in reasonable time for thousands of cities
- related problems with great number of applications
  - (vehicle) routing problem
  - fleet management
  - travelling purchaser problem

Even more graph problems

- *small world hypotheses* and the *seven degrees of Kevin Bacon*
- dependencies between financial institutions
- network theory and complex systems
- ...

Rendering graphs

graphviz and the DOT language I

- graphviz is a powerful graphic package for graph rendering
- *DOT* is a language to describe graphs

```
digraph myGraph {
   A -> B
   B -> C
}
```

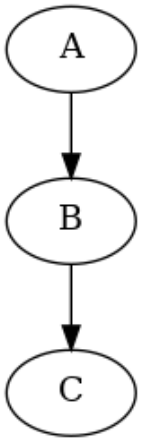


Figure 64: Above digraph rendered with graphviz

graphviz and the DOT language II

- add attributes to style the graph

```
digraph myGraph {
   A -> B
   B [color = red]
   B -> C [color = red, label = "5"]
}
```

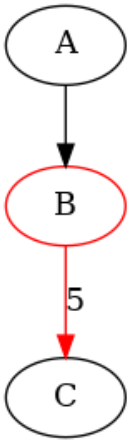


Figure 65: Digraph with one colored edge and one colored node. Visit webgraphviz to experiment

List of Tables

1 Output functions . . . 8
2 Escape sequences for special characters . . . 9
3 Integer formatting examples . . . 10
4 String formating options . . . 11
5 Floating point values formatting examples . . . 12
6 Format string conversions . . . 13
7 Format strings flags . . . 13
8 Java primitive types . . . 17
9 binary numbers . . . 17
10 hexadecimal numbers . . . 17
11 three bit signed integer . . . 18
12 Variable Naming . . . 21
13 Arity of operators . . . 23
14 Java Operator Precedence (highest at the top) and Associativity . . . 24
15 Arithmetic Operators . . . 24
16 Relational Operators . . . 46
17 Conditional Operators . . . 46
18 Truth Table (OR, AND, NOT) . . . 46
19 call count for `fib(8)` . . . 71
20 number of moves to solve Towers of Hanoi with n discs . . . 75

List of Figures

1 HelloWorld Flow Chart. Note the shapes of the start, end, and output nodes. . . . 5
2 HelloWorld Annotated Flow Chart. . . . 5
3 Flowchart ConditionalStatementDemo . . . 32
4 Flowchart ConditionalStatement . . . 34
5 Flowchart SingleAlternative . . . 37
6 Flowchart Consecutive Ifs (Example) . . . 40
7 Flowchart Nested Ifs (Example) . . . 42
8 Flowchart Multiple Alternative (Example) . . . 44
9 Flowchart of input validation with `if` . . . 48
10 Flowchart of input loop. Note the difference to validation with `if` . . . 50
11 Flowchart of the `while` loop . . . 51
12 For Example . . . 53
13 Flowchart of the `for` loop . . . 54
14 Flowchart of input loop with `do/while` . . . 56
15 Flowchart of the `do` loop . . . 57
16 ASCII-Table . . . 63
17 Towers of Hanoi with 8 discs on the middle peg . . . 71
18 Playing Towers of Hanoi with a single disc is trivial . . . 72
19 Playing Towers of Hanoi with two discs. . . . 72
20 Playing Towers of Hanoi with three discs. Overall strategy . . . 73
21 Playing Towers of Hanoi with three discs. Solving the smaller problem . . . 73
22 Array Indexing . . . 78
23 The copy of the reference to an array is passed to a function . . . 81
24 2D array as table . . . 84
25 2D array as one array of eight int arrays . . . 85
26 BubbleSort single run . . . 88
27 BubbleSort complete run . . . 89
28 InsertionSort - insertion of one element . . . 90
29 InsertionSort - complete run . . . 91
30 SelectionSort - sorting one element . . . 92
31 SelectionSort - complete run . . . 93
32 MergeSort - complete run . . . 94
33 QuickSort - sort one element . . . 96
34 QuickSort - sorting elements left and right from pivot element . . . 97
35 QuickSort - complete run . . . 98
36 Example linear search with N=10 unsorted elements . . . 101
37 Example binary search with 10 sorted elements . . . 102
38 Time complexity of linear and binary search. Note the difference in scaling of the y-axes! . . . 103
39 Person class diagram . . . 105
40 Person object diagram with one Person object p . . . 105
41 Person object diagram with two Person objects (p, p2) . . . 106

42 Two `Person` objects on the heap and three references (`p` , `p2` , `p3`) on the stack. Note how both `p2` and `p3` reference the same object. . . . 107

43 References `a` , `b` , `c` , `d` . Note that `d` is just another handle to 'Foo_1". . . . 108

44 References `a` and `p` reference the same object. . . . 109

45 Person and Date class diagram . . . 110

46 Object diagram. The Person object holds a reference to a Date object. . . . 110

47 Object diagram. Function call with attached obejct. . . . 111

48 Stack with LIFO mechanism . . . 113

49 Queue with FIFO mechanism . . . 115

50 ArrayStack. . . . 116

51 ArrayStack. . . . 117

52 ArrayStack. . . . 118

53 ArrayStack. . . . 118

54 Map of the UAS' area. . . . 120

55 Map with intersections, streets, and lengths marked. . . . 121

56 Intersections connections and lengths without map. Now it is a graph . . . 122

57 Adjacency list representation of our graph . . . 123

58 Adjacency matrix representation of our graph. Values in the main diagonal are 0 (not shown) all other empty cell are ∞ or `NaN` . . . 124

59 After Dijkstra's Algorithm . . . 125

60 Path reconstructed . . . 126

61 Find **all** shortest paths from A to J. . . . 127

62 A project as a graph. S is the start, F the finish. Nodes are work packages with a *duration*, edges represent dependencies. 129

63 Map with 10 cities the travelling sales person needs to visit. . . . 130

64 Above digraph rendered with graphviz . . . 131

65 Digraph with one colored edge and one colored node. Visit webgraphviz to experiment . . . 131